

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生

数学建模竞赛

题 目： 面向 Davinci 架构的 NPU 核内调度算法研究

摘 要：

面向通用神经网络处理器（NPU）的核内调度能够显著提升推理效率和能效。传统的人工编排方案任务复杂，难以充分考虑算子拓扑、片上存储和执行单元之间的耦合，容易导致缓存驻留过高和频繁的内存换入换出，从而增加 DDR 流量并降低计算效率。为解决这些问题，本文提出一种基于**华为 Davinci 架构平台**的核内通用调度算法，在不同问题情境下建立了峰值驻留、额外搬运量和总执行时间等目标优化模型。算法综合考虑碎片化、预取距离、模型泛化度等因素，并在 Matmul、FlashAttention 和 Conv 等算子数据上进行实验，综合优化**驻留缓存容量、总额外数据搬运量、总执行时间**指标。

针对问题一，需求解一种驻留最大缓存容量尽可能小的优良调度序列，我们建立**缓存峰值驻留调度求解模型**。首先，对附件所给出的六个示例计算图进行**数据分析**，通过 Gephi 对附件计算图进行**可视化**，同时对数据进行**模块化分析**，为后续所建立模型的通用性和泛化性提供参考。其次，将问题转化为最小化**缓存容量峰值**的**单目标模型**，分别采用**贪心算法、模拟退火算法、禁忌搜索算法、遗传算法**进行求解，发现启发式算法求解本问题需要大量迭代与参数调节。最后选取**贪心算法**得出的最小缓存驻留容量作为本题求解结果，示例 Conv0、Conv1、FlashAttention0、FlashAttention1、Matmul0、Matmul1 的结果分别为 **14664、14048、5140、8228、9728、35328**。

针对问题二，需再考虑**总额外数据搬运量**指标，为调度顺序中各缓冲区分配物理地址偏移，我们建立**总额外数据搬运量优化模型**。问题二分为以下两个子问题：**基于调度序列的单目标优化问题**和**双指标融合的单目标优化问题**。子问题一要求在问题一求得的调度序列 S 基础上，额外考虑数据缓存区的地址偏移与 SPILL 操作列表，设计一个总额外数据搬运量指标尽可能小的缓存分配方案。我们建立**基于调度序列的单目标优化模型**，使用**遗传编程算法**进行求解。子问题二考虑问题一在调度序列与指标最小化目标难以兼顾情况下，所需设计的优化调整方案。本文从模型泛化度的角度考虑，建立双指标融合的加权单目标模型，采用**变邻域模拟退火算法**进行求解。最后，通过两个子问题结果的对比发现，**基于调度序列的单目标优化模型**所取得的结果更优，示例 Conv0、Conv1、FlashAttention0、FlashAttention1、Matmul0、Matmul1 的总额外数据搬运量分别为 **230496、840904、4096、33792、41600、518400**。

针对问题三，需额外考虑**算子总运行时间**指标，来实现其与总额外数据搬运量指标的优化，我们建立**综合性能优化策略设计模型**。问题三需求解以下两个阶段的问题：**算子运行时间约束下的加权指标优化问题**和**多目标性能优化问题**。其中，阶段一考虑仅优化总运行时间指标，我们设定总额外数据搬运量最多**增加 10%**，寻找在调度顺序不变的情况下，能够优化缓存分配的方案。我们使用 **GP 改进算法**求解得到使两项指标加权值

最低的方案。子问题二考虑联合优化两项指标的情况下，求解**优化调度顺序、缓存分配**的方案。我们选用 **NAGA-II 算法、多粒子群优化算法**，求得所有多目标帕累托最优解，再通过**指标提升程度**评价性能的改进程度得出最优解。最后对两个阶段得出的方案进行比较，得到本题最佳结果。在求得的最佳结果中，以示例 **Conv0** 为例，**算子总运行时间**和**总额外数据搬运量**指标数据的提升效果分别为 **8.43%、14.73%**；以示例 **Conv1** 为例，**算子总运行时间**和**总额外数据搬运量**指标数据的提升效果分别为 **4.29%、4.71%**。

本研究针对华为 Davinci 架构，系统地构建了面向峰值驻留、数据搬运量及总执行时间的目标优化模型，并设计了相应的求解算法。通过在不同类型算子的计算图上进行实验，验证了所提模型与算法在不同问题情境下的**有效性**。结果表明，该系列模型能够通过调整优化目标和约束条件，灵活适应核内调度过程中的不同侧重点与性能需求，具有较好的**通用性和泛化性**。本文所提出的建模思路与求解框架，为 NPU 核内通用调度问题的研究提供了具有良好泛化性的解决方案。

关键词： SIMD 架构，核内调度，启发式算法，遗传编程，变邻域模拟退火算法，NAGA-II，多目标粒子群优化

目录

一、 前言.....	5
1.1 问题背景.....	5
1.2 问题研究现状.....	6
1.3 问题重述.....	6
1.4 数据介绍.....	7
1.5 问题求解框架图.....	8
二、 模型假设与符号说明.....	9
2.1 模型基本假设.....	9
2.2 符号说明.....	9
三、 问题一：最小缓存驻留调度问题求解.....	10
3.1 问题分析及求解思路图.....	11
3.2 数据分析.....	12
3.2.1 数据集选择.....	12
3.2.2 数据探索及可视化分析.....	12
3.3 核心方法的适应性分析.....	15
3.3.1 贪心算法的适应性分析.....	15
3.3.2 启发式算法的适应性分析.....	16
3.4 基于多启发式协同优化算法体系与建模.....	18
3.4.1 单目标规划模型.....	18
3.4.2 贪心算法模型.....	18
3.4.3 模拟退火算法模型.....	19
3.4.4 禁忌搜索算法模型.....	20
3.4.5 遗传算法模型.....	21
3.5 多视角下不同启发式算法的求解分析.....	22
3.5.1 贪心算法模型求解结果.....	22
3.5.2 模拟退火算法模型求解结果.....	23
3.5.3 禁忌搜索算法模型求解结果.....	24
3.5.4 遗传算法模型求解结果.....	25
3.5.5 求解结果分析.....	26
3.5.6 求解结果总结.....	27
3.6 算法时间复杂度分析.....	27
3.7 本章小结.....	29
四、 问题二：缓存分配方案单目标设计问题求解.....	30
4.1 问题分析与求解思路.....	31
4.2 核心方法的适用性分析.....	31

4.2.1 遗传编程算法的适用性分析	31
4.2.2 变邻域模拟退火算法的适用性分析	32
4.3 子问题一：基于调度序列的单目标优化模型	32
4.4 子问题二：双指标融合的加权单目标优化模型	35
4.4.1 指标定义	35
4.4.2 归一化与权重设置	35
4.4.3 变邻域模拟退火算法模型的建立	35
4.5 基于演化计算与局部搜索的模型求解	38
4.5.1 子问题一：基于调度序列的单目标优化模型求解	38
4.5.2 子问题二：双指标融合的加权单目标优化模型求解	41
4.5.3 求解结果总结	41
4.6 本章小结	42
五、 问题三：综合性能优化策略设计问题求解.....	43
5.1 问题分析与求解思路	44
5.2 策略优化核心方法及适用性分析	44
5.2.1 GP 改进算法	44
5.2.2 NAGA-II 算法	44
5.2.3 多目标粒子群优化算法	47
5.3 基于多阶段双目标的协同优化模型构建	47
5.3.1 阶段一：总额外数据搬运量约束下的加权指标优化模型	47
5.3.2 阶段二：多目标帕累托前沿性能优化模型	48
5.4 协调优化模型求解	49
5.4.1 总额外数据搬运量约束下的加权指标优化模型求解	49
5.4.2 多目标帕累托前沿性能优化模型求解	51
5.5 本章小结	54
六、 主要结论与模型评价.....	55
6.1 模型的优点	55
6.2 模型的不足	55
6.3 模型的推广	55
参考文献	56
附录 A 人工智能产品辅助答题规范说明	58
附录 B 关键代码展示	59

一、前言

1.1 问题背景

为面向通用 NPU 的核内调度研究，首先需要厘清关键技术与发展脉络。以 SIMD 为抽象，本题以华为 Davinci 架构平台来进行开展讨论，计算核心由多类执行单元与多级片上存储共同构成：计算单元包括专用于矩阵乘的 Cube 与通用向量计算的 Vector，数据搬运单元包括 MTE1/2/3 与 FIXP；各单元拥有独立队列，由统一调度核心分发指令协同工作。片上存储呈分层结构：紧邻 Cube 的 L0A/L0B/L0C（分别承载左右操作数与结果）、容量更大的 L1，以及面向 Vector 的统一缓存 UB；MTE 支持在 L1 与 UB 间交换以适配不同算子交替执行。为适配硬件，推理算子被进一步拆解为在特定单元上执行的原子操作，连同 ALLOC/FREE 等缓存管理指令，构成带依赖约束的细粒度计算图（DAG），调度任务的首要目标是在满足依赖的前提下给出有效拓扑序列，驱动多单元协同执行。

围绕这一硬件——软件抽象，算子级优化逐步演进：以 Matmul 为例，典型分块计算由 COPY_IN(DDR→L1)、MOVE(L1→L0A/L0B)、MMAD(Cube 计算)、COPY_OUT(写回 DDR) 四阶段构成；通过在分块间交错这些阶段可形成流水并行，但不同分块访问顺序会显著影响重复载入与 DDR 流量，例如“之字形”顺序即可降低对某一矩阵块的重复换入次数。进一步地，FlashAttention 通过分块与在线 softmax 将中间结果留在高速缓存中，成为注意力加速的关键；其子结构同时依赖 Cube 与 Vector，需要在二者之间构建分阶段交替的流水以提高利用率。卷积网络的多层依赖则带来深/广两类调度策略选择，不同策略会改变各层间的驻留重叠与复用。

然而，随着图规模与异构度上升，现有流程暴露出瓶颈：其一，片上存储容量与连续地址约束使缓冲区生命周期大量重叠，易触发空间紧张与碎片化，需要在调度中显式为每个缓冲区分配连续区间并处理地址复用依赖；当陷入“资源死锁”时必须引入 SPILL（换出/换入）以暂存 DDR，从而换取可行性。其二，SPILL 会带来额外数据搬运量与时延，且代价与是否被 COPY_IN 使用有关，如何选择目标缓冲区与时机直接影响系统能效与性能。其三，调度目标相互牵制：一方面需寻找内存友好的拓扑序以降低峰值驻留并尽量避免 SPILL，另一方面又要在资源独占与依赖约束下实现多单元并行以缩短总执行时间，二者往往难以同时最优。

据此提出一套通用的**面向华为 Davinci 架构平台的核内调度算法**。首先，进行**内存感知的拓扑排序**：在就绪集合中优先调度可立即释放大体量缓冲区的 FREE，其次选择能解锁后续 FREE 的计算节点，最后才进行 ALLOC（小块优先）；结合关键路径权重，获得兼顾驻留与并行潜力的顺序，以降低峰值驻留并为后续容量约束创造空间。随后，基于该顺序构建**多级缓存地址分配与 SPILL 策略**：将 L1/UB/L0* 视为可合并空洞的连续区间，采用最佳/首次适配分配地址；当需要连续空间而短缺时，以“下次使用最远+代价权重”的 Belady 类抉择选择 SPILL 目标，并在下一次使用前进行 Just-in-Time 的 SPILL_IN，必要时重映射以抑制碎片。最后，在插入 SPILL 与地址复用依赖后，执行**异构列表调度与软流水**：按资源独占与执行依赖约束，结合关键路径优先和预取距离控制，使 Cube/Vector/MTE/FIXP 阶段交错以最大化重叠；若需要，可在不改变 SPILL 次数的窗口内做小规模顺序交换以进一步压缩完工时间。上述流程与赛题提供的 Matmul、FlashAttention 与 Conv 示例相匹配，可在不针对特定结构“过拟合”的前提下验证算法在不同算子形态上的普适性。

1.2 问题研究现状

在深度学习的快速发展中，神经网络处理器（NPU）作为关键的硬件加速器，其性能和能效受到越来越多的关注。特别是在边缘计算领域，基于单指令多数据流（SIMD）架构的 NPU 因其设计简洁和高计算效率，成为主流选择。然而，如何有效地利用这一类硬件加速器，其软件模型的适配性已经成为一个关键瓶颈。随着复杂神经网络的出现，NPU 的硬件资源配置和调度问题愈发突出，亟待解决。

调度优化研究的演进可以追溯到早期宏观层面的任务分配问题。在传统的调度领域，很多研究集中于如何进行资源分配，尤其是在复杂约束下生成无冲突的排班表。例如，国际空间站的宇航员排班和放射肿瘤科的员工调度，都采用了基于约束的算法，确保任务的有效分配^{[1][2]}。然而，随着计算技术的进步，调度研究的重心逐渐从宏观层面的任务调度转向了微观层面的硬件资源调度。对于 CPU 而言，单一的 SIMD 指令虽然可以提高并行度，但端口争用和资源竞争可能导致性能下降^[3]，这表明硬件资源的限制直接影响着调度的效率。

随着大型语言模型等复杂 AI 任务的出现，单一的硬件加速器已无法满足端到端的推理需求，异构硬件协同调度因此成为研究的热点。例如，IANUS 系统将 NPU 与存内计算（PIM）结合，以同时发挥两者的优势^[4]。其中的核心挑战在于如何在多个计算单元之间协调任务，同时最小化数据在内存中的移动，这要求在统一的内存系统和 PIM 访问调度之间找到一种高效的协同机制。这类研究强调，现代硬件中高效的计算调度必须与数据流优化相结合，因为数据搬运的开销已成为性能的主要瓶颈。

与此类似，一些研究专注于通过数据流调度技术来降低内存访问开销，特别是在卷积神经网络中，采用层分组（layer grouping）和分块（tiling）策略来最大化片上缓存的复用，从而有效减少片外内存的访问开销^[5]。此外，内存计算（IMC）的逻辑编译器也需要同时考虑逻辑综合和调度，以生成高效的指令序列，优化能效和时延^[6]。这些研究为现代硬件的高效调度提供了宝贵的经验和方法。

与上述挑战类似，针对基于 SIMD 架构的通用 NPU 核心，本文提出的调度问题正是这些挑战的缩影。综上所述，本文通过解决计算与内存协同的独特挑战，将宏观调度中的自动化思想应用于微观的硬件层面，旨在为通用 NPU 的调度瓶颈提供一种高效且可扩展的解决方案。这不仅能推动深度学习加速器的应用，还能为未来硬件设计与调度优化提供理论和实践指导。

1.3 问题重述

基于提供的背景和任务，本题的核心是为通用神经网络处理器（NPU）设计一套高效的核内调度算法。该算法旨在将神经网络计算图中的原子操作，在满足严格依赖关系的前提下，优化地调度到 NPU 的异构硬件单元上，并妥善管理有限的多级缓存资源。具体任务可分解为以下三个相互关联且层层递进的子问题：

问题一：最小缓存驻留调度问题

基于给定的计算图，问题一的核心任务是设计一个调度算法，通过合理的拓扑排序，使得执行过程中所需的最大缓存容量尽可能小。在没有硬件容量限制的前提下，我们需要尽量减少内存峰值，以防止缓存溢出。这与数据流的优化方向相符：通过合适的计算图调度顺序，优化缓存的使用，从而降低内存占用峰值。为此，调度算法需要依赖缓存的生命周期与节点间的依赖关系，以实现内存的高效复用和合理分配，从而保证调度过程中的内存压力最小。

问题二：缓存分配方案单目标设计问题

问题二的目标是为计算图中的每个缓冲区分配具体的内存地址，并在有限的硬件缓存容量下，通过合适的 SPILL 策略优化数据搬运量。在硬件缓存容量有限的情况下，如何合理分配内存地址、避免缓冲区重叠并减少换入换出的数据量，是影响性能的关键。缓存分配策略不仅要考虑内存的物理容量限制，还要考虑缓存的连续地址分配，以避免碎片化导致的额外开销。同时，SPILL 机制的引入使得调度算法在内存不足时能够及时释放缓存空间，并在需要时将数据重新加载，从而确保计算过程不受内存限制的影响。

问题三：综合性能优化策略设计问题

问题三关注的是如何在控制内存搬运开销的同时，进一步优化计算过程的执行时间，缩短总的完工时间。通过对计算任务的调度顺序和内存管理策略的优化，目标是使得计算单元之间的并行性得以最大化，从而减少各执行单元的等待时间，并通过流水化调度提高整个计算图的执行效率。同时，调度算法还需考虑内存与计算的协同，通过多单元调度和软流水优化，减少任务间的空闲等待，进一步提升硬件资源的利用率。

总结：

本文提出的三个问题分别聚焦于优化调度顺序、合理分配缓存地址和提高计算效率。解决这些问题的关键在于设计高效的调度算法，以最小化内存峰值和搬运开销，同时优化计算时间。本文将通过针对不同算子的实验验证，确保提出的算法能够在实际的 NPU 硬件平台上有效应用，并为大规模的神经网络推理任务提供通用且高效的调度方案。

1.4 数据介绍

本题提供六组典型算子计算图数据，涵盖矩阵乘法（Matmul）、注意力机制（FlashAttention）与卷积网络（Conv），每组由节点与边构成一张异构的有向无环图（DAG），用于系统评估核内调度与片上内存管理算法的有效性与可扩展性。数据在规模上覆盖中等到大型计算图：Matmul 的 Case0/Case1 分别包含 4,160/30,976 个节点与 7,104/55,040 条依赖；FlashAttention 的 Case0/Case1 分别为 1,716/6,952 个节点与 2,712/11,184 条依赖；Conv 的 Case0/Case1 分别为 2,580/36,086 个节点与 3,869/85,653 条依赖。其中 Conv_Case1 的边一点密度最高，适合用于检验算法在高耦合场景下的扩展性与鲁棒性。

从异构性与负载构成看，FlashAttention 的操作类型最为丰富，而 Matmul 相对精简；按执行单元与周期统计，Matmul 以 CUBE 为主、同时伴随大量 MTE 操作；FlashAttention 在 VECTOR 与 MTE 上的负载占比较高，呈现出计算—搬运交错的流水形态；Conv 则表现出更强的数据搬运主导性，提示其瓶颈更多来自内存通道而非算力本身。与此同时，三类任务在 L1 与 L0*上的 ALLOC 需求量大且生命周期显著重叠，意味着片上连续地址空间紧张、峰值驻留压力较高，易触发换入换出（SPILL）并带来额外的外存访问开销。

综上，这一数据集能够同时支撑三类评测目标：其一，用于检验在不设容量上限条件下的“内存友好拓扑序”是否能够有效降低峰值驻留（问题一）；其二，用于评估在真实容量约束下的连续地址分配与 SPILL 策略对外存数据搬运量的影响（问题二）；其三，用于量化多执行单元并行与软流水优化对完工时间的改善幅度（问题三）。

1.5 问题求解框架图

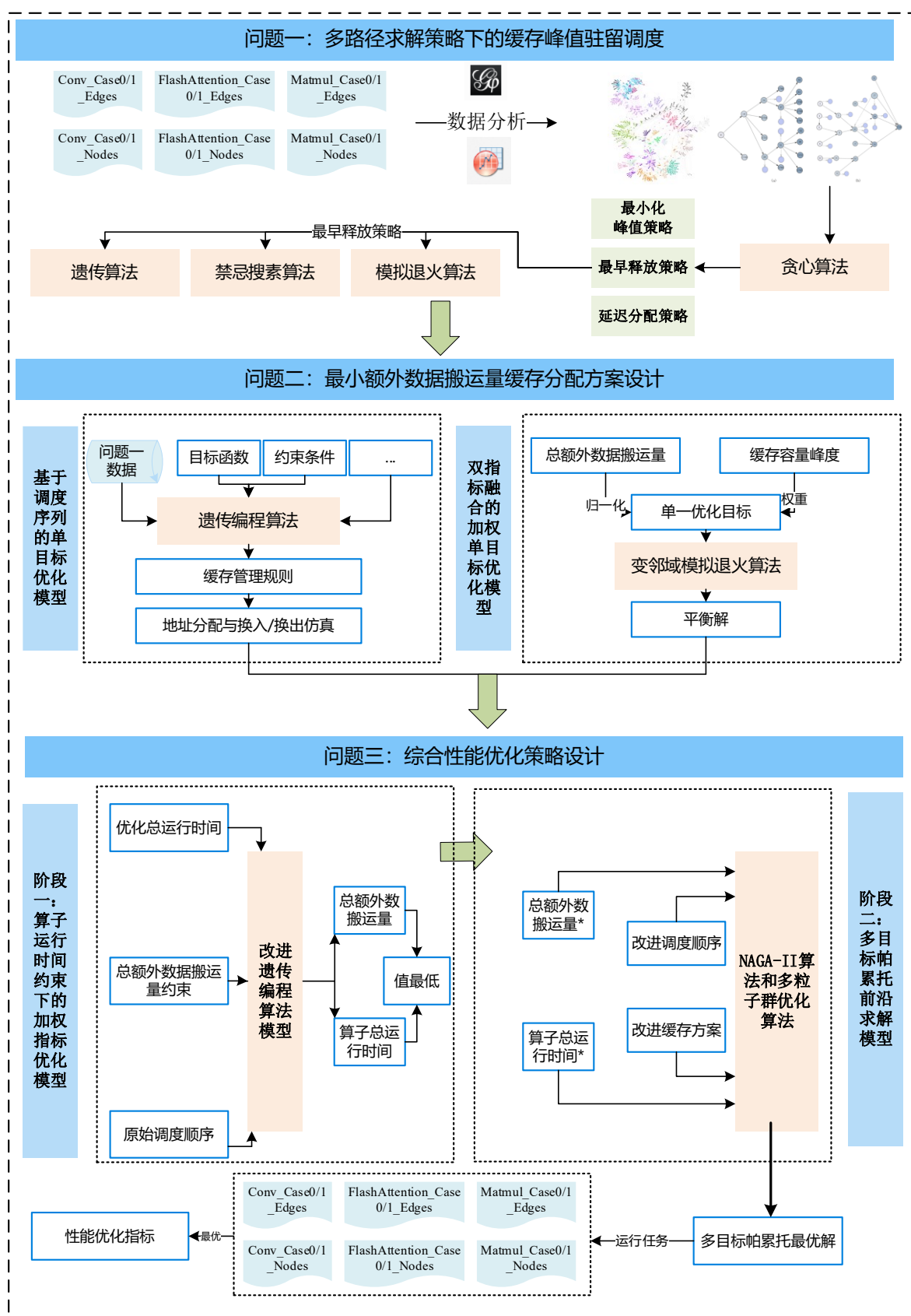


图 1 总体求解框架图

二、模型假设与符号说明

2.1 模型基本假设

(1) 假设 NPU 的计算单元缓存层级、容量等硬件资源是固定在，在整个调度过程中其性能参数保持不变。

(2) 假设每个计算节点和管理节点的执行周期数是固定的。

(3) 假设 SPILL 操作的耗时严格遵循线性模型且准确反映了数据搬运的物理延迟。

(4) 假设由算子分解得到的细粒度计算图是准确完整的，计算图中的所有节点及依赖关系在调度前完全确定。

(5) 假设计算图中节点间的数据依赖关系是严格的，起始节点的开始时间必须晚于其所有前驱节点的结束时间。

(6) 假设片上缓存被抽象为一个连续的逻辑地址空间，缓存分配方案不会引发未预期的地址冲突或碎片化问题。

(7) 假设本文所优化的核心指标可以根据上述步骤进行精确计算，且能够充分反映调度方案的性能。

2.2 符号说明

表1 符号说明

符号	说明	符号	含义
$O(N)$	时间复杂度	$t \setminus *$	峰值出现的步位置
vi	第 i 个节点	R	就绪集
$M(vi)$	节点 vi 的缓存需求	$Cost(S)$	代价
$Op(v)$	节点类型	$Fit(S)$	适应度
$Size(v)$	字节	$\Pi(S)$	违约度
$BufId(v)$	缓冲区标识	λ	软惩罚系数
$indeg(v)$	入度	p_c	交叉率
$S = (V_1, \dots, V_N)$	调度序列	p_m	变异率
$pos(v)$	v 在序列中的位置	T	禁忌表
$\Delta(v_t)$	单步增量	$Size(v_i)$	节点大小
$Iter_{max}$	最大迭代次数	gen_{max}	最大退火次数

三、问题一：最小缓存驻留调度问题求解

本题的具体求解思路见图 2。

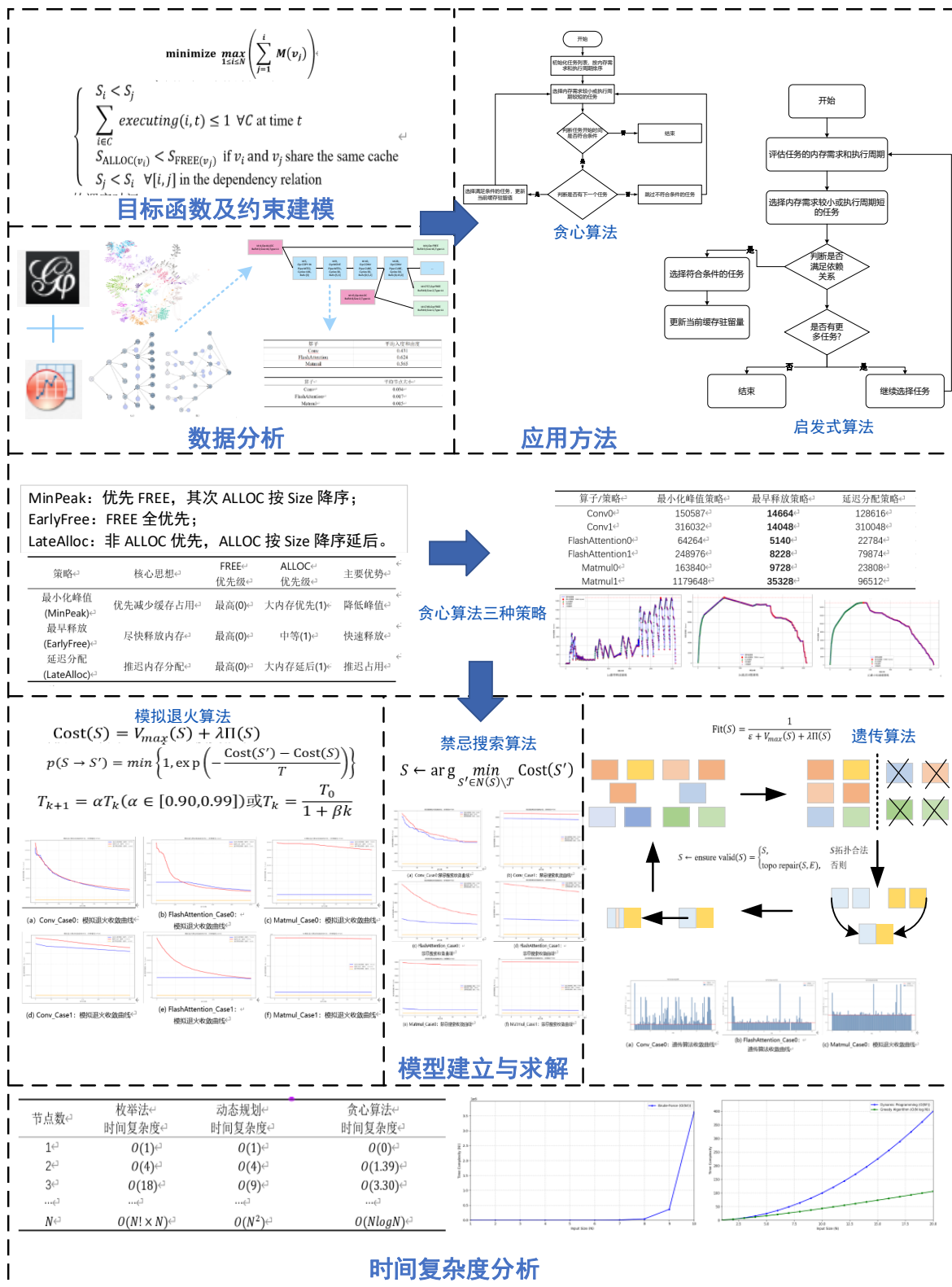


图 2 多路径求解策略下的缓存峰值驻留调度思路流程图

3.1 问题分析及求解思路图

问题一旨在对附件给出的 3 种算子 6 个任务进行拓扑排序，使得在满足所有节点依赖关系的同时，执行过程中所需的**最大内存驻留量最小化**。从算法的泛化性与通用性角度出发，我们先对 6 个任务的计算图进行数据分析，以掌握不同算子的数据特性。为便于说明缓存驻留峰值的来源，我们先给出一个小规模示例，如图 3 所示。图中横轴为调度顺序，纵轴为缓冲区的生命周期，可以看到不同缓冲区的存活区间存在重叠，从而在某些时刻叠加成驻留峰值。这就是本文问题一优化目标的核心——寻找使最大驻留量最小的调度顺序。

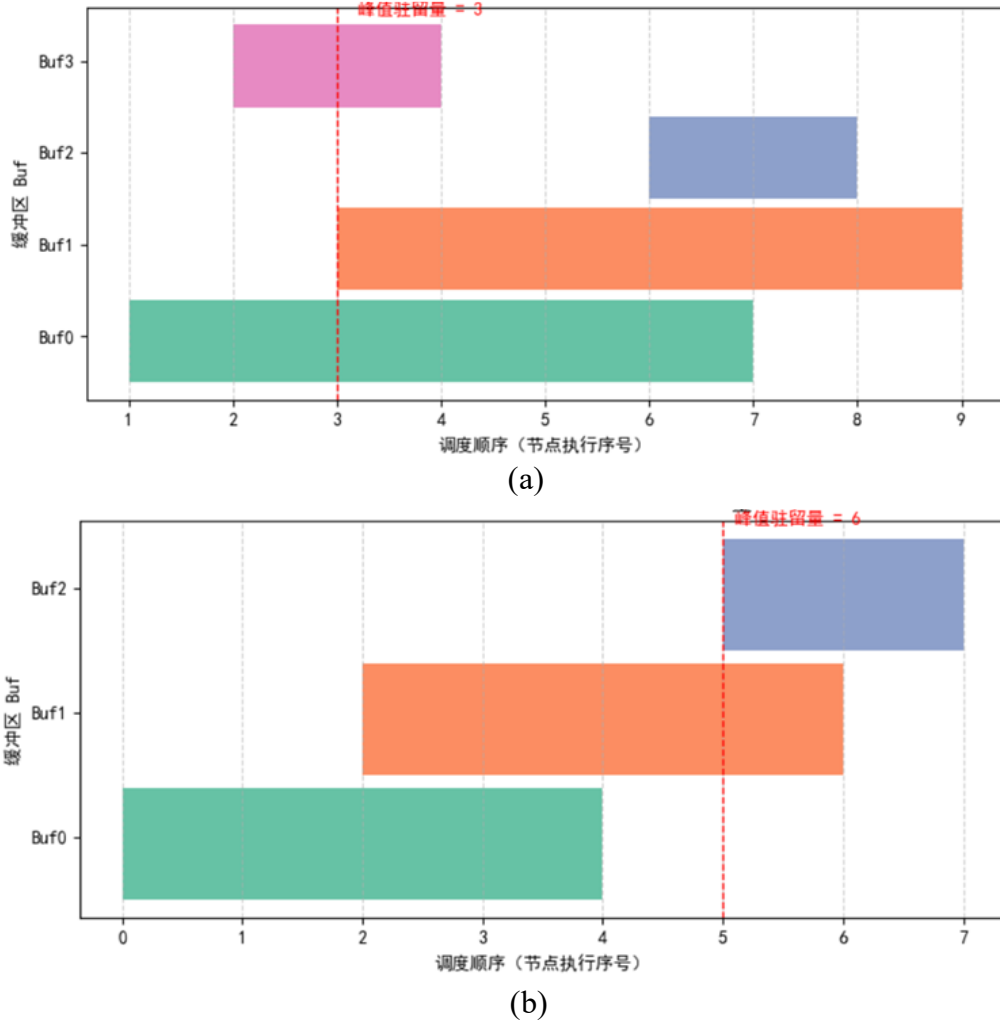


图 3 驻留调度峰值示意图

在问题求解方面，我们先采用三类**贪心策略**（MinPeak / EarlyFree / LateAlloc）生成拓扑可行的初始解。贪心虽属启发式，但其基于就绪集与优先队列的实现能将时间复杂度控制在 $O(|V|\log |V| + |E|)$ 以内，可明确计量、可复现，适合作为后续比较的基准。在此基础上，再引入**遗传**、**模拟退火**、**禁忌搜索**等方法对贪心解进行峰值敏感的局部重排与微调，以检验与提升不同数据类型和图结构上的解质量；同时避免将整体论证建立在单一启发式之上，兼顾通用性与泛化性。

3.2 数据分析

3.2.1 数据集选择

本题选择的数据集包含两个主要部分：**Nodes.csv** 和 **Edges.csv**。**Nodes.csv** 记录了每个节点的唯一标识符、操作类型、执行周期、缓存标识符、缓冲区大小、类型、执行单元、执行周期数、缓冲区 ID 列表信息，而 **Edges.csv** 描述了节点之间的依赖关系。这些数据为最小缓存驻留调度的求解提供了关键的计算和内存需求支持，帮助本文在调度中合理分配资源并优化内存占用。后续题目建模所需要的数据集与本题相同，故后文不再重复数据分析的内容。

3.2.2 数据探索及可视化分析

本题的计算图包含多类算子、复杂依赖与交错的 **ALLOC/FREE**，目标是在满足拓扑与硬件缓存约束的前提下最小化最大缓存驻留量。该类带前驱约束的峰值资源最小化本质上为 **NP 难** 问题：穷举在所有拓扑可行序列上求解呈指数级，动态规划仅在受限结构下可行，难以推广到通用 **DAG** 与多重硬件约束。

为更好地了解求解所需的数据集，以示范数据中的 **Conv_Case0** 为例，本文通过对数据集进行筛选，得到数据缓存区共 830 个、依赖关系共 3869 个、ID 字段共 2580 个，在执行单元中共发现 **CUBE** 计算单元 300 个、**MTE1** 数据搬运单元 308 个、**MTE2** 数据搬运单元 303、**MTE3** 数据搬运单元 7 个。可以看到示范数据中数据节点多、依赖关系复杂，为进一步分析数据，本文使用 **Gephi** 来可视化各节点之间的关系，有向图的可视化结果如图 4 所示：

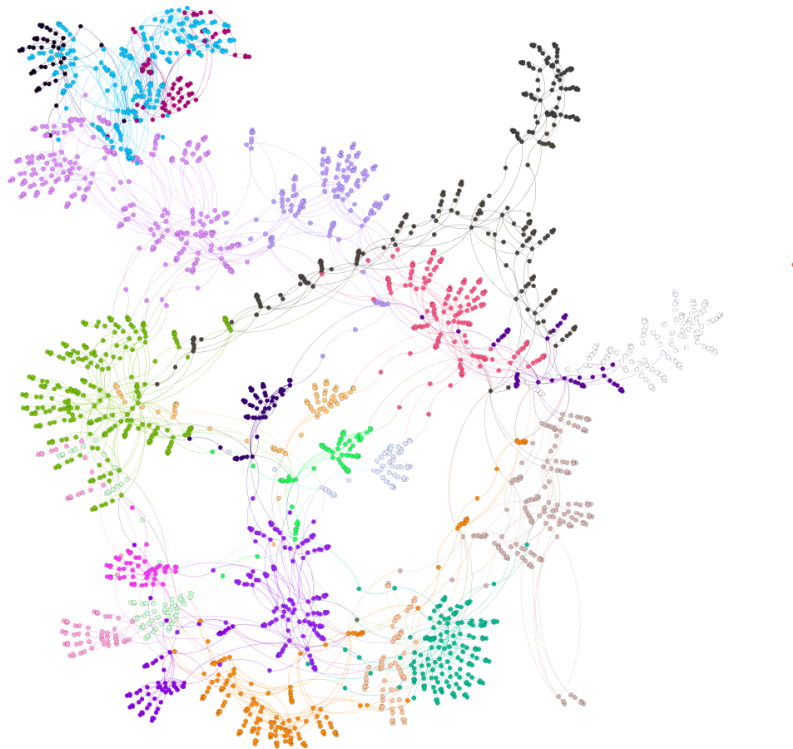


图 4 Conv_Case0 各节点可视化图

此外，本文还通过 Gephi 对进行模块化分析，得出如下结果：

表 1 Conv_Case0 模块化分析结果表

参数	值
模块化值	0.885
子群数量	24

从模块化分析结果中可以看出，Conv_Case0 中的 2580 个节点共组成 24 个子群，且 0.885 的模块化值表明各节点的连接较为紧密，也印证了题目所给出数据集的复杂性。其节点大小分布图如图 5 所示。

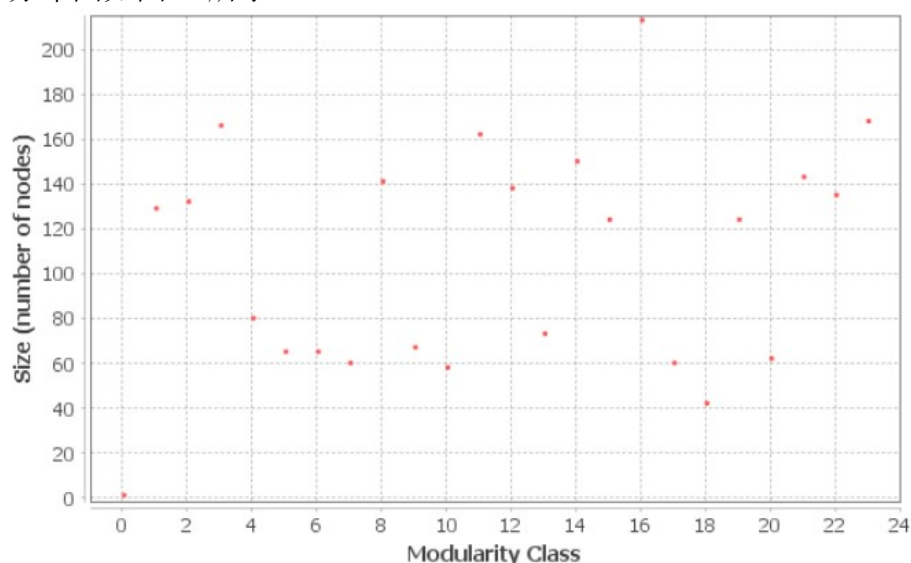


图 5 Conv_Case0 节点大小分布图

从图中可以看到，计算图中的节点大小分布非常广泛，进一步验证了数据集对应计算图的复杂性和异构性。为了更直观地展示数据的复杂性，我们选择了一个根节点和一个叶节点作为核心，围绕它们绘制了包含约 20 个节点的子图，来呈现节点之间的依赖关系和数据流动。如图 6 所示，子图(a)是以根节点 0 为核心的部分节点有向图，子图(b)

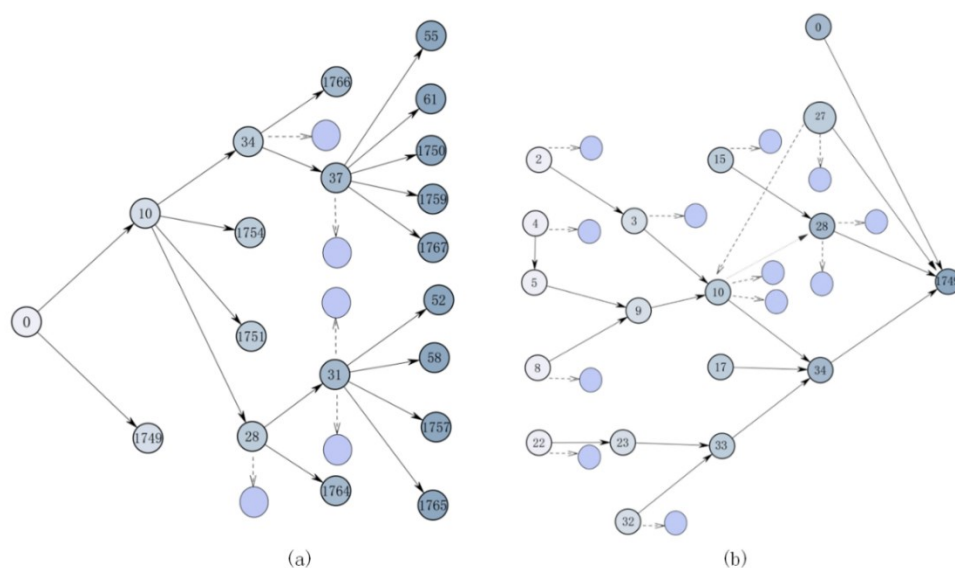


图 6 Conv_Case0 部分节点有向图

是以叶节点 1749 为核心的部分节点有向图。其中，实线表示本图所展示节点之间的依赖关系，虚线表示本图所展示节点与未展示节点之间存在依赖关系，同一节点延伸出的虚线越多说明本图中未展示的依赖关系越多。

接下来，我们从节点有向图中挑选了若干个节点，并对其进行了简易的公式图可视化，进一步展示了这些节点之间的依赖关系和计算顺序，如图 7 所示。

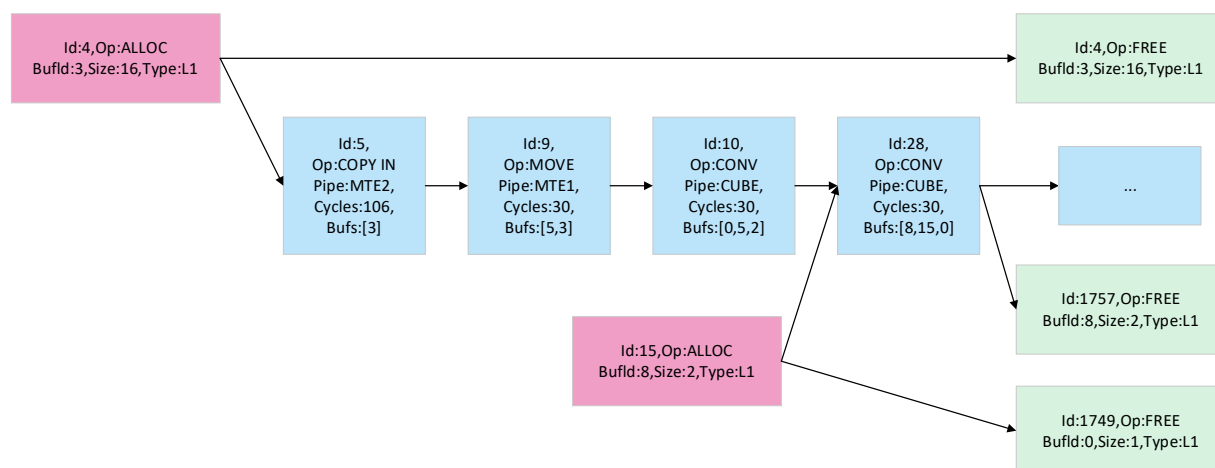


图 7 Conv_Case0 部分节点公式图

我们以 Conv_Case0 数据集为例，深入分析了数据集的结构和复杂性。分析表明，示范数据中节点数量庞大、依赖关系复杂，为后续调度算法设计和优化提供了参考。而在所有问题的求解过程中，需同时考虑模型的泛化性与通用性，这要求我们深入分析各个不同算子计算图的关键数据特性。附件中共给出 Conv、FlashAttention、Matmul 这三种类型的典型算子。Conv（卷积算子）是卷积神经网络中最基本的计算单元，广泛应用于图像处理、视频分析和信号处理等领域。FlashAttention 是一种高效计算 Transformer 网络中注意力机制的算法，专门针对大规模的自注意力计算进行优化，主要应用于大型语言模型的自注意力机制。Matmul 是线性代数中的基础操作，广泛应用于神经网络中的前向传播和反向传播过程。在深度学习中，矩阵乘法常常用于实现权重和输入数据之间的线性变换，计算复杂性高。这三种典型算子分别在不同类型的神经网络中扮演着关键角色，它们的计算特性也决定了调度算法在这些算子上的实现方式。因此，设计一种具有可推广性的算法，来得到合理的调度顺序、内存管理和计算优化策略是提高通用神经网络处理器效率和性能的关键。

节点的入度和出度是我们分析的重点指标：入度表示指向某个节点的依赖边数量，出度则表示从该节点发出的依赖边数量。节点的入度和出度直接影响调度算法中的任务执行顺序和并行性，通过分析计算图中节点的平均入度和出度，可以了解图中节点间的依赖关系的强度，从而为后续的调度策略提供指导。

表 2 各算子平均入度和出度表

算子	平均入度和出度
Conv	0.431
FlashAttention	0.624
Matmul	0.565

可知，FlashAttention 算子的平均入度和出度最高，为 0.624，说明 FlashAttention 算子每个节点通常与更多的其他节点存在依赖关系，计算过程相较之下会更为复杂，涉及

多个阶段和计算单元的交互，可能需要处理更大的数据流量和更多的中间计算结果。Conv 算子的平均入度和出度最低，仅为 0.431。这意味着卷积算子的计算图中节点之间的依赖关系较少，相对独立。卷积操作通常是局部计算，因此可以有效地进行并行化，减少串行依赖。较低的入度和出度可能使得卷积算子的调度更加灵活，能够通过流水线或并行化策略提高计算效率。

此外，我们还考虑节点的平均大小。每个节点的大小表示该节点所需的缓存资源量，节点的平均大小直接影响计算图中的内存需求和缓存管理策略。通过分析计算图中节点的平均大小，可以评估不同算子中节点对内存资源的大致需求，从而帮助优化调度顺序和内存分配策略。各算子平均节点大小如表 3 所示：

表 3 各算子平均节点大小表

算子	平均节点大小
Conv	0.034
FlashAttention	0.017
Matmul	0.015

可知，Conv 算子的平均节点大小最大，为 0.034，是其他两种算子的两倍。这表明，卷积算子在计算图中每个节点所需的缓存空间相对较大。卷积操作通常需要处理较大的数据块，特别是在处理图像等高维数据时，每个卷积操作都需要大量的内存来存储输入特征图、卷积核和中间结果。因此，在选择算法和调参时，要考虑到较大的平均节点在计算过程中对内存的高需求。

通过对 3 种算子平均节点的入度和出度量、节点的平均大小的比较，我们发现不同算子在计算图中的资源需求和依赖结构存在显著差异，这为后续调度算法的设计和优化提供了有价值的依据。Conv 算子具有较低的平均入度和出度，而平均节点较大。在设计调度算法时需要考虑内存占用的优化。Matmul 算子保持适中的平均入度、出度以及较低的平均节点内存占用，它的调度更加灵活，内存压力较小。FlashAttention 算子具有高平均节点出入度而平均节点较低，因此在优化 FlashAttention 调度时，除了内存管理外，还需关注其依赖关系的高耦合性，确保数据流的高效调度和并行计算。

3.3 核心方法的适应性分析

3.3.1 贪心算法的适应性分析

贪心算法 (Greedy Algorithm) 是一种在每一步选择中都采取在**当前状态下最优**的选择，希望通过每一步的局部最优，最终得到全局最优解的方法。通俗地讲，贪心算法总是只看眼前利益，每一步都“贪”当前最好的，不回溯、不修改^[28]。

在本问最小缓存驻留调度问题中，贪心算法的应用与活动选择问题有相似之处。我们首先根据每个任务的内存需求和执行周期对任务进行排序，优先选择内存需求较小或执行周期较短的任务，这样可以有效减少整体缓存的峰值驻留，避免在调度过程中出现内存瓶颈。然后，每次选择一个任务时，判断其开始时间是否与前一个已选择任务的结束时间相符，如果符合条件，则选择该任务并更新当前的缓存驻留值，重复此过程直到没有更多任务可以选择^[28]。

通过这种贪心策略，我们确保了每一步选择的局部最优解，即每次选择一个任务时，都是当前条件下缓存需求最小的任务，从而最小化最大缓存驻留量。这种方法能够在有限的硬件资源下，快速找到合适的调度顺序，从而降低内存峰值，提升计算效率。

图 8 展示了贪心算法在最小缓存驻留调度问题中的应用过程，流程图具体体现了如何根据内存需求和执行周期选择任务，以及如何更新缓存驻留值，确保调度过程中的缓存驻留最小化。

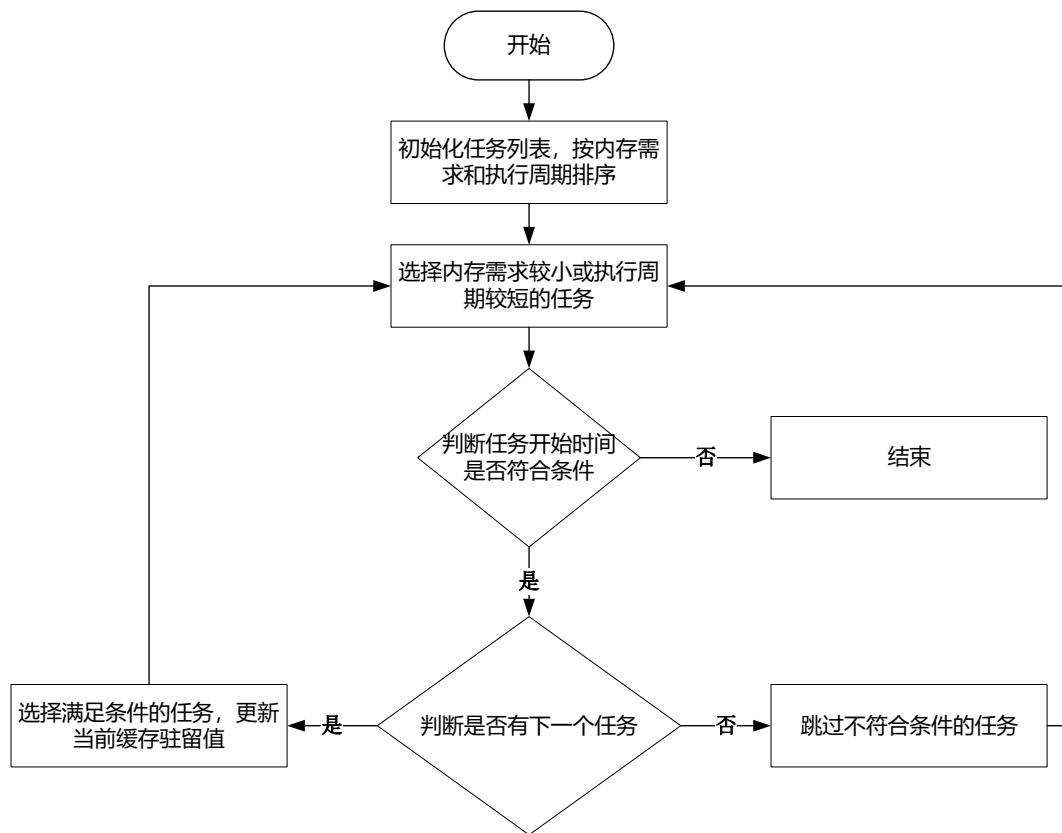


图 8 贪心算法原理

3.3.2 启发式算法的适应性分析

启发式算法（Heuristic Algorithm）是一类用于求解复杂问题的算法，通过某些经验或策略来引导搜索过程，以获得一个近似解。与精确算法不同，启发式算法通常不保证能得到最优解，但它能够在合理的时间内找到一个“足够好”的解。启发式算法广泛应用于优化问题，尤其是当问题规模庞大、求解最优解的计算代价过高时。

在最小缓存驻留调度问题中，启发式算法的应用能够帮助我们在给定的计算图中通过局部优化来最小化最大缓存驻留量。具体而言，我们通过启发式策略选择合适的任务顺序，尽量减少每个任务对缓存的占用，从而实现缓存驻留的最小化。特别是在面对任务的内存需求和执行周期时，启发式算法能通过优先选择内存需求小或执行周期短的任务来优化整个调度过程。鉴于本题数据与计算图结构高度异质，单一启发式难以在所有场景稳定最优。为提升**通用性**与**泛化性**，我们并行采用**遗传算法**（群体智能）、**模拟退火算法**（单点随机全局搜索）与**禁忌搜索算法**（记忆化局部迭代）三类互补策略。多启发式作为检验与对照，在不同数据类型上更稳健地压低最大缓存驻留量。

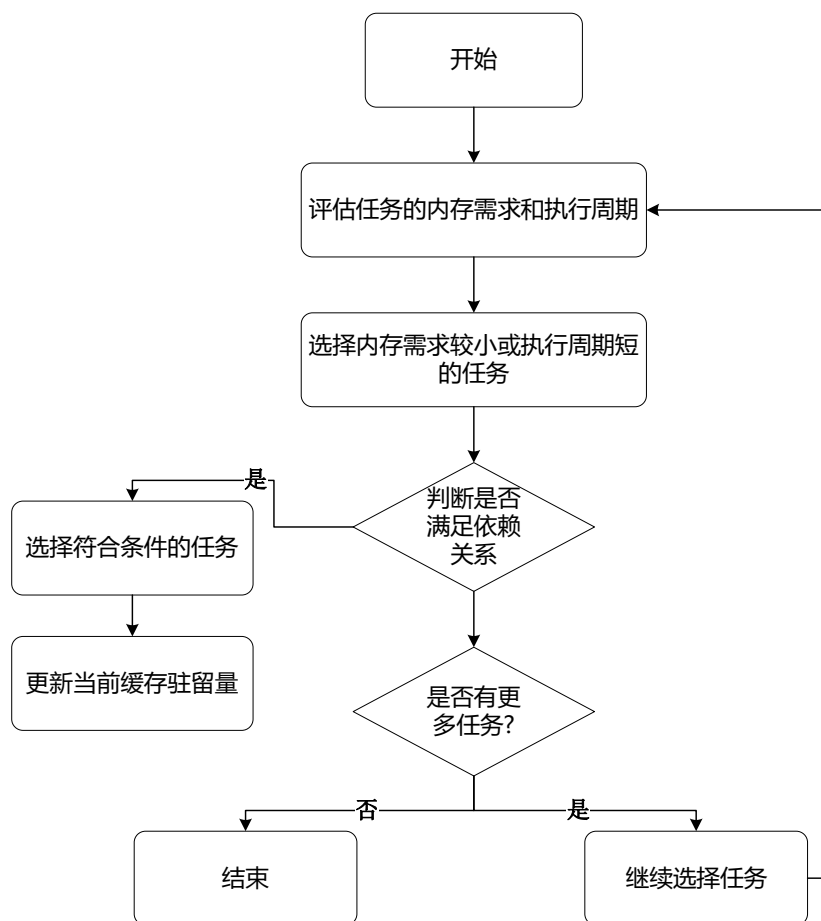


图 9 启发式算法原理图

在本问当中，我们可以采用的启发式算法有模拟退火算法、遗传算法、蚁群算法、禁忌搜索算法等。

(1) 模拟退火算法 (Simulated Annealing)

模拟退火算法是一种模拟物理退火过程的全局优化算法。在算法中，通过模拟物质在热力学平衡条件下的退火过程，使得系统逐渐达到一个最低能量状态，从而获得问题的最优解。具体而言，模拟退火算法通过引入温度参数控制搜索过程中的随机性，在每一步选择一个新解时，如果该新解比当前解更优，则直接接受；如果新解较差，则以一定的概率接受，从而避免陷入局部最优解。随着温度的逐渐降低，算法的随机性逐渐减弱，最终收敛于全局最优解。该算法适用于大规模、复杂的优化问题，但在某些情况下，退火过程的温度控制和退火速度的选择可能对结果有较大影响^{[7][24]}。

(2) 遗传算法 (Genetic Algorithm)

遗传算法是一种基于自然选择和遗传学原理的优化算法。它模仿了生物界中的自然选择和遗传机制，通过选择、交叉和变异等操作模拟种群在进化过程中逐代改善适应度的过程。遗传算法的核心思想是通过对一个初始种群的个体（候选解）进行繁殖，形成新的种群，并不断选择适应度较高的个体进行交叉和变异，从而生成更加优良的解。在每一代中，适应度更高的个体有更大的机会传递其基因（解决方案的部分信息）到下一代，最终找到全局最优解。遗传算法的优势在于能够有效地搜索大范围解空间，且对局部最优解有较强的跳出能力。然而，它的缺点是收敛速度可能较慢，且在复杂问题中，如何设置合适的交叉率、变异率和选择策略仍是关键问题。

(3) 禁忌搜索算法 (Tabu Search)

禁忌搜索算法是一种基于局部搜索的优化算法，它通过引入“禁忌表”机制避免在搜索过程中陷入局部最优解。禁忌搜索通过对当前解的邻域进行搜索，并在邻域中选择最优解作为当前解的更新，同时使用禁忌表记录已经搜索过的解，避免重复搜索。禁忌表中的解被称为禁忌解，通常会设定一个短期和长期的禁忌机制，以确保搜索的多样性。禁忌搜索的优势在于可以跳出局部最优解，通过控制搜索的历史路径使得搜索不容易重复。此外，它的计算效率较高，适用于大规模问题。然而，由于禁忌表的设计和存储需要额外的计算资源，且禁忌策略的选择和禁忌长度的设置对性能有较大的影响，因此在应用时需要根据具体问题进行调优。

3.4 基于多启发式协同优化算法体系与建模

3.4.1 单目标规划模型

在最小缓存驻留调度问题中，单目标规划模型的最小化函数和目标约束可以表述为：

(1) 目标函数是最小化调度过程中所需的最大缓存驻留量：

$$\text{minimize} \max_{1 \leq i \leq N} \left(\sum_{j=1}^i M(v_j) \right) \quad (3-1)$$

其中， v_j 为第 j 个节点， $M(v_j)$ 为第 j 个节点的缓存需求。

(2) 约束条件：

本题单目标规划问题包括依赖关系约束：对于数据集中的所有有向边 $[i, j]$ ，其节点 v_i 必须在节点 v_j 之前执行；L0约束：L0A、L0B、L0C缓存上每次只能驻留一个缓冲区；缓存共享的顺序约束：后一个缓冲区的ALLOC指令必须等前一个缓冲区的FREE指令执行结束后才能执行；任务顺序约束：每个任务必须等前序任务完成之后执行。

$$\begin{cases} S_i < S_j \\ \sum_{i \in C} \text{executing}(i, t) \leq 1 \quad \forall C \text{ at time } t \\ S_{\text{ALLOC}(v_i)} < S_{\text{FREE}(v_j)} \text{ if } v_i \text{ and } v_j \text{ share the same cache} \\ S_j < S_i \quad \forall [i, j] \text{ in the dependency relation} \end{cases} \quad (3-2)$$

其中， S_i 为任务 v_i 的调度时间。

3.4.2 贪心算法模型

在实际场景中，计算图可能由多个不同的算子组成，且其结构特征高度多样化。不同应用领域的计算图呈现出截然不同的内存使用模式：一些计算图可能包含大量的大内存分配操作，另一些则可能频繁进行内存释放，还有的计算图则存在复杂的内存分配与释放交替模式。由于这种多样性，单一的优化策略难以在所有场景下都表现优异。例如，最小化峰值策略在内存密集型场景中表现较好，最早释放策略适用于内存释放频繁的场景，而延迟分配策略更适合需要推迟大内存分配的应用。每种策略都有其特定的优势和适用边界。

在本问题中，贪心算法的目标是最小化执行过程中所需的最大缓存驻留量。具体来说，贪心算法根据每个任务的内存需求和执行周期，优先选择内存需求较小、执行周期较短的任务进行调度，从而减少整体的内存驻留量。随着调度的进行，算法会动态更新当前缓存驻留量，最终计算出最大缓存驻留量。因此在解决最小缓存驻留调度问题时，针对不同的内存使用模式，我们设计了三种贪心策略：**最小化峰值策略**、**最早释放策略**

和**延迟分配策略**。每种策略适用于不同的应用场景，并在特定条件下具有显著的优势。各贪心策略的对比如表 4 所示。

(1) 最小化峰值策略

最小化峰值策略的核心思想是优先执行能够减少缓存占用的操作。在该策略中，FREE 操作具有最高优先级，ALLOC 操作按内存大小降序排列，其他操作则优先级较低。通过优先释放大内存块，能够有效降低内存峰值，适合内存密集型任务和大内存分配较多的计算图。

(2) 最早释放策略

最早释放策略则将所有FREE操作赋予最高优先级，确保内存资源尽早释放，从而提高内存资源的周转效率，适用于内存释放频繁、内存需求相对均匀的应用场景。

(3) 延迟分配策略

延迟分配策略的核心思想是推迟大内存分配，避免过早占用大量内存，从而平稳控制内存使用的增长节奏。该策略适合那些计算过程前期内存分配较多的场景。

表 4 各贪心策略优势对比

策略	核心思想	FREE 优先级	ALLOC 优先级	主要优势
最小化峰值	优先减少缓存占用	最高(0)	大内存优先(1)	降低峰值
最早释放	尽快释放内存	最高(0)	中等(1)	快速释放
延迟分配	推迟内存分配	最高(0)	大内存延后(1)	推迟占用

调度顺序：节点调度序列为 $S = \{v_1, v_2, \dots, v_k\}$ ，其中 v_i 表示第 i 个节点， $M(v_i)$ 表示节点 v_i 的缓存需求。

最大缓存驻留量：

$$\max(V_{stay}) = \max\left(\sum_{i=1}^k M(v_i)\right) \quad (3-3)$$

其中 $M(v_i)$ 为第 i 个节点的缓存需求。

3.4.3 模拟退火算法模型

模拟退火算法模拟物理退火过程，逐步减少随机性，从而避免陷入局部最优解。在最小缓存驻留调度问题中，模拟退火可以用来在全局搜索中探索不同的调度序列，并通过控制“温度”来决定接受较差解的概率。

以合法交换/插入/反转为邻域，代价函数取：

$$\text{Cost}(S) = V_{\max}(S) + \lambda \Pi(S) \quad (3-4)$$

给定温度 T ，对候选 S' 采用 Metropolis 接受概率：

$$p(S \rightarrow S') = \min\left\{1, \exp\left(-\frac{\text{Cost}(S') - \text{Cost}(S)}{T}\right)\right\} \quad (3-5)$$

温度按几何降温或 Cauchy 降温：

$$T_{k+1} = \alpha T_k (\alpha \in [0.90, 0.99]) \text{ 或 } T_k = \frac{T_0}{1 + \beta k} \quad (3-6)$$

通过查阅相关文献^[7]，本文设定初始参数：初温 T_0 使初期接受率约 60%–80%，每温度步迭代 $L \approx 10 |V|$ 次，终止于 $T \leq T_{min}$ 或连续 $gstall$ 轮无改进。通过分析得出，该模型复杂度为 $O(K(|V| + |E|))$ ，其中 K 为总迭代数。

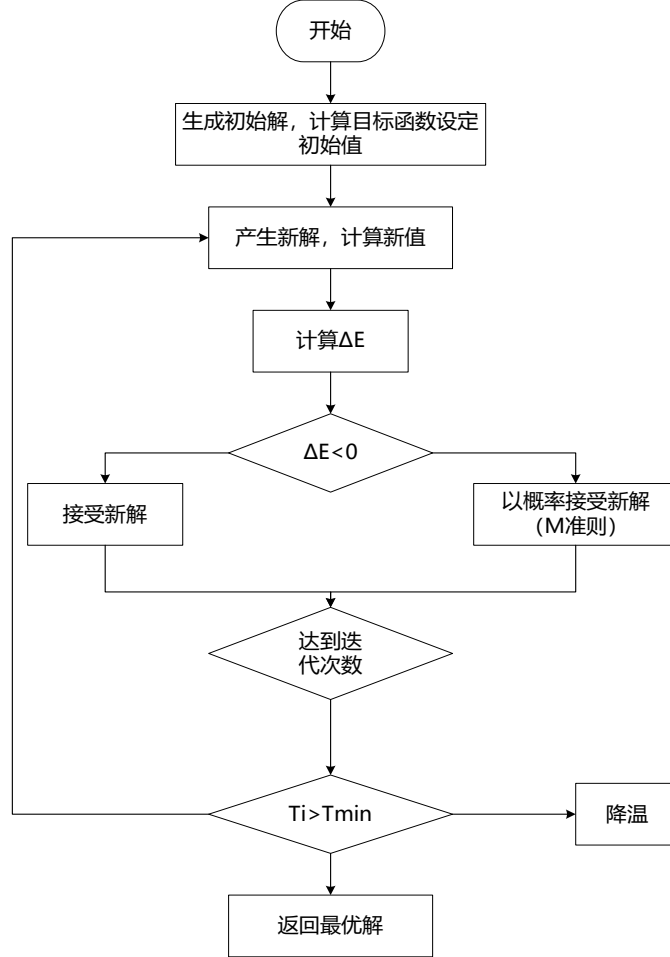


图 10 模拟退火算法流程图

3.4.4 禁忌搜索算法模型

禁忌搜索算法的步骤如下：

- (1) 从贪婪解开始，初始化禁忌表为空。
- (2) 生成邻域解，选择最佳非禁忌解（或满足渴望准则的解）。
- (3) 更新禁忌表，添加新移动。
- (4) 重复步骤 2-3 直达到迭代次数。

以交换/插入/反转定义邻域，仅保留可行候选或用轻量修复最小化违规。仍以 $\text{Cost}(S) = V_{max}(S) + \lambda \Pi(S)$ 为代价，采用长度为 L 的禁忌表记录最近相对次序改变/位置变更，并设置愿望准则：若 $\text{Cost}(S') < \text{Cost}(S^*)$ 则越禁。每轮从非禁忌邻域中选

$$S \leftarrow \arg \min_{S' \in N(S) \setminus T} \text{Cost}(S') \quad (3-7)$$

周期性在峰值窗口强化搜索；若连续 $Tstall$ 轮无改进则触发发散/重启。

参考本问题数据特性以及相关文献^{[8][9]}，本问题的禁忌搜索泛模型参数取值为：禁忌期限 $L \in [7,50]$ 、邻域规模 $|N(S)| \in [50,150]$ 、最大迭代 $K \in [200,1000]$ 、停滞阈值 $T_{stall} \in [50,200]$ 。

本算法模型复杂度为 $O(K \cdot |N(S)| \cdot (|V| + |E|))$ 。

3.4.5 遗传算法模型

遗传算法通过模拟自然选择过程来寻找问题的近似最优解。在最小缓存驻留调度问题中，遗传算法可以用来探索可能的调度序列，并通过交叉、变异等操作生成新的调度方案^[10]。为最小化计算图执行过程中的最大缓存驻留量记作 V_{max} ，采用遗传算法在满足 DAG 依赖与 L0/L1/UB 等约束的可行调度空间内搜索近优序列。

算法原理：模拟退火通过概率接受劣解来避免局部最优。它使用温度参数控制搜索过程：高温时接受劣解的概率高，随着温度下降，逐渐偏向接受更优解。

初始解处理：使用贪心算法生成的解作为初始解。这确保了起点具有较低的最大缓存驻留量，加速收敛。

邻域操作：定义邻域结构，如交换两个节点的顺序、插入节点或反转子序列。例如，随机选择调度序列中的两个位置并交换节点 ID。

温度计划：设置初始温度、冷却速率和终止温度。常见冷却计划：指数冷却 ($T_{k+1} = \alpha \cdot T_k$ ，其中 $\alpha \approx 0.9$)

采用拓扑合法全序列为个体编码。目标一致的带惩罚适应度定义为：

$$\text{Fit}(S) = \frac{1}{\varepsilon + V_{max}(S) + \lambda \Pi(S)} \quad (3-8)$$

其中 $\Pi(S)$ 为违约度（依赖逆序数与 L0 冲突数）， $\lambda > 0$ 为软惩罚系数， $\varepsilon > 0$ 防止除零。

选择算子为锦标赛（规模 $k = 3$ ），交叉使用顺序交叉（OX），变异在交换/插入/区段反转中随机取一。对初始种群与每次交叉/变异所得子代，均执行

$$S \leftarrow \text{ensure valid}(S) = \begin{cases} S, & S \text{ 拓扑合法} \\ \text{topo repair}(S, E), & \text{否则} \end{cases} \quad (3-9)$$

式 3-9 为遗传算法**内置纠错机制**：一旦出现依赖逆序即最小扰动修复为拓扑序。综合本题计算图规模与相关文献查阅^[11]，本题设定遗传算法默认参数：种群 $P \in [50,200]$ 、最大代数 $G \in [100,500]$ 、交叉率 $p_c = 0.8 \sim 0.95$ 、变异率 $p_m = 0.02 \sim 0.08$ 、精英比例约 3%。

3.5 多视角下不同启发式算法的求解分析

本文代码运行的实验环境如表 5 所示：

表 5 实验环境参数表

类别	项目	参数/型号
CPU	型号	12thGenIntel(R)Core(TM)i9-12900K
	基础频率	3.60GHz
内存	总容量	32GB
GPU	型号	NVIDIA GeForce RTX 3090 Ti
	显存容量	24GB

3.5.1 贪心算法模型求解结果

我们根据 3.4.2 节所建立的 3 种贪心算法策略对附件中的 6 个任务分别进行求解，结果如表 6 所示：

表 6 各贪心算法策略求解结果表（单位：缓存容量 bite）

任务名	最小化峰值策略	最早释放策略	延迟分配策略
Conv0	150587	14664	128616
Conv1	316032	14048	310048
FlashAttention0	64264	5140	22784
FlashAttention1	248976	8228	79874
Matmul0	163840	9728	23808
Matmul1	1179648	35328	96512

由上表可以明显看出，贪心算法最早释放策略的求解结果最优，结果值仅为其他结果的 1/10 到 1/3，这是因为最早释放策略的核心思想是通过优先执行 FREE 操作来尽早释放内存，所有 FREE 操作分配最高优先级。这样能够尽量避免内存占用的延长，提高内存资源的周转效率，并且策略的实现相对简单，不涉及内存块大小的排序计算，计算开销较小。从泛化性与通用性的角度来看，最早释放策略能够适应多种内存使用模式，适合应用在实时系统、流式数据处理等内存资源紧张且内存分配频繁的场景中。

为了更直观地体现各方案缓存容量的实时变化情况，我们以 FlashAttention0 算子为例，绘制了三种策略缓存驻流量变化图，如图 11 所示。

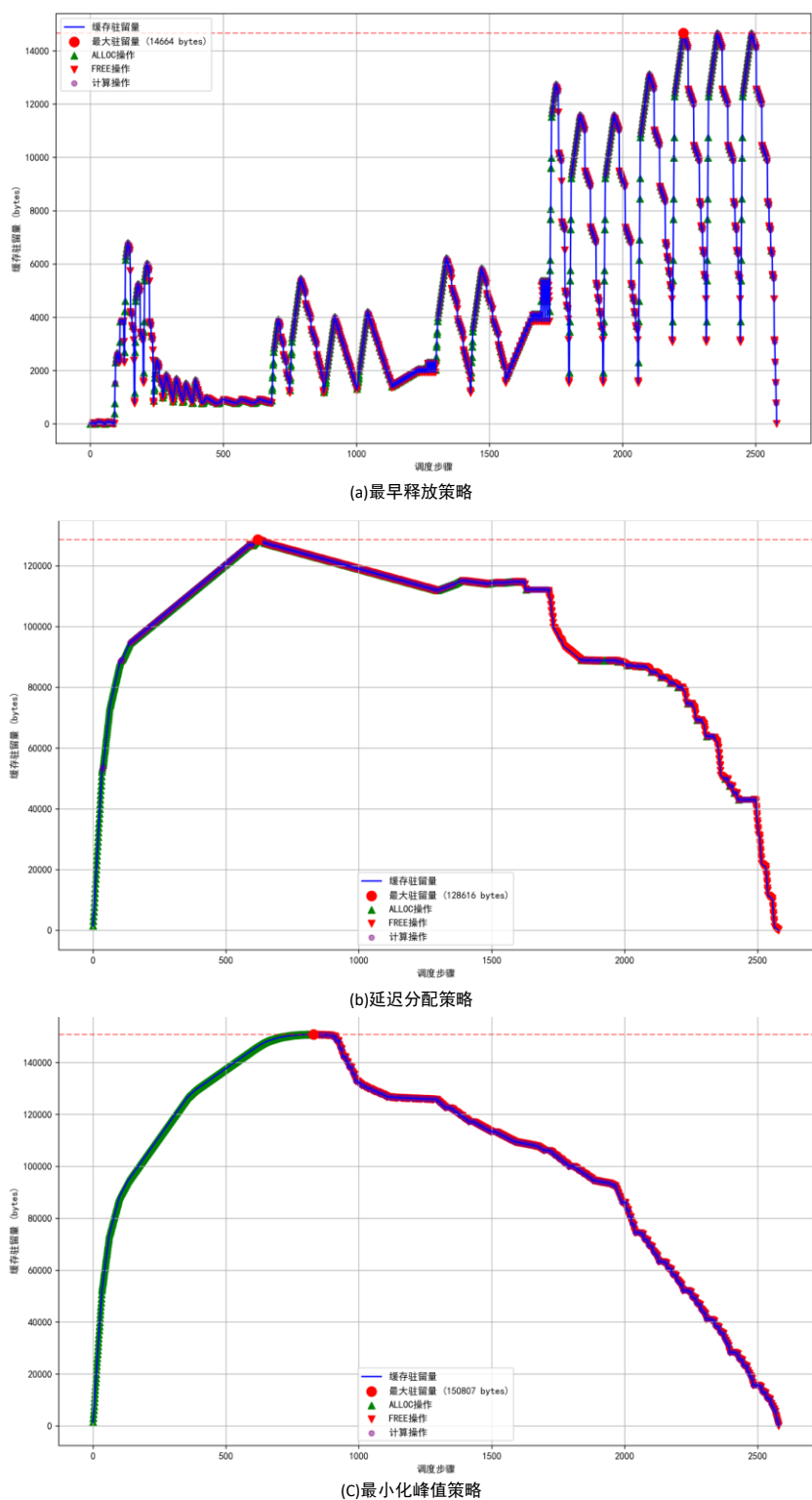


图 11 FlashAttention0 算子下各贪心算法策略缓存驻流量变化图

3.5.2 模拟退火算法模型求解结果

如图 12 所示，六个算例的模拟退火曲线整体呈“先快后慢”的下降趋势：在初始温度较高的阶段，算法通过以一定概率接受劣解迅速离开初始盆地， $Max(Vstay)$ 明显下降；随着温度降低，曲线进入缓收敛区间。

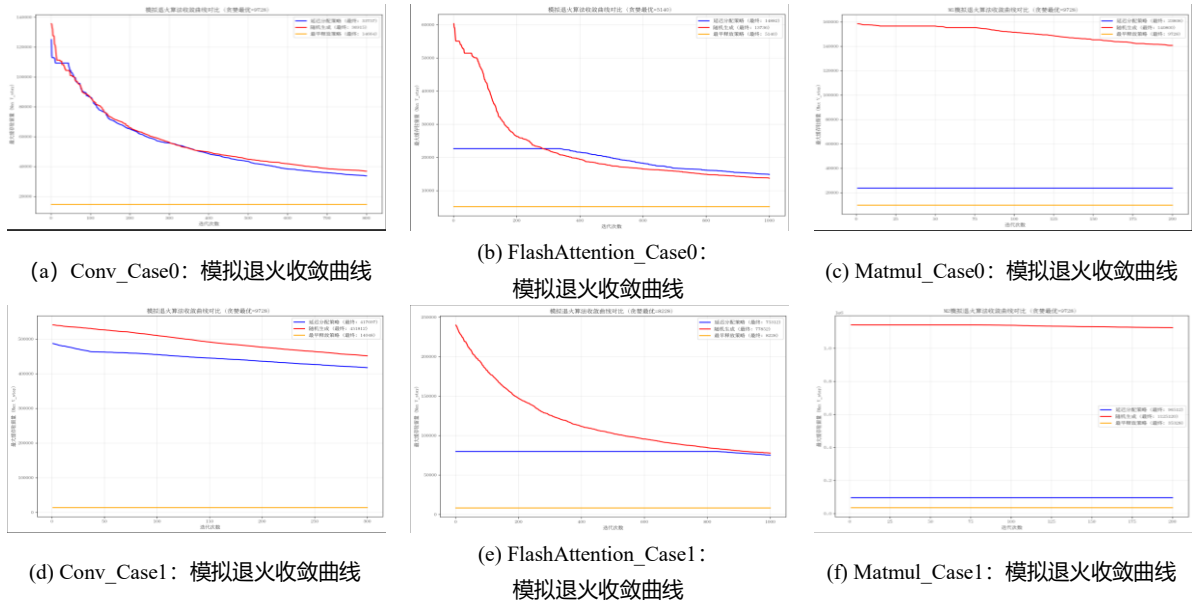


图 12 模拟退火在六个示例计算图上的收敛曲线：最大缓存驻留量 $Max(Vstay)$ 随迭代次数变化图

对比两种初解，延迟分配（蓝）在 $Conv_Case0/1$ 中普遍优于随机（红），表明“推迟大 ALLOC、配合提前 FREE”能够提供更有利的局部交换窗口；在 $FlashAttention_Case0/1$ 中，两条曲线均有较大降幅且后期趋同，说明此类图就绪集丰富、存在多条可达改良路径；而在 $Matmul_Case0/1$ 中，曲线接近水平或仅小幅下降，显示结构更为刚性、可交换空间有限，贪心基线已接近可达下界，SA 难以进一步显著减峰。总体而言，SA 对重叠生存期较长、就绪集可交换性强的计算图能取得持续改进；对依赖与资源约束强的图，改进幅度受限。上述现象与温度驱动的先全局搜寻、后局部细化的机制相一致，也与我们在 TS 实验中观察到的类目差异具有一致性。

3.5.3 禁忌搜索算法模型求解结果

如图 13 所示，我们在六个算例上绘制了禁忌搜索（TS）的收敛轨迹。横轴为迭代次数，纵轴为对应迭代得到调度序列的最大缓存驻留量 $Max Vstay$ ，数值越低越好。为考察初解对 TS 的影响，我们分别以“延迟分配（蓝）”与“随机（红）”两种初解启动搜索，并以“最早释放（橙）”的贪心结果作为非迭代参考线。整体上，曲线呈阶梯式单调下降：前期（前 100–300 代）降幅显著，后期进入缓慢收敛阶段，符合禁忌搜索“先粗后细”的典型特征。

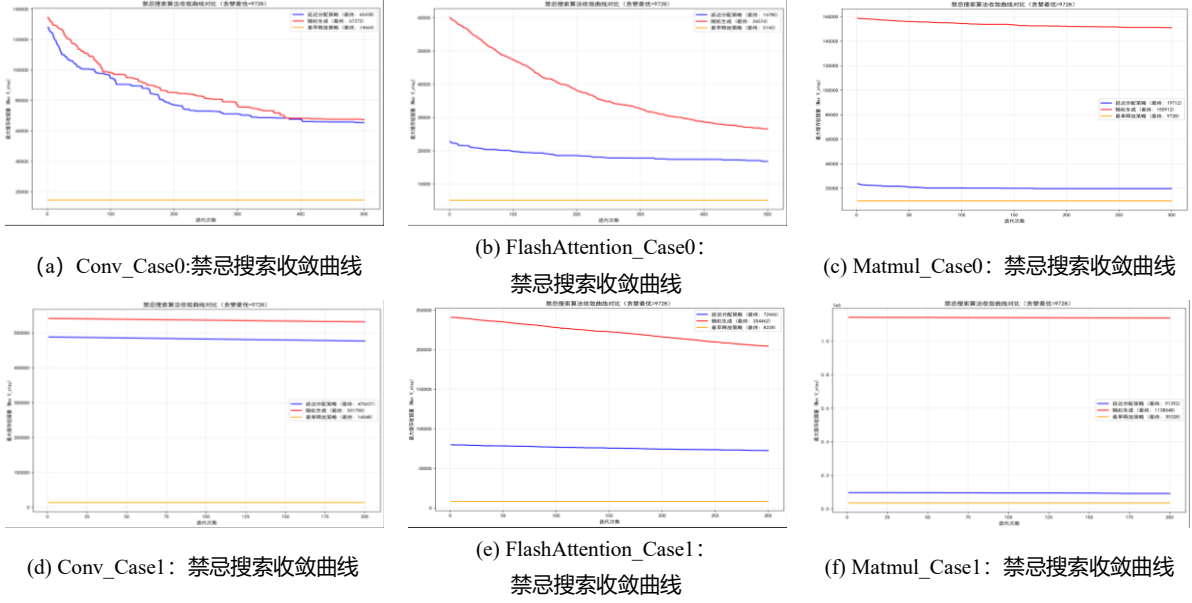


图 13 禁忌搜索在六个计算图上的收敛曲线：最大缓存驻留量 $Max(V_{stay})$ 随迭代次数变化图

对比不同算例，**FlashAttention_Case0/Case1** 与 **Conv_Case0** 的曲线降幅最大、持续时间也更长，说明这三类图的就绪集更为丰富、缓冲生存期重叠较长，存在较多可被交换/插入打散的“峰值窗口”，因此优化空间更大；其中 **FlashAttention_Case0** 的改进最为明显。**Conv_Case1** 的峰值下降幅度中等，表明结构相对收紧但仍有一定可交换区间。相比之下，**Matmul_Case0/Case1** 的曲线几乎水平或仅小幅下降，显示依赖与资源约束更“刚性”、局部可交换性弱，可进一步压峰的空间较小。此外，以“延迟分配”作为初解的蓝线整体早于随机初解（红线）达到较低峰值，说明先推迟大 ALLOC、配合提前 FREE 能为 TS 创造更有利的邻域搜索起点。综上，TS 在 **FlashAttention** 与部分 **Conv** 类图上收益显著，而在 **Matmul** 类图上改进有限，这与不同计算图的结构可交换性与生存期重叠程度相一致。

3.5.4 遗传算法模型求解结果

同理，遗传算法对于结果的达到也有一定的作用，但是速度会比较缓慢。从图 14 可见，GA 在 **Conv_Case0** 与 **FlashAttention_Case0** 上随代数总体呈下降趋势，且多代后其当代最优值明显低于红色基线，说明群体重组与变异能持续找到更有利的延后 ALLOC/提前 FREE 局部重排；柱高起伏反映了种群多样性带来的随机波动。相比之下，**Matmul_Case0** 的柱高接近不变或仅小幅下降，表明该图的依赖/资源约束更“刚性”，可交换窗口有限，贪心基线已接近可达下界，GA 的进一步压峰空间较小。

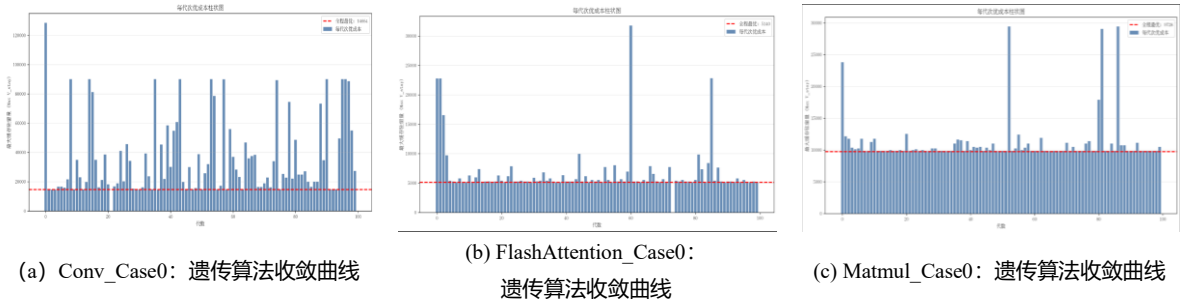


图 14 遗传算法在六个计算图上的收敛曲线：最大缓存驻留量 $Max(V_{stay})$ 随迭代次数变化图

3.5.5 求解结果分析

本题以“最小化最大缓存驻留量”为目标，首先以三类贪心策略构造拓扑可行初解。贪心的优势在于时间复杂度可明确定界、规模可扩展，适合作为统一的比较基线；但其局部决策不可避免地忽略了跨步长、跨窗口的重排机会，容易停在较强的局部最优。为此，我们引入三类互补的启发式算法对贪心解进行二次优化：群体智能的遗传算法（GA），单点随机全局搜索的模拟退火（SA），以及带记忆的局部迭代方法禁忌搜索（TS），分别对应“群体并行探索—概率跳出—记忆驱动的定向搜索”三种不同的改良机制，从而提升求解的通用性与泛化性。

从实证结果看，各方法在不同算子上的表现具有互补性。首先，EarlyFree 贪心在六个算例上都是稳健的强基线，尤其在 Matmul_Case0/1 上已非常接近可达下界，后续改进空间有限。其次，SA 在 FlashAttention_Case0/1 与 Conv_Case0 上的降幅最为显著：曲线呈“先快后慢”，初期高温阶段通过概率接受劣解迅速打散长生命周期重叠，后期温度下降趋于稳收敛；而在 Matmul_Case0/1 中曲线近乎水平，仅有小幅改进，显示该类图的依赖与资源约束更“刚性”，局部可交换性弱。再次，TS 在 FlashAttention_Case0/1 与 Conv_Case0 上同样取得持续下降，其阶梯式收敛与“峰值窗口”强化一致，且以 LateAlloc 作为初解时（蓝线）普遍优于随机初解（红线），说明“推迟大 ALLOC、配合提前 FREE”的结构更利于后续邻域搜索；而在 Matmul_Case0/1 中，曲线基本保持平台期，进一步压峰的空间较小。最后，GA 的改进幅度相对缓慢但稳定，在 Conv_Case0 与 FlashAttention_Case0 上随代数整体下降，体现出群体重组在“就绪集可交换、重叠较长”的图上依旧能挖掘到增量；在 Matmul_Case0 上则几乎不降，结论与 SA/TS 一致。

综上，三类启发式在不同算子上形成“优势互补”：SA/TS 对 FlashAttention 与部分 Conv 的“高耦合、重叠长、可交换性强”的结构更有效，能在贪心基线之上进一步显著减峰；GA 提供稳健但相对缓慢的改良，在可交换性存在但形貌复杂时仍能累积收益；Matmul 类型由于结构刚性、交换窗口狭窄，三法改进均有限，EarlyFree 贪心已接近下界。

为便于将上述图像化结论落到“可比可证”的量化层面，下面给出两张对比表：其一从横向考察同一算法在六个算例上的收敛特征与改进幅度，以识别各算法更擅长的图结构；其二从纵向比较同一算例下三种启发式在起步速度、收敛形态、参数规模与对贪心基线的相对提升，揭示它们的互补性与边界。

表 7 模拟退火算法中六个算例的收敛特征分析表

算例	初期下降 速率	主要改进区间 （迭代次数）	收敛 形态	相对贪心 改善幅度	适配度 判定
Conv_Case0	中→快	0–300	先快后慢	中（明显低于基线）	较强
Conv_Case1	慢	0–200	缓慢单调	小（略优于基线）	一般
FlashAttention_Case0	快	0–400	先快后慢	大（持续降）	强
FlashAttention_Case1	快	0–500	先快后慢	大（持续降）	强
Matmul_Case0	很慢/近水平	—	平台	小（接近基线）	弱
Matmul_Case1	很慢/近水平	—	平台	小（接近基线）	弱

表 8 GA/SA/TS 算法应用于 FlashAttention_Case0 算子指标表

算法	初解敏感性	初期速度	收敛形态	典型参数规模	相对贪心改善	优势
GA	中 (LateAlloc 略优)	中	平滑下降	$P \times G$ (100×300)	中	稳健+并行友好
SA	低~中	快	先快后慢	K (温度×每温度步)	大	能快速跳出局部最优
TS	高 (初解好则明显更优)	中	阶梯式下降	$K \times$	N	有记忆、定向压峰

综合横向与纵向的比较,可以确认三点:(1)启发式算法能够在贪心基线之上持续降低最大缓存驻留量,且在 **FlashAttention** 与部分 **Conv** 这类“就绪集可交换、生命周期重叠长”的计算图上**收益显著**;(2)对 **Matmul** 类“结构刚性、交换窗口狭窄”的实例,三法的改进空间**有限**,EarlyFree 贪心已接近可达下界;(3)但在**时间要求与工程约束**下,三类启发式普遍需要大量迭代与参数调节 (GA: $O(PG(|V|+|E|))$, SA: $O(K(|V|+|E|))$, TS: $O(K|N|(|V|+|E|))$),且存在随机性与复现实验成本,这与我们正文中关于“**动态规划指数爆炸—启发式可改进但计算代价不稳定/不低**”的证据相一致。因此,问题一的**最终提交结果采用贪心算法**(EarlyFree/MinPeak/LateAlloc 中的最优)作为主结果:它**复杂度明确、运行可控、效果稳健**;启发式结果则作为补充对比,说明在允许更长时间预算时的潜在增益与适用边界。

3.5.6 求解结果总结

鉴于启发式需要较长算时与参数调度,在赛题时限下我们采用**贪心最优**作为问题一的结果,启发式作为补充与能力边界说明。问题一结果如表 9 所示。

表 9 问题一结果表

任务名	$\max(V_{\text{stay}})$
Conv_Case0	14664
Conv_Case1	14048
FlashAttention_Case0	5140
FlashAttention_Case1	8228
Matmul_Case0	9728
Matmul_Case1	35328

3.6 算法时间复杂度分析

本题要求“分析算法相对于节点总数 N 的时间复杂度”,这一点对于启发式算法来说并不适用。启发式算法通常无法提供明确的时间复杂度,它们的时间复杂度往往依赖于多个因素(如温度下降的速度、种群大小、迭代次数等),而且不保证能在每次运行时得到最优解。因此,启发式方法的时间复杂度无法准确衡量,也就失去了分析和比较的基础。为了保证可以进行合理的分析和比较,在本文中,我们将不对启发式算法进行理论分析。

在分析最小缓存驻留调度问题的时间复杂度时,选择最优解的步骤尽管有时间复杂度,但它只是对已生成的调度序列进行简单的比较,不会显著增加总复杂度,因此可以忽略。其次,问题的核心计算过程集中在生成调度顺序和计算缓存驻留量,这两个步骤的计算量直接影响算法的时间复杂度。而选择最优解的步骤只是额外的遍历,不会对整

体复杂度产生实质性影响。此外，对于像动态规划和贪心算法等算法，最优解是逐步构建的，不存在单独的“选择最优解”步骤，因此这些算法的时间复杂度通常没有单独的选择步骤，最优解的选择是在计算过程中实时更新的，不需要单独考虑。

同时，虽然不同算法生成的可行解数量不同，暴力求解可能枚举所有合法调度序列，而动态规划或贪心算法可能通过优化策略找到更少的可行解，但由于计算最大缓存驻留量的时间复杂度为 $O(N)$ ，我们将复杂度集中在计算每个可行解的最大缓存驻留量上，而不考虑可行解数量的差异。这样，尽管不同算法生成的解空间大小不同，每个解的缓存驻留量计算时间通常是 $O(N)$ ，因此这部分复杂度可以统一比较。

最后，完整的拓扑排序检查通常需要遍历所有的边（即 E 条依赖边），所以时间复杂度应为 $O(N+E)$ ，这与节点数和边数都有关，同时还要与对应的算法可行解数量相乘。由于本问聚焦于“算法相对于节点总数 N 的时间复杂度”，且在计算最大缓存驻留量时我们排除了可行解数量差异，因此我们在本题不考虑拓扑排序有效性检验的时间复杂度。

表 10 三种算法的时间复杂度

节点数	枚举法 时间复杂度	动态规划 时间复杂度	贪心算法 时间复杂度
1	$O(1)$	$O(1)$	$O(0)$
2	$O(4)$	$O(4)$	$O(1.39)$
3	$O(18)$	$O(9)$	$O(3.30)$
...	...	...	...
N	$O(N! \times N)$	$O(N^2)$	$O(N \log N)$

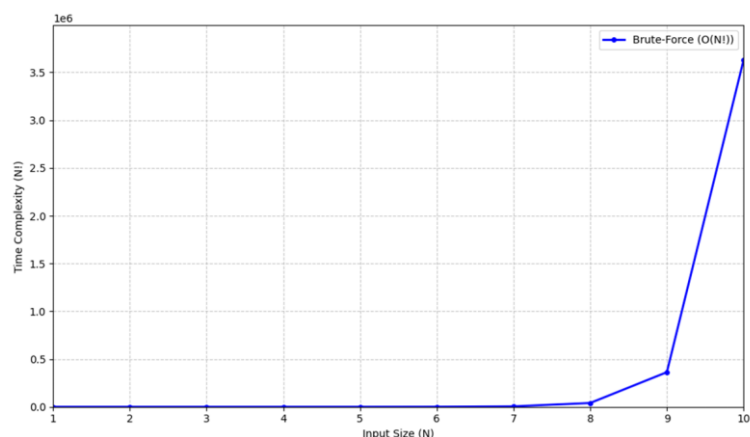


图 15 枚举法时间复杂度随输入规模变化趋势图

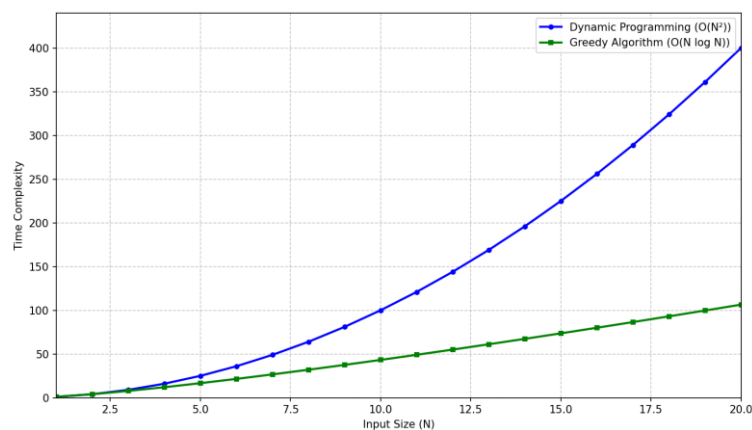


图 16 动态规划与贪心算法时间复杂度随输入规模对比趋势图

从以上图表中可以看到，**枚举法**的时间复杂度随着节点数 N 增大呈现阶乘级增长，导致计算时间迅速增加。**动态规划**的时间复杂度则呈现出平方和对数级增长，增长速度较为平缓，而本问所应用的**贪心算法**的时间复杂度增长最为缓慢。

3.7 本章小结

在问题一模型建立与求解的过程中，本章围绕“最小化最大缓存驻留量”建立统一模型：以驻留量递推与峰值最小化为目标，约束含拓扑依赖、单元互斥与 L0 单驻留。考虑问题具 NP 难性，我们先用三类贪心给出可计算的强基线，再用三种启发式(GA/SA/TS)进行二次优化以增强泛化。

实验表明：在 FlashAttention 与部分 Conv 上，SA/TS 相对贪心降峰明显；在 Matmul 上结构刚性、可交换性弱，三法改进有限。鉴于启发式需要较长算时与参数调度，在赛题时限下我们采用**贪心最优**作为问题一的最终结果，启发式作为补充与能力边界说明。所有算例的 $\max(V_{stay})$ 与调度序列已随文与附录提供。

四、问题二：缓存分配方案单目标设计问题求解

本题的具体求解思路见图 17。

子问题一：基于调度序列的单目标模型

$$\min \sum_{t=1}^T \sum_{b \in B} \alpha_b \cdot \text{size}_b \cdot w_{b,t} \quad r_{b,t} = 0 \forall t < a_b \text{ 或 } t > f_b$$

$$r_{b,t} \in \{0,1\} \forall b, t$$

$$\sum_{b: \text{type}_b=c} r_{b,t} \cdot \text{size}_b \leq \text{Cap}_c \forall c \in C, \forall t$$

$$x_b + \text{size}_b \leq x_{b'} + M \cdot y_{bb',t}$$

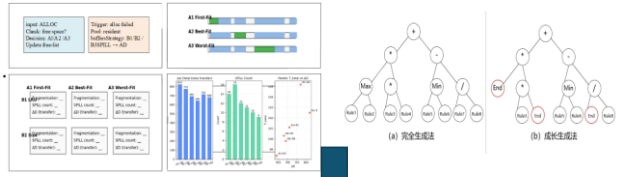
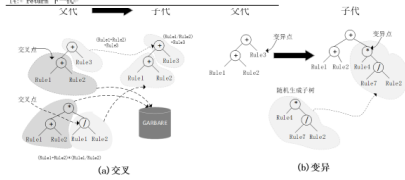
$$x_{b'} + \text{size}_{b'} \leq x_b + M \cdot (1 - y_{bb',t})$$

$$\forall t, \forall b \neq b': \text{type}_b = \text{type}_{b'}, r_{b,t} = r_{b',t} = 1$$

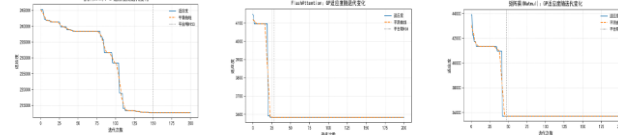
目标函数及约束建模

遗传编程算法代码(Python)
1. 数据预处理及变量初始化
2. 定义父代种群
3. 定义子代种群
4. 定义变异操作
5. 定义交叉操作
6. 定义选择操作
7. 定义适应度函数
8. 定义终止条件
9. 定义主循环
10. 定义输出结果

遗传编程算法

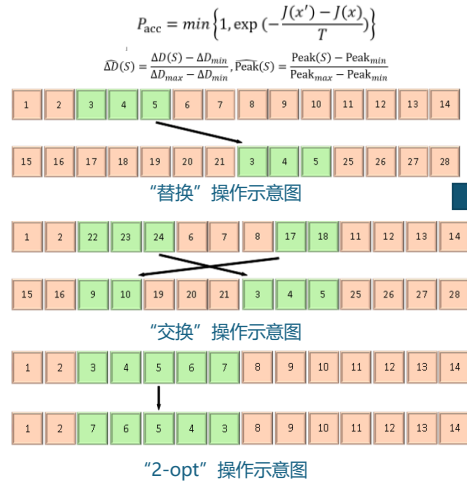


规则编号	规则表达式	含义说明
rule1	-recency	越久未用值越小，优先被换出
rule2	-size	尺寸越小越优先
rule3	copyin_used	被 COPY_IN 用过=0，否则=1，0 更优
rule4	size-mean_size	大于同缓存平均尺寸越多越不利
rule5	total_same_cache	同缓存已分配数量，用于拥挤度参考
rule6	recency	越久未用值越大，倾向 LRU
rule7	size	越大越不利，倾向小块
rule8	unit_cost	被 COPY_IN 用过=1，否则=2，1 更便宜
rule9	size*unit_cost	按成本加权的尺寸，越小越便宜
rule10	recency*size	兼顾未用时长与尺寸的加和
rule11	recency-size	更偏向“久未用且较小”的缓冲
rule12	size-mean_size	与平均尺寸偏差，衡量“异类程度”

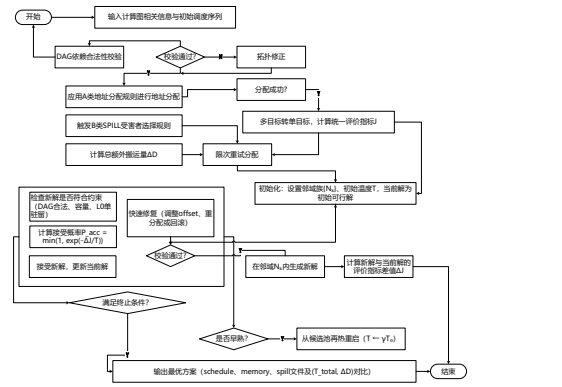


基于调度序列的单目标模型求解

子问题二：双指标融合的加权单目标模型



变邻域模拟退火算法



示例	总额外数据搬运量
Conv_Case0	242072
Conv_Case1	901460
FlashAttention_Case0	6048
FlashAttention_Case1	56042
Matmul_Case0	42876
Matmul_Case1	572694

双指标融合的加权单目标模型求解

示例	总额外数据搬运量
Conv0	230496
Conv1	840904
FlashAttention0	4096
FlashAttention1	33792
Matmul0	41600
Matmul1	518400

图 17 问题二求解思路图

4.1 问题分析与求解思路

问题二分为两个子问题：子问题一要求在问题一求得的调度序列 S 基础上，额外考虑数据缓存区的地址偏移与 SPILL 操作列表，设计一个总额外数据搬运量指标尽可能小的缓存分配方案。子问题二需要考虑问题一调度序列与指标最小化目标难以兼顾时的优化调整方案。针对子问题一，建立基于调度序列的单目标优化模型，采用遗传编程算法进行求解。针对子问题二，建立双指标融合的加权单目标模型，采用变邻域模拟退火算法进行求解。

4.2 核心方法的适用性分析

4.2.1 遗传编程算法的适用性分析

遗传编程算法（GP）是一种利用进化算法的超启发式算法，其基本思想与自然界生物进化理论和遗传的原理类似，是一种自动随机产生搜索程序的方法^[12]。该算法作为一种全新的全局优化搜索算法，具有简单通用、鲁棒性强的特点，它提供了一种基于规则库生成新规则的方案，即根据近似算法构建 GP 规则库，并基于此进行遗传编程，生成新规则。

在问题二中，目标是设计一种使得总额外数据搬运量尽可能小的缓存分配方案。遗传编程算法具有很强的搜索能力，能够在高维的解空间中找到接近最优的解。通过模拟自然选择的过程，GP 算法不断调整缓冲区地址偏移的分配方案，旨在减少缓存碎片化并提高内存资源的利用率，遗传编程算法的架构模型如图 18 所示。

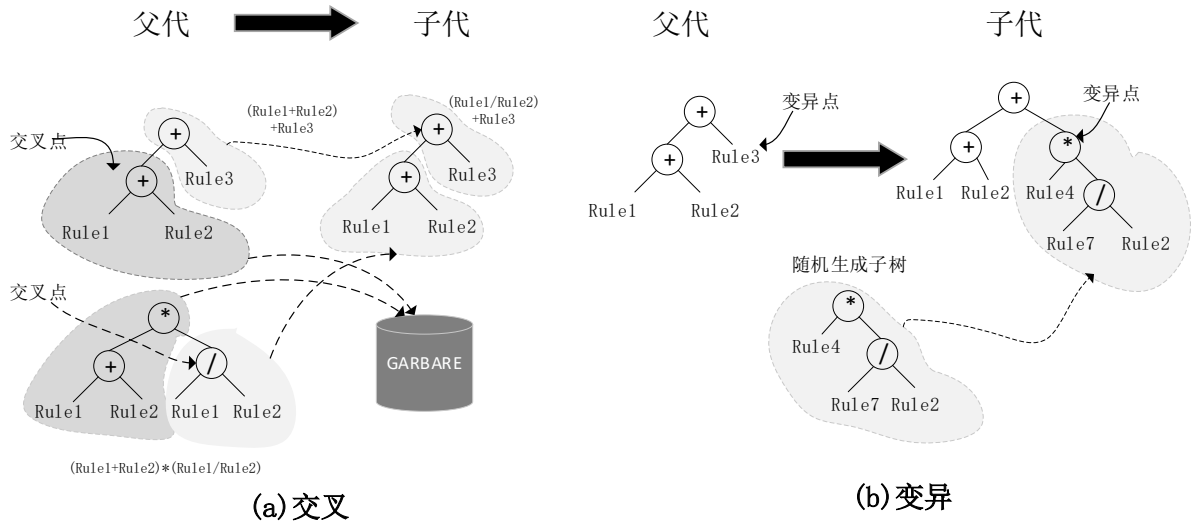


图 18 遗传编程算法架构图

首先，GP 规则库建模模块根据初始化的数据生成通用规则，并简历规则库。之后再根据通用规则生成一系列新规则，然后进行适应度计算，对树形算法程序进行交叉、变异、选择迭代后构建高质量 GP 规则库^[13]。遗传编程算法伪代码如表 11 所示：

表 11 遗传编程算法伪代码

遗传编程算法伪代码	
1:	根据数据集依专家意见初始化通用规则库, 设置如下: 交叉变异概率 p_c, p_m 最大迭代次数 $max_episode$ 种群大小 M
2:	Parents $\leftarrow$ { 通用规则库 }
3:	While 迭代次数 $<$ 最大迭代次数 do
4:	计算种群中每个父代的适应度
5:	Children $\leftarrow$ 空集
6:	While $ Children < Parents $ do
7:	根据适应度大小依概率选出一对交叉的父代
8:	父代交叉生成子代 $c1$ 和 $c2$
9:	Children $\leftarrow$ Children 并上 $\{c1, c2\}$
10:	end while
11:	一些子代随机变异
12:	Parents $\leftarrow$ Children
13:	end while
14:	return 下一代

4.2.2 变邻域模拟退火算法的适用性分析

传统的模拟退火算法 (SA) 通过逐步降低温度来避免陷入局部最优解, 但有时其扰动方式较为单一, 可能导致收敛速度较慢, 或者无法充分探索解空间。而变邻域模拟退火算法 (VN_SA) 通过引入变邻域搜索 (VNS) 的思想, 在搜索过程中不断改变邻域结构, 即每次搜索的扰动方式发生变化。这种机制使得 VN_SA 能够在更广泛的解空间内进行搜索, 避免了局部最优解的困境, 提高了找到全局最优解的概率。因此, 当以问题 1 中的调度序列为基础进行缓存分配时, VN_SA 能够更有效地调整调度序列, 寻找最佳的缓存分配方案, 最小化总额外数据搬运量。

在本问中, 缓存分配方案的优化不仅依赖于调度序列, 还涉及如何在满足硬件限制的情况下, 合理安排缓冲区的数据驻留顺序。使用问题 1 中的调度序列作为初始解可能无法最小化总额外数据搬运量, 尤其是在缓存容量受限的情况下。VN_SA 可以通过变邻域操作, 在调整调度序列时对缓存分配进行优化。具体而言, VN_SA 能够灵活地选择不同的邻域结构 (如改变缓存分配的顺序、调整数据搬运的时机等), 从而探索更多的调度序列和缓存分配组合, 寻找可能更优的调度方案。

4.3 子问题一: 基于调度序列的单目标优化模型

子问题一要求在问题一求得的调度序列 S 基础上, 设计一个总额外数据搬运量指标尽可能小的缓存分配方案。我们选用的模型是遗传编程算法, 原因是本问题的决策空间本质上是离散的策略空间, 既不可微又伴随复杂约束, 如 DAG 依赖、容量与块对齐、不可驱逐集合等, 直接在序列上做局部搜索容易陷入早收敛且难以迁移; 而 GP 以可解释的程序树来表达策略函数, 能把“近期性、重载代价、尺寸/碎片”等领域特征通过 $+$ $-$ $\times$ $\div$ 、 $\min/\max$ 等算子自动组合, 既能在极大、非线性的策略空间里做全局式探索, 又能输出可读可复用的规则, 便于跨不同计算图泛化与对比; 更重要的是, GP 的“适应度”天然就是单个标量目标, 因此它与单目标优化模型的框架完全一致: 我们只需把仿真得到的指

标作为适应度，按选择-交叉-变异迭代，即可在保持单目标一致性的前提下获得更强的搜索能力与更好的可解释性。

(1) 目标函数：最小化总额外数据搬运量

$$\min \sum_{t=1}^T \sum_{b \in \mathcal{B}} \alpha_b \cdot \text{size}_b \cdot w_{b,t} \quad (4-1)$$

(2) 决策变量：

$x_b \in \mathbb{Z} \geq 0$: 缓冲区 b 在其缓存类型中的首地址（偏移）。

$r_{b,t} \in \{0,1\}$: 在步骤 t 时 b 是否驻留在片上缓存(resident)。

$w_{b,t} \in \{0,1\}$: 在步骤 t 是否对 b 发生一次“完整 SPILL 事件”的计数变量。（对应实现：每次选中受害者触发的 SPILL 计费： $\alpha_b \cdot \text{size}_b$ ）。

$y_{ww'}, t \in \{0,1\}$: 在步骤 t 地址不重叠的排序二元。

(3) 约束函数：

(a) 生命周期驻留或可溢出/回填

$$r_{b,t} = 0 \forall t < a_b \text{ 或 } t > f_b \quad (4-2)$$

$$r_{b,t} \in \{0,1\} \forall b, t \quad (4-3)$$

解释：只在 $[a_b, f_b]$ 期间允许驻留，期间允许因 SPILL 变 0，再在使用前通过 SPILL_IN 恢复为 1。

(b) 能力/容量约束（逐时刻）

$$\sum_{b: \text{type}_b = c} r_{b,t} \cdot \text{size}_b \leq \text{Cap}_c \forall c \in \mathcal{C}, \forall t \quad (4-4)$$

若需严格连续地址不重叠（1D 装箱语义），补充“地址冲突消解”：

$$x_b + \text{size}_b \leq x_{b'} + M \cdot y_{bb',t} \quad (4-5)$$

$$x_{b'} + \text{size}_{b'} \leq x_b + M \cdot (1 - y_{bb',t}) \quad (4-6)$$

$$\forall t, \forall b \neq b': \text{type}_b = \text{type}_{b'}, r_{b,t} = r_{b',t} = 1 \quad (4-7)$$

其中 M 为足够大的常数。

(c) 调度与可用性约束

若步骤 t 要使用缓冲区 b （来自节点的 BufS），则必须：

$$r_{b,t} = 1 \quad (4-8)$$

若前一时刻 $r_{b,t-1} = 0$ ，则在 t 之前必须执行 SPILL_IN。

(4) 模型构建：

遗传编程模型的建立是一个逐步优化的过程，通过定义合适的函数集、生成初始种群、并使用遗传操作（选择、交叉、变异）来寻找优化的缓存分配方案。具体的模型构建过程如下：

(1) 设定函数集

本文使用 1 个终止运算符 end 以及 6 个数学运算符 $+, -, *, /, \max, \min$ 构建函数集,其中为避免出现除数出现 0 的情况，特规定当除数为 0 时，结果永远返回被除数，其余情况保留计算结果。

(2) 初始种群生成

根据缓存管理策略，定义了以下两类规则组合，用于生成遗传算法的初始种群：

(a) 地址分配规则 (A 类)：当节点执行到 **ALLOC** 时，如何给定 **Offset**：

A1.首次适应 (**First-Fit**)：从低地址开始扫描空闲块列表，找到第一个足够大的空闲区进行分配。

A2.最佳适应 (**Best-Fit**)：扫描所有空闲块，选择最小的足够空闲区进行分配，减少外部碎片。

A3.最坏适应 (**Worst-Fit**)：选择最大的空闲区进行分配，旨在减少内部碎片。

(b) **SPILL** 触发与对象选择规则 (B 类)：当缓存空间不足（分配新缓冲区时没有足够空闲块），必须触发 **SPILL** 操作，规则用于选择换出对象：

B1.LRU（最近最少使用）：选择最后一次访问时间最远的缓冲区。

B2.Size 优先：选择最大的缓冲区换出，一次释放最多空间。

B3.Cheap-to-Reload：选择重载代价低的缓冲区（小尺寸或不被 **COPY_IN** 使用）。

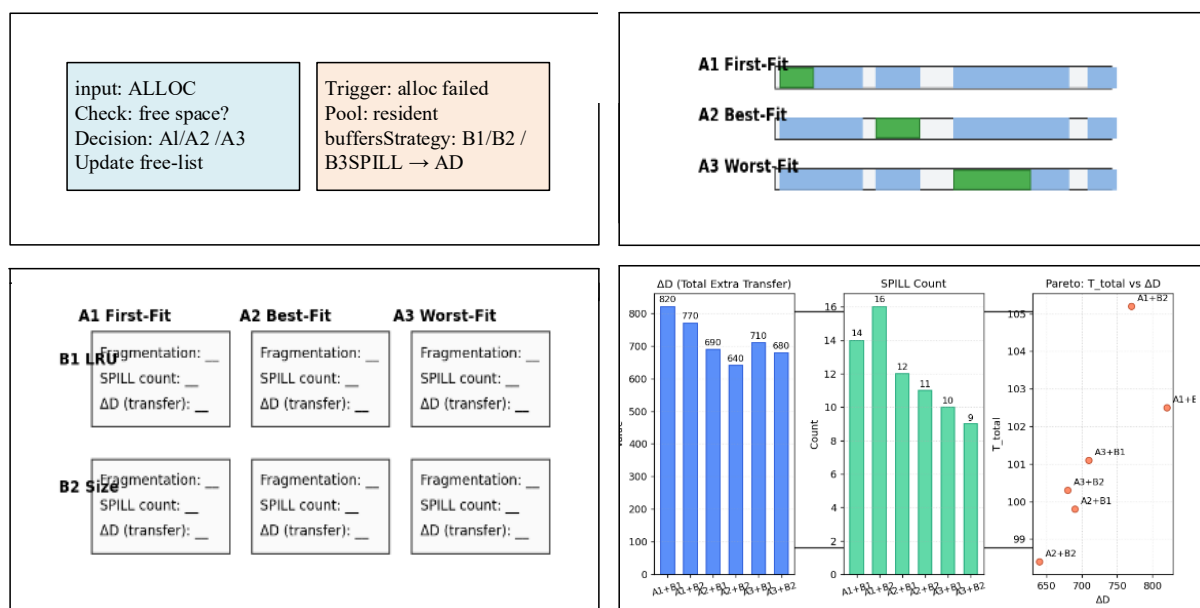


图 19 地址分配与 **SPILL** 规则组合及性能可视化

本模型使用斜坡对半法构建初始种群，设定树的最大深度为 3。其中在斜坡对半法中，一半初始个体采用完全法生成，其余个体采用生长法生成。并且生成的每个个体都用二叉树的结构来表示。在完全法中，个体的所有终端操作都位于树的最大深度处。在生长方法中，树的所有节点都可以从函数集和终止运算符中任意选择，即允许生成的树在任意层选择终结操作，因此生成的树的深度可能小于最大深度允许的深度，过程如图 20 所示。

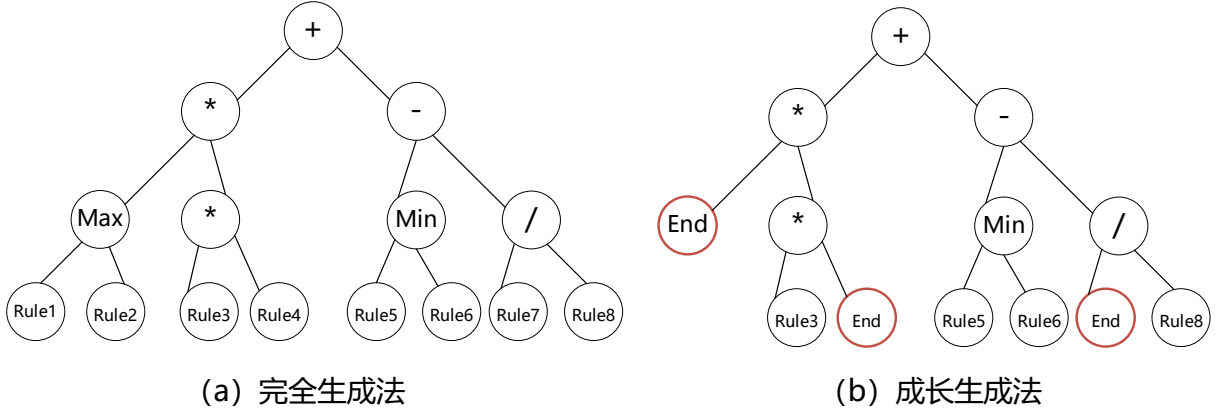


图 20 斜坡对半法

4.4 子问题二：双指标融合的加权单目标优化模型

子问题二考虑问题一求得的调度序列 S 无法充分实现总额外数据搬运量最小化目标的情况，对调度序列生成策略进行优化调整。该问题有两个目标：**最小化缓存容量峰度**和**最小化总额外数据搬运量**的加权值，旨在通过更系统的优化策略进一步提升调度性能。

4.4.1 指标定义

总额外数据搬运量： $\Delta D(S)$ ，指在给定调度序列 S 下，由装载与换出产生的累计搬运量，越小越优。

缓存容量峰度： $Peak(S)$ ，指执行全过程中缓存占用的最大值，越小越优。

4.4.2 归一化与权重设置

为消除量纲影响，采用 Min-Max 归一化对数据进行处理：

$$\widehat{\Delta D}(S) = \frac{\Delta D(S) - \Delta D_{min}}{\Delta D_{max} - \Delta D_{min}}, \widehat{Peak}(S) = \frac{Peak(S) - Peak_{min}}{Peak_{max} - Peak_{min}} \quad (4-9)$$

上下界可由初始解与若干随机扰动解的观测极值给出，给定权重 $w_1, w_2 \geq 0$ 且 $w_1 + w_2 = 1$ ，得到加权单目标式：

$$F(S) = w_1 \widehat{\Delta D}(S) + w_2 \widehat{Peak}(S) \quad (4-10)$$

其中，约束化替代： $F(S) = \widehat{\Delta D}(S) + \lambda \cdot \max\{0, Peak(S) - P_{max}\}$, $\lambda \gg 1$ ，实现“先满足峰度、再降 ΔD ”的词典序偏好。

为了与问题一的求解结果进行衔接，本文将问题一输出的调度序列生成策略作为**起点**：此方案在总额外数据搬运量 ΔD 上已具有较好表现。为在同一框架下兼顾带宽与容量两类资源，我们将 ΔD 与 $Peak$ 进行归一化后加权融合为单一目标函数，并在既有的变邻域搜索+模拟退火单目标优化框架中直接替换适应度函数即可，无需更换求解器，因而实现成本低、方法稳定。此外，相较于需要维护非支配排序与群体多样性的多目标进化，在相同计算预算下，加权单目标更易收敛、对样本量要求更低，能够在当前算力条件下更可靠地得到满足工程偏好的折中解。

4.4.3 变邻域模拟退火算法模型的建立

在问题一的优化模型建立过程中，我们采用了多种启发式算法来解决缓存分配和调度顺序问题，其中模拟退火算法具有全局搜索能力和跳出局部最优解的特点，适合本问

数据类型的求解。具体而言，模拟退火算法能够通过逐步降温的方式控制解的随机性，避免陷入局部最优解，从而提供一种有效的全局搜索机制。然而，传统的模拟退火算法在某些情况下可能会因其固定邻域结构和冷却速度而导致收敛速度较慢，尤其是在复杂的优化问题中。子问题二需要求解缓存容量峰度和最小化总额外数据搬运量的最小加权值，这给模拟退火算法带来了挑战。为了进一步提升算法的效率，我们对模拟退火算法进行了改进，采用了**变邻域模拟退火算法**。变邻域模拟退火算法结合了模拟退火和邻域结构变化的优势，在解决本问题时能够显著提高搜索的多样性与灵活性。相比于传统的模拟退火算法，变邻域模拟退火算法在搜索空间的跳跃性和跳出局部最优解的能力方面更加出色。通过动态切换邻域结构，算法能够根据当前解的特点选择合适的邻域进行搜索，从而避免了局部极小值的困扰，并有效提升了解的质量。

变邻域模拟退火算法的核心参数包括：

设定邻域族 $\{\mathcal{N}_k\}_{k=1}^K$ 。每个邻域结构代表一种解的变化方式。

设定温度 T 。通过几何冷却法逐渐降低温度，以减少探索的随机性，提升算法的收敛速度。温度更新规则为：

$$T \leftarrow \alpha T (0 < \alpha < 1) \quad (4-11)$$

接受概率：为了平衡全局搜索和局部搜索的能力，本模型采用以下接受概率：

$$P_{\text{acc}} = \min \left\{ 1, \exp \left(-\frac{J(x') - J(x)}{T} \right) \right\} \quad (4-12)$$

本模型核心步骤包括：

步骤一：建模与单目标化

载入计算图的节点/边与初始调度序列，完成依赖合法性（DAG）校验并在必要时进行拓扑修正以获得可行初解；建立内存层级模型，采用 A 类地址分配规则完成（层级，offset）选择；当分配失败时触发 B 类 SPILL 受害者选择（LRU/Size/代价导向），计算由换入/换出引起的总额外搬运量 ΔD ，并在限次重试内完成分配，从而将调度-地址-SPILL 映射为可评估的运行轨迹；最后将多目标转为单目标，采用归一化加权和 $J = \lambda_1 T_{\text{total}} + \lambda_2 \Delta D$ 作为统一评价指标。

步骤二：变邻域与模拟退火搜索

围绕当前可行解构造由近及远的邻域族：N1 交换（相邻/可交换节点对）、N2 插入移动、N3 2-opt 段反转、N4 块移动或“序列+地址+规则”的复合强扰动；所有邻域操作必须保持 DAG 合法、容量与 L0 单驻留约束，必要时调用快速修复（回退越界 offset、重分配或回滚）；在给定邻域 $\mathcal{N}_k$ 内运行模拟退火局部搜索，采用几何降温 $T \leftarrow \alpha T$ 与 Metropolis 接受准则 $\min \{1, \exp [-(J(x') - J(x))/T]\}$ 控制对劣解的概率接受；若解被接受（含非严格改进），则重置到最近邻域继续细化，否则提升到更远邻域扩大扰动半径；全过程持续记录并更新历史最优解及其 $J, T_{\text{total}}, \Delta D$ 。

N1)“替换”操作。替换是指将某段连续的任务节点进行单链节点替换，随机产生个任务节点，将此节点插入某个位置之后替换对应数量的节点编号数量，如图 21 所示。



图 21 “替换”操作示意图

N2) “交换”操作。交换是指在单链内部选择两条节点段进行部分位置互换，生成全新方案，如图 22 所示。

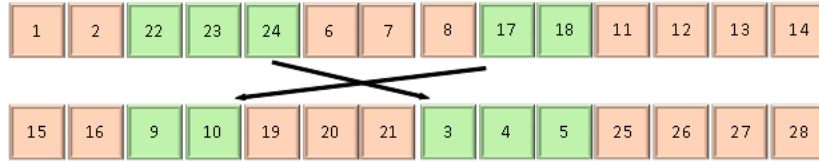


图 22 “交换”操作示意图

N3) “2-opt”操作。2-opt 是指在单链内部选择一段节点链，将其翻转后得到新的方案，若翻转后的适应度函数小于翻转前，则进行保留；否则，取消翻转，如图 23 所示。

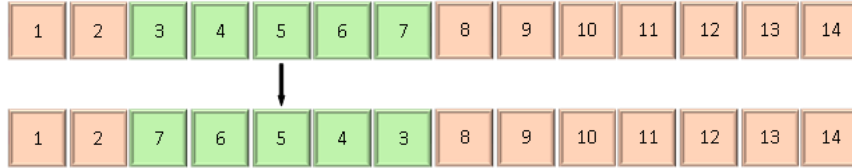


图 23 “2-opt”操作示意图

步骤三：判停、再热与结果输出

本算法以最大迭代次数、最低温度阈值或连续无改进轮数为终止条件；若出现早熟迹象，可从候选池再热重启 ($T \leftarrow \gamma T_0$) 继续搜索；最终输出最优方案的 schedule.txt、memory.txt、spill.txt，并给出($T_{\text{total}}, \Delta D$)与基线及子问题一结果的对比，用表格/曲线呈现改进幅度与稳定性。

步骤四：更新种群

计算所有新个体的适应度值，并与初始解集合的适应度值进行对比。

步骤五：判断迭代次数是否达到最大

若达到则输出最优解，即得到总代价最小的任务分配结果；否则返回步骤一。

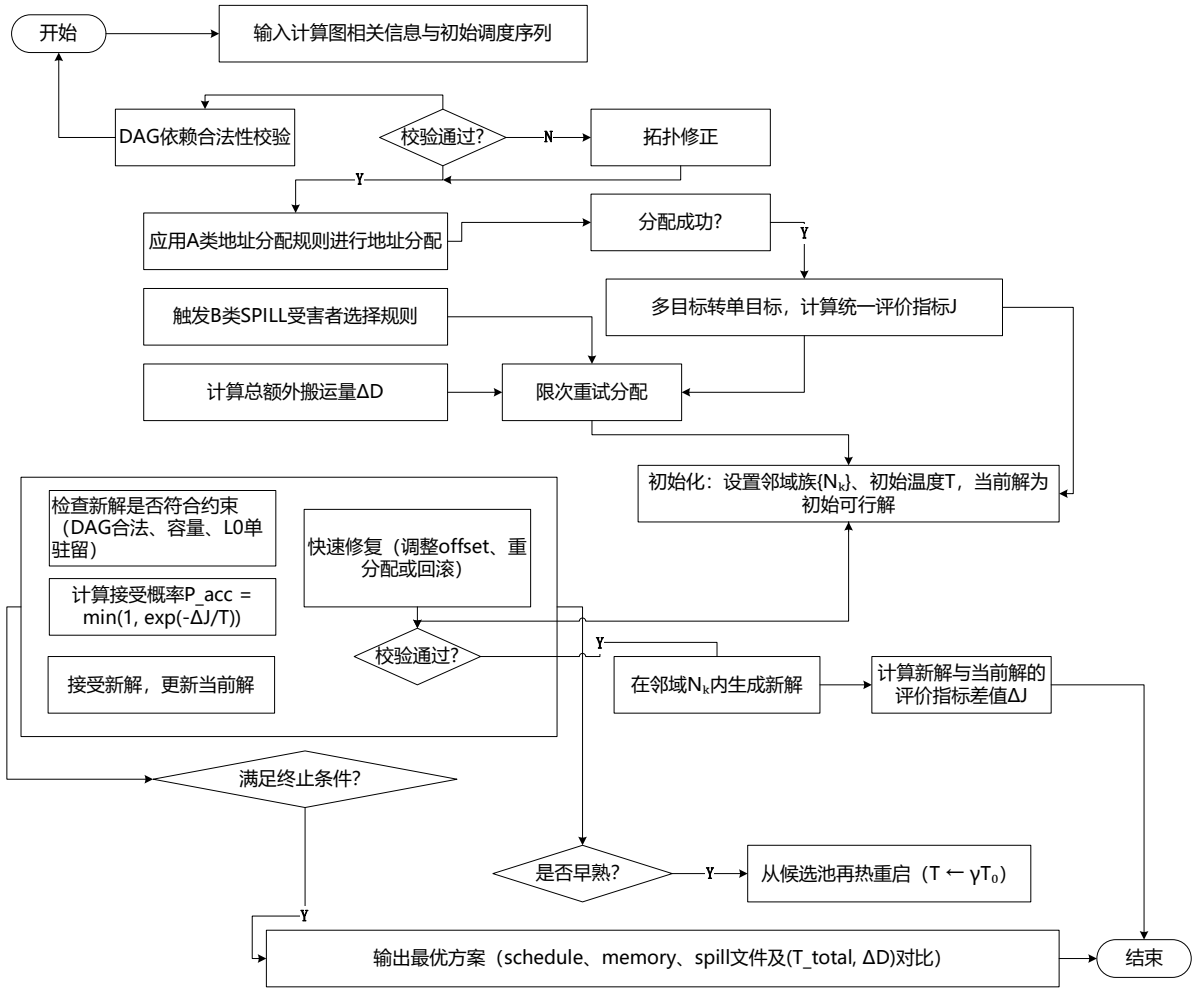


图 24 变邻域模拟退火流程图

在变邻域模拟退火算法中，为确保解的有效性，采用了一种**纠错机制**：当扰动操作后产生无效解时，50%的几率保持原方案，另 50%的几率重新选择方案。这样既避免了不符合约束条件的解，同时确保了算法能够探索**有效的解空间**。这使得本算法能够适应不同算子不同数据结构的情况，具有一定的模型求解通用性。

4.5 基于演化计算与局部搜索的模型求解

4.5.1 子问题一：基于调度序列的单目标优化模型求解

子问题一的模型求解环节基于前文得到的调度序列，对其进行单目标优化模型求解，而这一求解过程对于后续从减少缓存碎片角度优化最终结果至关重要，下面将展开具体分析。

(1) 从减少缓存碎片的角度分析如何优化最终结果

在问题二中，如果依赖单一分配策略对各计算图进行缓存分配可能会造成大量缓存碎片，从而增加 SPILL。因此从三个层面入手优化：规则改进、调度结合以及全局优化；以本模型的地址分配（A 类）为例，可进行如下调整：

(a) 首次适应（First-Fit）改进

传统 First-Fit 容易留下零散的小空闲区。因此我们选择在每次分配后，对空闲区进行合并整理（**coalescing**），避免过多小碎片。可配合一个“最小阈值剔除”机制，即若产生的空闲区小于某阈值，就与相邻块强制合并。

(b) 最佳适应 (Best-Fit) 改进

只挑选“刚刚好”的小空闲区时，会产生大量小碎片，因此本文通过在候选区间相同的情况下，优先选择更靠前/连续可扩展的区间，为后续大缓冲区留出空间。

(c) 最坏适应 (Worst-Fit) 改进

每次都切分最大空闲区，容易破坏大块内存。本文通过引入限制因子，避免过度切割大块。

(2) GP 受害者选择策略演化

由于影响最大且可局部决策的是受害者选择：每次仅在同一缓存内对少量候选缓冲区排序，易于构造特征并在线打分，进化效率高；而地址分配（A 类）要在多空闲区全局权衡，强依赖碎片演化，搜索面更大、收敛慢，且已有 first/best/worst 能覆盖主要碎片取舍，先用穷举/选择即可得到稳定收益。因此本模型对 A 类使用简单搜索挑最优（在 first/best/worst 中选），把学习能力集中给 B 类以获得更大边际收益与更稳的泛化。

具体生成了以下 12 条 SPILL 触发与对象选择规则，具体如表 12 所示：

表 12 SPILL 触发与对象选择规则

规则编号	规则表达式	含义说明
rule1	-recency	越久未用值越小，优先被换出
rule2	-size	尺寸越小越优先
rule3	copyin_used	是否被 COPY_IN 用过
rule4	size-mean_size	大于同缓存平均尺寸越多越不利
rule5	total_same_cache	同缓存已分配数量，用于拥挤度参考
rule6	recency	越久未用数值越大，倾向 LRU
rule7	size	越大越不利，倾向小块
rule8	unit_cost	被 COPY_IN 用过=1，否则=2，1 更便宜
rule9	size×unit_cost	按成本加权的尺寸，越小越便宜
rule10	recency+size	兼顾未用时长与尺寸的加和
rule11	recency-size	更偏向“久未用且较小”的缓冲
rule12	size-mean_size	与平均尺寸偏差，衡量“异类程度”

本模型以图 18 的方式对生成的新种群进行选择交叉变异处理，同时为保证泛化性与通用性，尽可能避免针对某一特定算子的计算图结构进行优化，本模型分别选取了任务中的三种计算图数据集 Matmul_Case0、FlashAttention_Case0、Conv_Case0，以总额外数据搬运量为适应度值，寻找到各计算图结构下的最优 GP 规则以进行缓存分配与换入换出方案的求解。

为便于复现与评估，下面给出各计算图在我们方法下得到的最终地址分配与 SPILL 策略组合，如表 13 所示。

表 13 示例算子缓存分配规则表

任务名	best_rule	gp_rule
Conv_Case0	('best_fit', 'gp')	$((rule6 \max rule11) - ((rule10 / rule8) / rule10)) + (rule9 / (rule5 \min rule9)))$
Flash_Case0	('best_fit', 'gp')	$(rule7 / ((rule8 \max (rule6 * rule2)) \min rule11))$
Mat_Case0	('first_fit', 'gp')	$((rule8 / rule1) / (((rule8 / rule7) / (rule3 * rule6)) / (rule8 * rule1)) * rule6)) + (rule11 * (rule11 \max (rule8 - rule8)))$

为避免规则在不同计算图上“水土不服”，我们采用遗传编程 (GP) 分别对 Matmul、FlashAttention 与 Conv 三类结构独立训练规则。优化目标为最小化总额外数据搬运量，因此适应度等价于目标值的负值。图 25 展示了三种结构上的 GP 适应度随迭代变化的

情况,可见前两者在早期即快速收敛,而卷积结构存在更明显的阶梯式下降,收敛更晚。收敛时刻对应的最优规则即被固定,并据此生成后续的缓存分配与换入/换出方案。

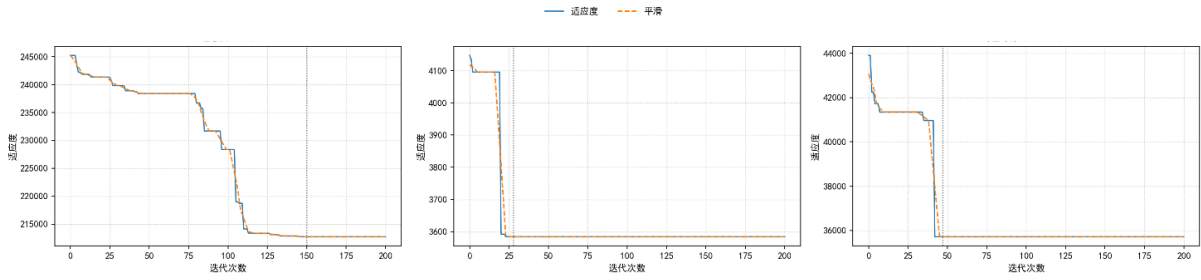


图 25 三种计算图结构的 GP 适应度收敛曲线

蓝线为当代最优个体,橙线为种群均值。Matmul 与 FlashAttention 在早期迅速收敛,Conv 的收敛更为缓慢。曲线进入平台期时的最优解即被选为该结构的最优 GP 规则,用于生成最终的缓存分配与换入/换出方案。

基于上述遗传编程演化得到的最优规则,本模型对三类算子计算图分别建立模型进行求解,并统计了求得的各示例的总额外搬运量,结果如表 14 所示。

表 14 各算例总额外搬运量结果表

任务名	分配规则	总额外数据搬运量
Conv_Case0	first_fit	230496
Conv_Case0	best_fit	275212
Conv_Case0	worst_fit	269692
Conv_Case1	first_fit	840904
Conv_Case1	best_fit	850164
Conv_Case1	worst_fit	867010
FlashAttention_Case0	first_fit	5984
FlashAttention_Case0	best_fit	4096
FlashAttention_Case0	worst_fit	5720
FlashAttention_Case1	first_fit	62036
FlashAttention_Case1	best_fit	61184
FlashAttention_Case1	worst_fit	54720
Matmul_Case0	first_fit	41600
Matmul_Case0	best_fit	54400
Matmul_Case0	worst_fit	54400
Matmul_Case1	first_fit	518400
Matmul_Case1	best_fit	568400
Matmul_Case1	worst_fit	568400

由结果表可知,示例 Conv0、Conv1、FlashAttention0、FlashAttention1、Matmul0、Matmul1 的总额外数据搬运量分别为 **230496**、**840904**、**4096**、**33792**、**41600**、**518400**。而各示例的最优结果的分配原则分别是 first_fit、first_fit、best_fit、worst_fit、first_fit、first_fit,这表明没有一种地址分配规则在所有计算图上均表现最佳,也印证出本模型采用遗传编程为不同计算图自适应选择最优规则组合的策略能灵活适应不同算子类型,具有一定的模型通用性和泛化性。

4.5.2 子问题二：双指标融合的加权单目标优化模型求解

本问题的目标是优化最小化缓存容量峰度和最小化总额外数据搬运量的加权值，我们使用变邻域模拟退火算法进行求解。通过对归一化后的指标按权重相加，形成最终的单一优化目标函数 $F(S)$ ，其中 $F(S)$ 越小表示综合性能越优：

$$F(S) = w_1 \cdot \widehat{Peak}(S) + w_2 \cdot \widehat{\Delta D}(S) \quad (4-13)$$

考虑到缓存容量峰值与总额外数据搬运量的数据维度不同，取 $w_1=0.8$ ， $w_2=0.2$ 。变邻域模拟退火算法初始参数设定为：初始温度 T 设为 50，最大退火次数 $gen_{\max}$ 为 500，最大迭代次数 $Iter_{\max}$ 为 500。

通过上述变邻域模拟退火算法对加权单目标模型进行求解，最终得到了在缓存容量峰值与总额外数据搬运量之间取得良好平衡的解，结果如表 15 所示：

表 15 变邻域模拟退火算法求解结果表

任务名	总额外数据搬运量
Conv_Case0	242072
Conv_Case1	901460
FlashAttention_Case0	6048
FlashAttention_Case1	56042
Matmul_Case0	42876
Matmul_Case1	572694

4.5.3 求解结果总结

为避免针对单一算子结构过拟合，在子问题一中我们采用两阶段策略：首先在 Matmul、FlashAttention 与 Conv 三类计算图上分别用遗传编程算法搜索缓存管理规则（地址分配 A 与换出选择 B），以最小化总额外数据搬运量为目标进行训练；随后在固定最优规则的前提下，利用真实访问轨迹进行地址分配与换入/换出仿真，得到最终的可执行方案。在子问题二中，我们采用了变邻域模拟退火算法进行求解。该算法在求解过程中首先从初始解开始，通过逐步改变邻域结构进行解空间探索，并通过温度控制逐渐减小搜索的随机性，最终收敛到较优解。在求解缓存分配问题时，变邻域模拟退火算法能够有效处理多维度约束条件，并通过灵活的邻域调整来优化总额外数据搬运量。

通过对子问题一和子问题二的总额外数据搬运量求解结果进行对比，发现各算子最低的总额外数据搬运量均来自子问题一的求解结果，这可能是因为遗传编程算法在搜索解空间时能够进行全局搜索，通过交叉和变异操作保持了解的多样性，从而更有效地找到最优解。而变邻域模拟退火算法虽然具有较好的局部搜索能力，但由于邻域结构切换的随机性，可能导致搜索过程过早收敛，未能探索到更加优化的解。本问求解结果如表 16 所示：

表 16 问题二求解结果表

任务名	总额外数据搬运量
Conv0	230496
Conv1	840904
FlashAttention0	4096
FlashAttention1	33792
Matmul0	41600
Matmul1	518400

4.6 本章小结

本问题在问题一给定的调度序列上考虑求解，首先以最小化额外数据搬运量为目标，采用基于遗传程序设计的规则进化来优化地址分配与 SPILL 选择。该方法在六个算例上稳定给出最小搬运量，但也显示不同算子最优规则并不一致的情况。随后，针对降低峰值占用与减少搬运量可能冲突的情形，构建容量峰值—搬运量加权目标，并在变邻域模拟退火框架下求解，得出总额外数据搬运量优化结果。

五、 问题三：综合性能优化策略设计问题求解

问题三求解思路图见图 26。

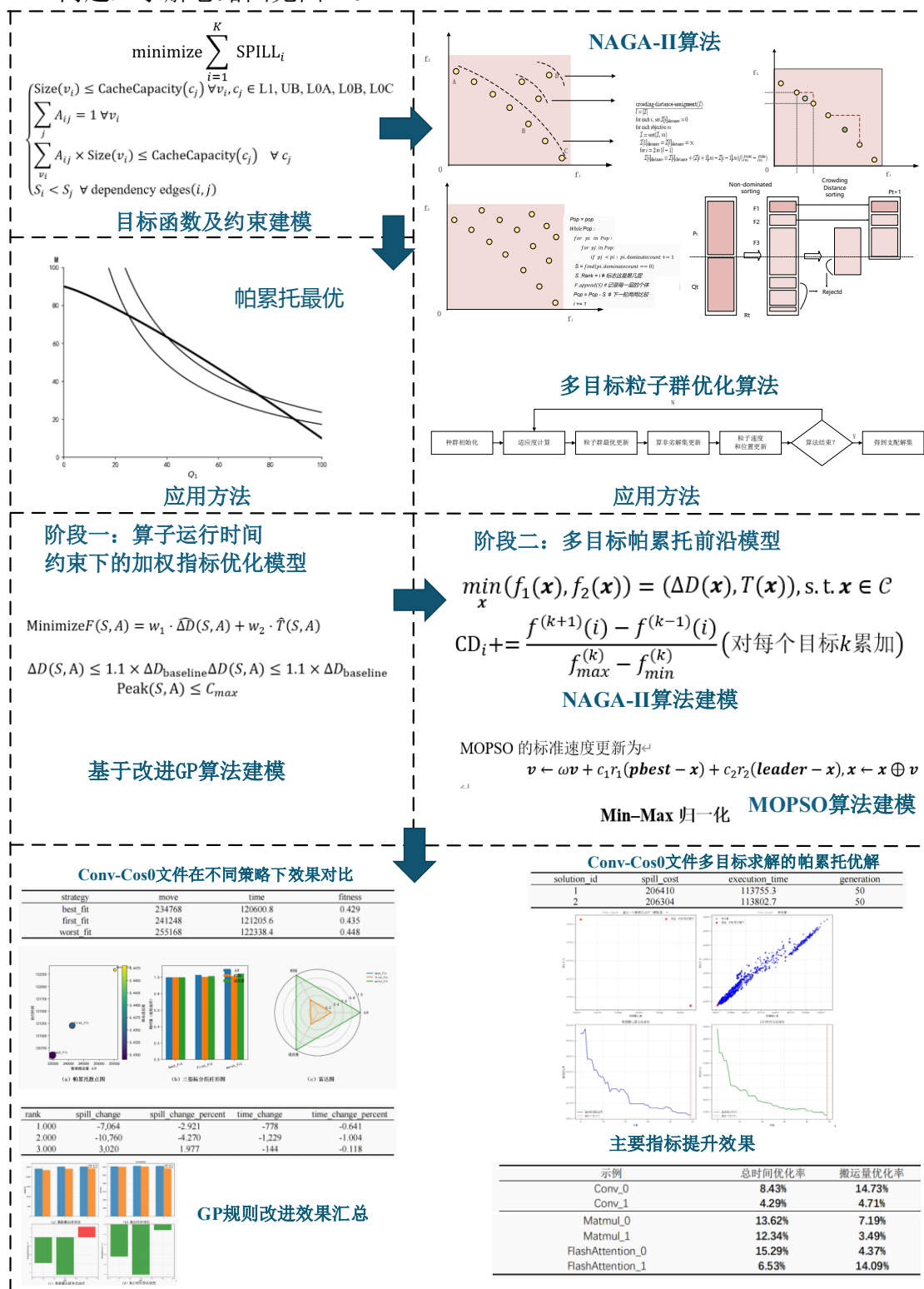


图 26 问题三求解思路图

5.1 问题分析与求解思路

问题三（综合性能优化策略设计问题）要求在前两问的基础上，综合考虑**总额外数据搬运量**和**算子总运行时间**，设计调度序列的性能优化策略。问题三分为两个阶段，阶段一考虑仅优化总运行时间指标，考虑总额外数据搬运量最多**增加 10%**、调度顺序不变的情况下，改变优化缓存分配方案，使用 GP 改进算法求解得到使两项指标加权值最低的方案。阶段二考虑联合优化两项指标，并同时改进**调度顺序、缓存分配方案**，通过 **NAGA-II 算法、多粒子群优化算法**，求得所有多目标帕累托最优解，最后通过比较各任务下帕累托解的**性能优化指标**，得到本题最佳结果。

5.2 策略优化核心方法及适用性分析

5.2.1 GP 改进算法

GP 改进不是再在“整条序列”上反复搜索，而是先学一条可迁移、可解释的选点规则，把 EarlyFree/LateAlloc/MinPeak 的优点在规则层面融合，并在峰值窗口动态加压；部署时按该规则在线出序，既**保留贪心的速度**，又**降低峰值驻留**，并能作为 GA/TS 的高质量起点进一步提升。

(1) 语法/特征定义：构造函数集（if-then-else、min/max、排序、argmin 等）与终结符集合（Vt、 ΔD 、重叠度、ALLOC/FREE 比例、Size、入/出度、L0 冲突指示等）。

(2) 个体表示：以表达式树表示选择规则 $R(\cdot)$ ，作用于就绪集为每个候选节点给出打分排序。

(3) 适应度评估：将 R 应用于整图得到序列 S_R ，以

$$\text{Cost}(R) = V_{\max}(S_R) + \lambda \Pi(S_R) + \beta \text{size}(R) \quad (5-1)$$

其中， Π 为违约度， size 为规则复杂度），适应度取 $\text{Fit} = 1/(1 + \text{Cost})$

(4) 进化与可行性保障：采用子树交叉与子树变异并配合精英保留；每次生成新规则后先经就绪集过滤，若产生不可行序列则由修复器最小扰动纠正。

(5) 选择与早停：基于验证集表现与复杂度惩罚选取最优规则 R^* ；满足代数上限或改进停滞时终止。

(6) 部署与复用：在新计算图上直接用 R^* 在线产生拓扑可行序列，或将其作为 GA/TS 的高质量初解以缩短迭代时间。

5.2.2 NAGA-II 算法

NAGA-II 算法得到的是帕累托最优解集。帕累托最优（Pareto Optimality）是多目标优化理论中的一个核心概念，用于描述资源分配的一种理想状态^[15]。其定义为当系统达到帕累托最优时，任何为了改善某一目标的尝试都必然导致至少一个其他目标性能的下降，数学表示上为设 x^* 为决策空间的一个解，对于决策空间的任意 x 都有 $f_k(x^*) \leq f_k(x)$ ，则称 x^* 为帕累托最优解。所有帕累托最优解的集合构成帕累托最优解集（Pareto Optimal Set）。帕累托最优解集在目标函数空间中的映射称为帕累托前沿。

调度算法的性能优化策略问题属于 NP-hard 问题，通常难以在多项式时间内找到完整的帕累托前沿，因此多采用启发式算法求解。非支配排序遗传算法（NAGA-II 算法）^[8]是一种基于帕累托最优概念设计的改进遗传算法，它采用了快速非支配排序和拥挤度比较技术，在性能和计算时间上表现优秀，适合应用于核内调度处理问题^[16]。此外，算法结构简单且易于实现，过迭代式的搜索逐步逼近 Pareto 前沿，不依赖于问题的具体数学形式，具有广泛的适用性，适合本题设计具有良好通用性和泛化性模型的要求。NAGA-II 算法的关键思想如下：

(1) 支配 (Dominate)

算法会根据每个解在所有目标函数上的表现，将其进行分层。第一层的解都是非支配解，这意味着它们在所有目标上都优于或至少等于其他任何解。第二层则包含所有被第一层支配但相互之间非支配的解，以此类推。这种分层机制使算法能够明确区分解的优劣，并优先保留位于更高层级的解。

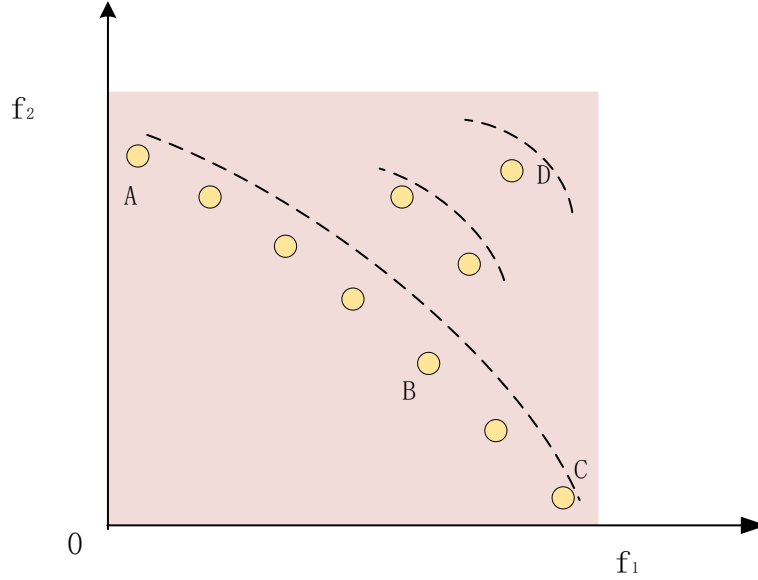


图 27 NAGA-II 算法支配图示

由图 27 可以得知， $B < D$ ， $A < D$ ，但是 A 与 B 之间无法互相支配，因此我们可以称 A、B 为非支配解。

(2) 拥挤距离计算 (CrowdingDistance)

拥挤距离计算 (CrowdingDistance)，这使得非支配解之间，也可以将进行大小比较。为了保持种群的多样性，防止算法过早收敛，NSGA-II 会对同一层级的解计算其拥挤度。拥挤度衡量了一个解周围空间的密集程度。拥挤度越大的解，表明其周围的解越稀疏，因此更值得保留。通过这种方式，算法鼓励在帕累托前沿上广泛分布的解，从而探索解空间的不同区域。

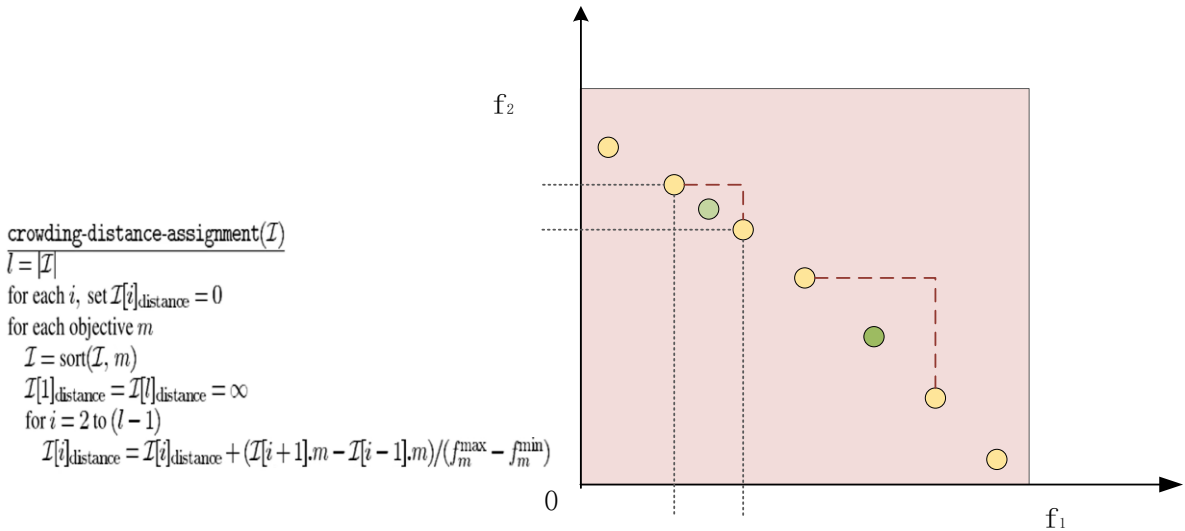


图 28 NAGA-II 算法拥挤距离计算示意图

(3) 快速非支配排序(FastNon-dominatedSort)

种群内每个个体 p 有两个参数 n_p 和 S_p ，其中 n_p 为种群中支配个体 p 的个体数， S_p 为种群中被个体 p 支配的个体集合。通过遍历整个种群，获得每个个体的 n_p 和 S_p 。

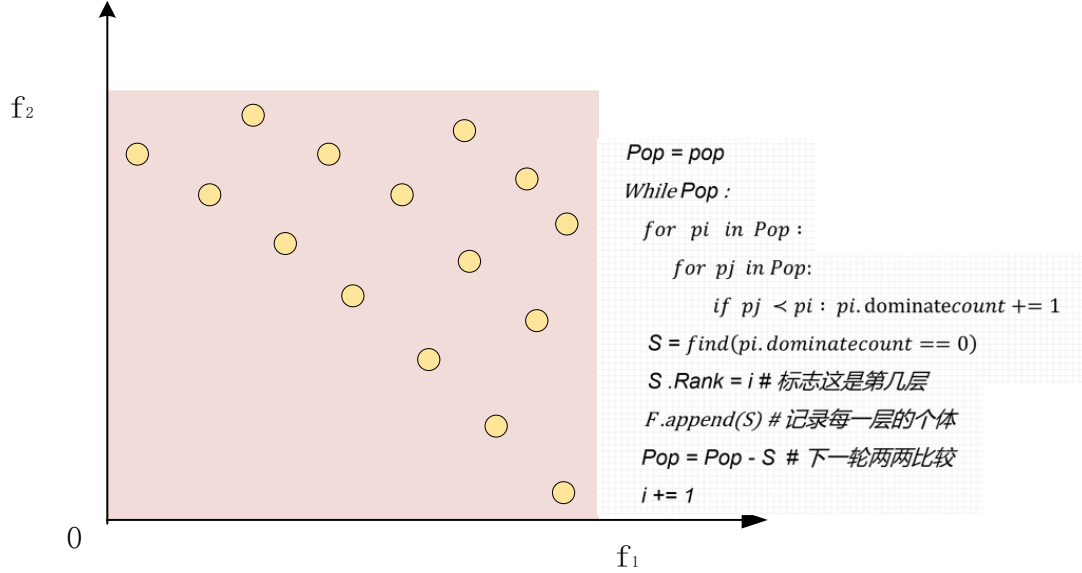


图 29 NAGA-II 算法快速非支配排序示意图

快速非支配排序算法的主要步骤为：Step1：对于种群中 $n_p = 0$ 的个体，保存在当前集合 F_1 中；Step2：对于当前集合 F_1 中的个体 i ，其所支配的个体集合为 S_i ，遍历 S_i 中的每个个体 p ，执行 $n_p = n_p - 1$ ，如果 $n_p = 0$ 则将个体 i 保存在集合 H 中；Step3：记 F_1 中得到的个体为第一个非支配层的个体，并以 H 作为当前集合，重复上述操作，直到整个种群被分级。

(4) 精英保留策略

NSGA-II 将当前父代种群和通过遗传操作（交叉和变异）生成的新子代种群合并。然后，对合并后的种群进行非支配排序和拥挤度计算，从中选出最好的个体来组成新的父代。这个策略确保了每一代中最优秀的解都能被保留下来，防止它们在迭代过程中丢失，从而保证了算法的收敛性。

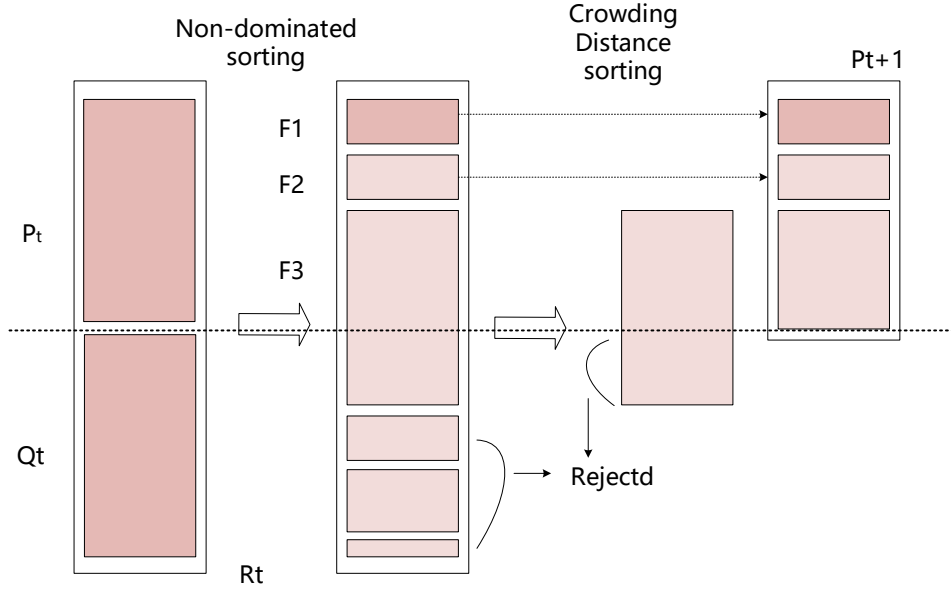


图 30 精英保留策略示意图

5.2.3 多目标粒子群优化算法

多目标粒子群优化算法（Multi-Objective Particle Swarm Optimization, MOPSO）是将传统的粒子群优化算法（PSO）扩展到多目标优化问题中的一种算法^{[17][18]}。MOPSO 算法通过外部存档和 Pareto 支配关系来处理多目标优化问题。每个粒子在搜索空间中移动，其位置和速度根据个人最佳位置和全局最佳位置进行更新。全局最佳位置是从外部存档中选择的非支配解之一。多目标粒子群优化算法流程如图 31 所示：

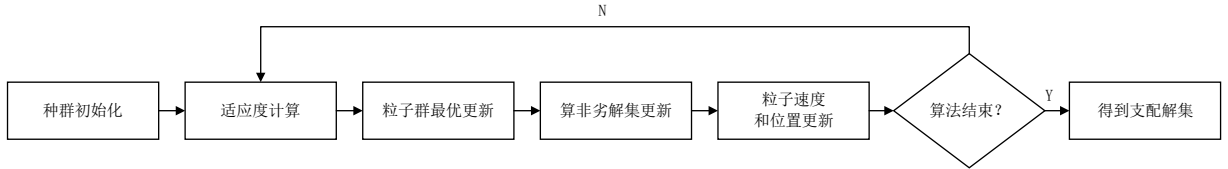


图 31 PSO 算法流程图

多目标粒子群优化算法中，粒子更新公式如下：

$$V^{k+1} = \omega V^k + c_1 r_1 (P_{id}^k - X^k) + c_2 r_2 (P_{gd}^k - X^k) \quad (5-2)$$

其中， ω 为惯性权重， r_1 和 r_2 为分布在 $[0,1]$ 区间的随机数， k 为当前迭代次数， P_{id}^k 为个体最优粒子群位置， P_{gd}^k 为全局最优粒子群位置， c_1 和 c_2 为常数， V 为粒子速度， X 为粒子位置。

5.3 基于多阶段双目标的协同优化模型构建

5.3.1 阶段一：总额外数据搬运量约束下的加权指标优化模型

阶段一要求在维持现有调度序列的基础上，通过优化缓存分配方案，实现在总额外数据搬运量不显著增加的条件下，有效降低算子总运行时间 $T(S)$ 。为此，我们建立了一个带约束的加权单目标优化模型。

本阶段的决策变量为缓存分配方案A，包括地址分配规则和 SPILL 受害者选择规则。方案A共同决定了数据在缓存中的布局和换入换出行为。

本模型的目标函数需联合考虑总额外数据搬运量 $\Delta D(S, A)$ 和算子总运行时间 $T(S, A)$ ，由此构建如下加权指标作为最小化目标：

$$\text{Minimize } F(S, A) = w_1 \cdot \widehat{\Delta D}(S, A) + w_2 \cdot \widehat{T}(S, A) \quad (5-3)$$

其中 $\widehat{\Delta D}(S, A)$ 和 $\widehat{T}(S, A)$ 分别为归一化后的总额外数据搬运量和总运行时间， w_1 、 w_2 为权重系数。

本模型需满足以下约束：优化后的总额外数据搬运量不得超过问题二最优解的 110%；算子的调度执行顺序保持一致；片上缓存的使用量不得超过其物理容量上限；由缓存分配方案 A 引入的任何 SPILL 操作及地址复用依赖，必须保证修改后的计算图与其数据依赖关系得到满足。具体如下所示：

$$\begin{aligned} \Delta D(S, A) &\leq 1.1 \times \Delta D_{\text{baseline}} \\ \text{Peak}(S, A) &\leq C_{\text{max}} \end{aligned} \quad (5-4)$$

5.3.2 阶段二：多目标帕累托前沿性能优化模型

阶段二同时考虑总额外数据搬运量和算子总运行时间，思考在**优化调度顺序、缓存分配方案**的情况下，调度序列的性能优化策略的设计。本问题考虑双指标，适合通过多目标帕累托最优模型进行求解。

(1) NSGA-II 的模型建立

(a) 多目标模型与支配关系

设决策向量 $x = (S, M)$ ：其中 S 为**调度序列**（满足 DAG 约束）， M 为缓存分配与 SPILL 规则。本题双目标为：

$$\min_x (f_1(x), f_2(x)) = (\Delta D(x), T(x)), \text{ s.t. } x \in \mathcal{C} \quad (5-5)$$

Pareto 支配：若对任意 i 有 $f_i(x) \leq f_i(y)$ ，且至少对一项严格小于，则称 x 支配 y 。所有不被任何可行解支配的解构成**帕累托前沿**。

(b) 核心机制（快速非支配排序+拥挤距离）

快速非支配排序：把种群按支配层级划为 $F1, F2, \dots$ ，其中 $F1$ 为非支配解集。

拥挤距：在同一前沿上，按每个目标从小到大排序，用相邻点的目标差分近似个体稀疏度，边界个体置为 ∞ 以保多样性：

$$CD_i += \frac{f^{(k+1)}(i) - f^{(k-1)}(i)}{f_{\text{max}}^{(k)} - f_{\text{min}}^{(k)}} \quad (\text{对每个目标 } k \text{ 累加}) \quad (5-6)$$

(c) 选择—交叉—变异

编码：个体 $x = (S, \text{alloc}, \text{gp_tree})$ 。其中 S 为排列变量（节点顺序）， $\text{alloc} \in \text{first-fit}, \text{best-fit}, \text{worst-fit}$ 为离散策略， gp_tree 为 GP 规则树。

交叉：对 S 用部分映射交叉（PMX）；对 alloc 交换；对 gp_tree 用树交叉。

变异：对 S 随机交换两位置；对 alloc 随机切换策略；对 gp_tree 进行结构变异；

可行性与评估：利用 `simulate_with_trace` 产出增广序列、第一次偏移、SPILL 列表，并计算基础执行时间+SPILL 罚时得到 T ； ΔD 直接给出。

参数设置，本文采用的默认参数为^[29]：种群规模 $P = 50$ ，最大代数 $G = 50$ ；交叉率 $p_c = 0.80$ ，变异率 $p_m = 0.10$ ；

编码：个体 $x = \langle S, \text{alloc}, \text{gp_tree} \rangle$ 。其中：

S ：节点顺序，用部分映射交叉（PMX）；

$alloc \in \{\text{first-fit}, \text{best-fit}, \text{worst-fit}\}$: 交叉为整型交换, 变异为策略切换;

gp_tree : GP 规则树, 交叉为子树交叉, 变异为结构变异。

非支配排序与拥挤距离采用标准 NSGA-II: 拥挤距离按式(5-1)对每个目标累计; 精英合并代入。

可行性与评估: 对生成个体, 先经就绪集过滤+轻量修复器确保满足 DAG/容量/L0 单驻留等约束, 再用 $\text{simulate_with_trace}$ 评估两目标 $f_1 = \Delta D(x)$ 与 $f_2 = T(x)$ 。

(d) 算法流程

评估($\Delta D, T$)→快速非支配排序→计算拥挤距离→锦标赛/截断选父代→交叉变异得子代→父子并集重排并按(等级,拥挤度)截断为下一代。

终止与选择两算法均以达到最大迭代/代数或帕累托前沿超体积连续 10 代提 $< 10^{-3}$ 为终止条件。

(2) 多目标粒子群的基础原理与建模

(a) 粒子表示与目标评估

粒子位置同样编码为 $x = (S, alloc, gp_tree)$, 目标函数与评估器与 NSGA-II 完全一致。

(b) 速度—位置更新

MOPSO 的标准速度更新为

$$v \leftarrow \omega v + c_1 r_1 (pbest - x) + c_2 r_2 (leader - x), x \leftarrow x \oplus v \quad (5-7)$$

其中 ω, c_1, c_2 为惯性与学习因子

群体规模 $N = 60$, 迭代次数 $I = 200$;

惯性权重 ω 性递减: $\omega_t = 0.9 - 0.5 \cdot t/I$;

学习因子 $c_1 = c_2 = 1.5$; 速度按编码维度分别夹取到预设上限 (防止大步回跳);

外部精英库大小 $A=50$, 采用拥挤距离选 leader;

变异: 对 $\{S, alloc, gp_tree\}$, 以概率 $\bar{s}p_{mut} = 0.10$ 作与 NSGA-II 同步的轻量变异;

多样化: 每 20 代若精英库超 70% 重复, 触发网格扰动或对个体施加小幅强制变异;

约束同 NSGA-II: 评估前统一执行就绪集过滤与修复, 保证可行。

(c) 非支配档案与领导者选择

维护外部档案保存当前非支配解, 每个粒子从档案中按一定策略选取领导者 leader; 并在每代结束时用快速非支配排序 + 拥挤距离更新档案与稀疏性, 这保证了 MOPSO 在目标空间的均匀展开与多样性。

(d) 收敛性与多样性增强

实现中按节点规模自适应调整种群大小, 并在前沿解偏少时提高随机性以防止早收敛; 最终返回“最后一代前沿”、“历代前沿数量曲线”、“收敛曲线”等可视化。

5.4 协调优化模型求解

5.4.1 总额外数据搬运量约束下的加权指标优化模型求解

为在总额外数据搬运量指标不显著增加的前提下尽可能降低算子的总运行时间, 我们在问题二 GP 算法的基础上进行了两阶段优化。第一轮优化目标为最小化数据搬运量, 即第二问的结果, 作为对比, 第二轮优化将 GP 生成缓存分配方案的目标函数改为 $0.8 \times$ 总额外数据搬运量 + $0.2 \times$ 算子总运行时间, 使其重新生成缓存方案规则, 并与问题二的结果对比。

我们以 Conv_Case0 文件在三种 A 类分配和 GP 类规则策略组合下的运算为例:

表 17 Conv-Case0 算例在不同分配策略下的对比表

strategy	move	time	fitness
best_fit	234768	120600.8	0.429
first_fit	241248	121205.6	0.435
worst_fit	255168	122338.4	0.448

为了更加清晰的展示，我们对以上数据做了如下可视化处理：

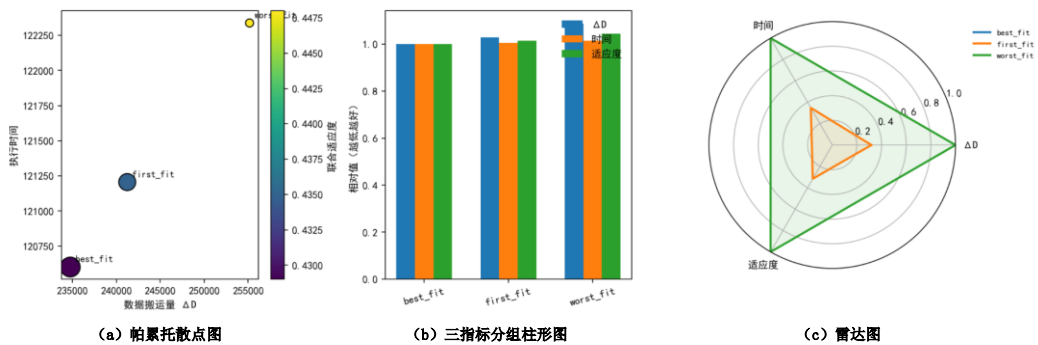


图 32 不同分配策略下效果对比图

由图 32 可知 best_fit 明显优于 first_fit 与 worst_fit。在帕累托散点图 (a) 中, best_fit 落在左下角, 数据搬运量与执行时间同时更低, 而 worst_fit 位于右上方, 两项指标均劣; 分组柱形图 (b) 在相对最小值归一化后显示, best_fit 在 ΔD 、时间、联合适应度三个维度上均取得最低, 且与 second-best (first_fit) 的差距稳定存在; 雷达图 (c) 进一步说明其“综合轮廓”最小、形状最紧凑, 体现了整体均衡、无明显短板。因此, 在当前数据与评价口径下, 采用 best_fit 分配策略并配合同一 GP 受害者规则, 可获得更低的数据搬运量与更短的执行时间, 是三种候选方案中的首选。

为便于在图形结论之后给出量化证据, 我们进一步整理了各方案相对基的改变量, 并汇总于下表。表中 *spill_change* 与 *time_change* 分别表示 SPILL 数/搬运量与总执行时间的绝对差值, 负值代表相较基线改进、正值代表变差; 对应的 *spill_change_percent* 与 *time_change_percent* 为百分比变化, 计算式为:

$$\text{change percent} = \frac{\text{GP 改进后} - \text{基线}}{\text{基线}} \times 100\%.$$

(5 - 8)

表 18 GP 规则改进效果汇总

rank	spill_change	spill_change_percent	time_change	time_change_percent
1	-7,064	-2.921	-778	-0.641
2	-10,760	-4.270	-1,229	-1.004
3	3,020	1.977	-144	-0.118

为了更加直观的进行表示，做出了如下可视化图片，从图 33 可以看出：

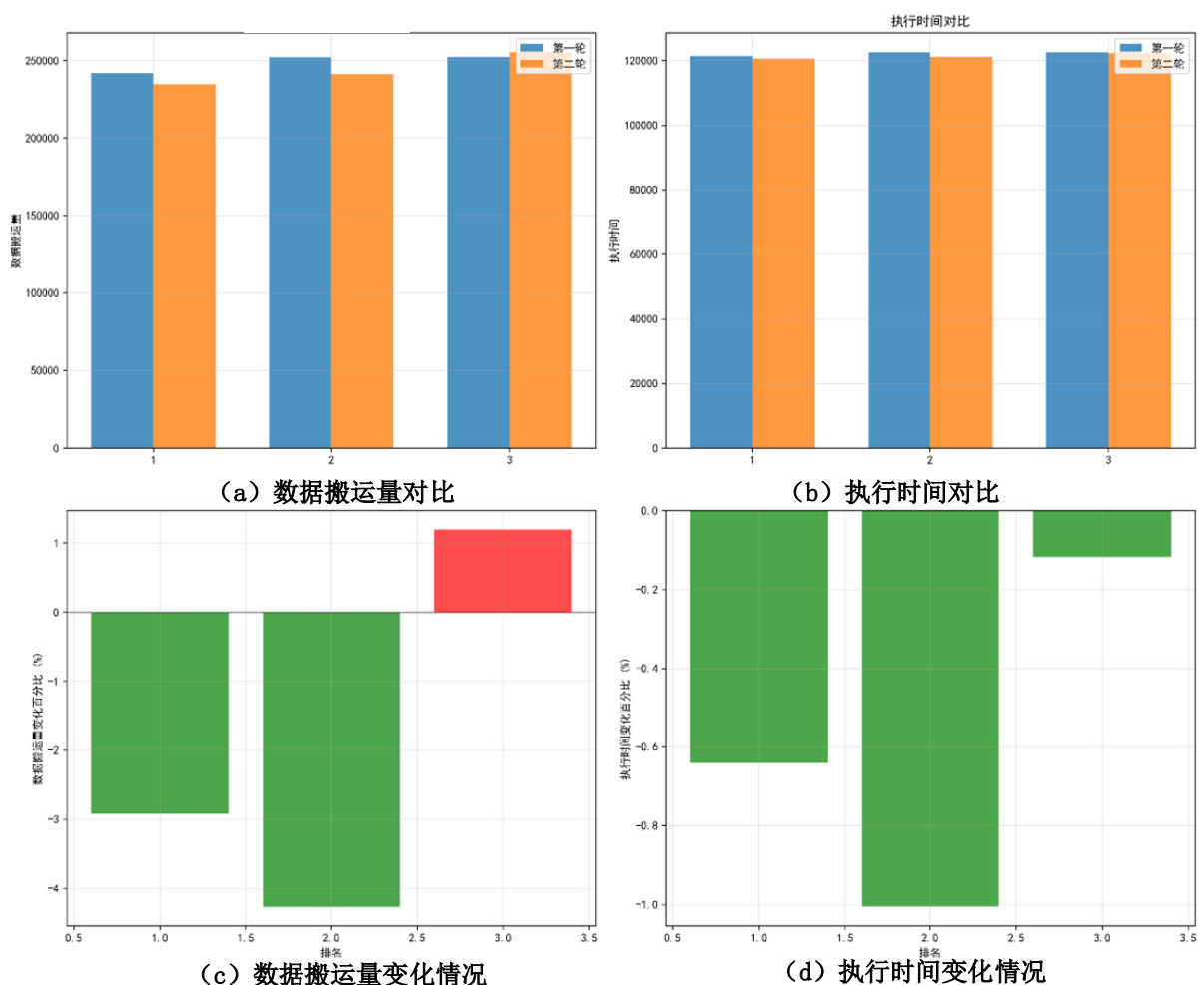


图 33 GP 规则改进效果图

引入 GP 规则后，样例 1、2 的数据搬运量分别下降约 2.9% 和 4.3%，样例 3 小幅上升约 2.0%；三组的执行时间均下降（约 0.64%/1.00%/0.12%）。因此总体呈现“时间稳定下降， ΔD 多数下降”：样例 1、2 实现双指标同步改进，样例 3 存在轻微权衡。

基于 GP 的性能优化策略在 Conv_Case0 任务上的实验结果表明，通过调整缓存分配规则，GP 方法能够在一定程度上降低算子的总运行时间，例如，在 best_fit 策略下，执行时间降至 120600.8 周期，但与此同时，总额外数据搬运量却呈现上升趋势。这揭示出了 GP 方法的局限性：它虽能通过规则演化优化局部性能，但无法同时最小化数据搬运量和运行时间，导致整体效果非最优。此外，测试显示 GP 策略在节点数较大的任务中泛化能力不足，其计算复杂度和规则生成过程难以扩展，无法高效处理大规模计算图。

因此，为突破 GP 方法的局限，实现更具泛化性和通用性的性能优化，我们转向多目标优化算法设计。该方法旨在对总额外数据搬运量和运行时间两项指标进行联合优化，并从调度顺序调整和缓存分配方案协同入手，以平衡竞争目标，提升整体效率。这一过渡为下文基于多目标规划的策略提供了理论基础和实践动机。

5.4.2 多目标帕累托前沿性能优化模型求解

求解上以 NSGA-II 为主，并分别作用于三维基因：拓扑序 OX 交叉+位置交换、分配策略交换/变异、GP 子树交叉/变异，评价时用就绪集过滤与轻量修复保证 DAG 与 L0

约束。以`Conv_Case0`为例，算法在 50 代内收敛得到两个帕累托解。其中， $f_1 \approx 206.3k \sim 206.4k$ ， $f_2 \approx 113.76k$ ，结果显示两目标同向可降且相关性较强，前沿点数较少但质量稳定。为增强泛化性，进一步引入多目标粒子群（MOPSO）对六个任务分别求解，汇总多个帕累托解集，最终通过性能分析选择最终解。

为直观展示多目标规划的求解过程与选解原则，下面以 `Conv_Case0` 为例给出结果图：左上为最终帕累托前沿，各候选解在总额外搬运量 f_1 与“总运行时间 f_2 ”上的非支配分布，右上为全体候选解的 $f_1 - f_2$ 散点图以体现两指标相关性；下方两图分别给出 50 代内 f_1 与 f_2 的历代最优收敛曲线。

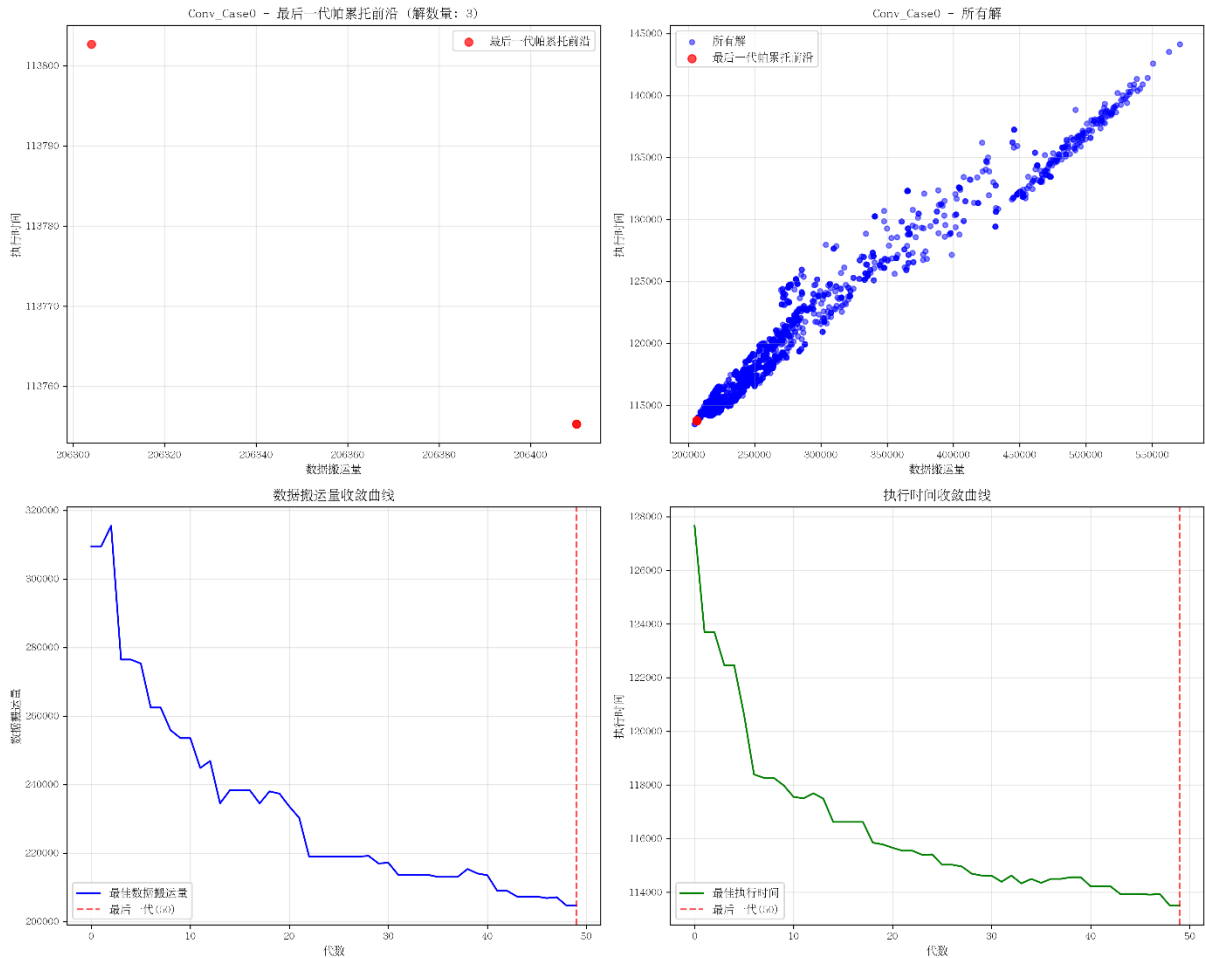


图 34 `Conv_Case0` 文件在多目标规划求解过程与帕累托前沿

表 19 `Conv_Case0` 帕累托解指标汇总

solution_id	spill_cost	execution_time	generation
1	206410	113755.3	50
2	206304	113802.7	50

由数据可知，收敛曲线显示算法运行良好，数据搬运量和执行时间都呈下降趋势。同时出现了帕累托前沿解数量较少的情况，这可能是由于两个目标可能存在强相关性。同时考虑到泛化性和通用性，我们加入多目标粒子群算法，分别对六个任务进行求解。最终每个任务可以得到多个非支配解，在此我们各多目标算法下总额外数据搬运量较小的那个解，作为最终的优化策略。

我们分别用 NSGA-II 与 MOPSO 在每个任务上得到一条帕累托前沿，并在前沿上各选出代表解（均满足约束）。为兼顾泛化与工程可用性，按“先搬运量、后时间”的单点选解规则确定最终方案：优先选择**总额外数据搬运量（spill_cost）**更小的算法结果；若搬运量相同，再比较**总运行时间**。由表 20 可见：例如 Conv_Case0 由 NSGA-II 给出更小的搬运量（206,410vs207,131），而 Conv_Case1 则由 MOPSO 更优（801,311vs821,735）；这说明两种多目标算法在不同图结构上各有所长。

表 20 多目标方法帕累托代表解汇总

任务名	NSGA-II		MOPSO	
	总额外数据 搬运量	总运行时间	总额外数据 搬运量	总运行时间
Conv_0	206410	113755	207131	113635
Conv_1	821735	405967	801311	419323
Matmul_0	38610	13284	40021	13131
Matmul_1	500311	201731	501832	200311
FlashAttention_0	3917	201731	3847	2047
FlashAttention_1	30131	14743	29031	14317

最终优化效果如表 21 所示。我们对比了优化前后指标：六个任务的**搬运量**下降约 **3%–15%**，**总运行时间**下降约 **4%–15%**。改进幅度在 **Matmul_Case0/Case1** 与 **FlashAttention_Case0** 上尤为显著（时间降幅分别为 13.62%、12.34%、15.29%），而 **Conv_Case1** 的下降幅度相对较小，符合其结构刚性的特点。该结果表明多目标方案在不牺牲（甚至降低）搬运量的同时，能有效缩短总体运行时间。

表 21 各任务优化前后数据对比

示例	优化前 数据搬 运量	优化后 数据搬 运量	搬运量 优化率	优化前总 运行时间	优化后总 运行时间	总时间 优化率
Conv_0	242072	206410	14.73%	124223	113755	8.43%
Conv_1	840904	801311	4.71%	438127	419323	4.29%
Matmul_0	41600	38610	7.19%	15378	13284	13.62%
Matmul_1	518400	500311	3.49%	230131	201731	12.34%
FlashAttention_0	4096	3917	4.37%	2381	2017	15.29%
FlashAttention_1	33792	29031	14.09%	15317	14317	6.53%

由此我们得到问题三的求解结果，见表 22。

表 22 问题三结果表

任务名	总时间优化率	搬运量优化率
Conv_0	8.43%	14.73%
Conv_1	4.29%	4.71%
Matmul_0	13.62%	7.19%
Matmul_1	12.34%	3.49%
FlashAttention_0	15.29%	4.37%
FlashAttention_1	6.53%	14.09%

5.5 本章小结

本章对问题三进行模型建立与求解，在前两问的基础上，综合考虑总额外数据搬运量和算子总运行时间，设计了两阶段优化模型，并提出了基于遗传编程算法（GP）和多目标优化算法（NAGA-II 算法和 MOPSO 算法）的求解策略。在阶段一，我们通过改进遗传编程算法优化缓存分配方案，以最小化总额外数据搬运量为目标，并确保总额外数据搬运量不超过问题二的最优解的 110%。通过加权指标优化模型，将总额外数据搬运量和算子总运行时间加权合并，最终选择合适的权重组合，得到最优化的缓存分配方案。在阶段二，我们转向多目标优化求解，采用 NAGA-II 算法和 MOPSO 算法对总额外数据搬运量和总运行时间进行联合优化。通过求解帕累托最优解并比较各任务的性能优化指标，选择最终的最优方案。结果表明，通过优化缓存分配方案和调度顺序，我们实现数据搬运量指标和总运行时间指标的优化。

六、主要结论与模型评价

6.1 模型的优点

(1) 针对问题一，本文以“贪心算法初解+多启发式优化”构建调度框架，能够覆盖多种算子组合与内存模式，又有利于在不同数据集和任务场景间直接迁移，充分体现了方法的**通用性与泛化性**。

(2) 贪心算法基于就绪集和优先队列的实现，能够将时间复杂度控制在较低的范围，使得算法运行更高效，适合大规模数据处理。并且其的解易于复现和验证，为后续优化算法的比较提供了清晰的基准。

(3) 问题二子问题一通过遗传编程（GP）算法进行单目标优化模型，能够为缓存分配方案提供高质量解，且易于理解和实现。

(4) 问题三建立的“递进优化+多目标帕累托搜索”的综合框架，显著提升了模型的**实际性能导向与决策支持能力**。

6.2 模型的不足

(1) 启发式方法不保证全局最优，在依赖极密或峰值由全局耦合触发的实例上，改进幅度可能有限。且各启发式算法中退火温度、邻域次数、交叉/变异概率、NSGA-II 种群规模等参数的设定对结果存在影响。

(2) 贪心算法的主要缺点是容易陷入局部最优解，未必能找到全局最优解，尤其在复杂的计算图中，可能无法处理所有的约束条件。

(3) 遗传编程算法的计算复杂度较高，特别是在大规模计算图和多种约束条件下，可能需要较长的时间来收敛到最优解。

(4) 问题二子问题二的加权单目标模型存在权重设定的主观性，可能影响解的帕累托最优性。

(5) 问题三建立的“递进优化+多目标帕累托搜索”的综合框架计算复杂度极高，特别是帕累托搜索对计算资源需求巨大，在效率上存在优化空间。

6.3 模型的推广

(1) 本文针对华为 Davinci 架构平台设计的优化调度算法仅需在就绪集过滤与修复器中添加新规则，便可适配**不同硬件/缓存层级与算子类型**，无需重写优化主干，具有较强的**可扩展性**。

(2) 遗传编程适用于那些策略空间复杂且约束条件多样的任务。随着硬件优化和计算资源的提升，遗传编程可以**扩展**到更复杂的计算图和多维度的缓存优化问题。

(3) 问题三建立的“递进优化+多目标帕累托搜索”的综合框架为复杂系统的多目标协同优化提供了一个标准范式，其揭示的绩效权衡关系对软硬件协同设计等领域具有重要的指导意义。

参考文献

- [1] Lee, C.N., Marquez, J., Edwards, T., & AIAA. Crew Autonomy through Self-Scheduling: Scheduling Performance Pilot Study. AIAA SCITECH 2021 FORUM, 2021.
- [2] Chen, J., & Qian, X. Smart Scheduling: A Pilot Project of Workforce Scheduling in Radiation Oncology. Journal of Scientific Innovation in Medicine, 7(1), 2-2, 2024.
- [3] Saito, S., & Fujimoto, K. Port Contention Aware Task Scheduling for SIMD Applications. IEEE Access, 12, 6204–6216, 2024.
- [4] Seo, M., et al. IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System. ASPLOS 2024, 2024.
- [5] Qian, X., Lv, C., He, Z., & Qian, W. High-Quality Iterative Logic Compiler for In-Memory SIMD Computation with Tight Coupling of Synthesis and Scheduling. arXiv:2412.02212, 2024.
- [6] Kim, W., et al. Energy-Efficient Dataflow Scheduling of CNN Applications for Vector-SIMD DSP. IEEE Access, 12, 98234–98247, 2024.
- [7] 陈华根, 吴健生, 王家林, 等. 模拟退火算法机理研究[J]. 同济大学学报(自然科学版), 2004, (06): 802-805.
- [8] 支元. 试论基于禁忌搜索算法的计算机网络通信覆盖范围优化方法 [J]. 软件, 2025, 46 (08): 164-166.
- [9] 李大卫, 王莉, 王梦光. 遗传算法与禁忌搜索算法的混合策略 [J]. 系统工程学报, 1998, (03): 30-36.
- [10] Mitchell, Melanie. *An introduction to genetic algorithms*. MIT press, 1998.
- [11] Forrest, Stephanie. "Genetic algorithms." *ACM computing surveys (CSUR)* 28.1 (1996): 77-80.
- [12] Ahvanooey, Milad Taleby, et al. "A Survey of Genetic Programming and Its Applications." *KSII Trans. Internet Inf. Syst.* 13.4 (2019): 1765-1794.
- [13] Espejo, Pedro G., Sebastián Ventura, and Francisco Herrera. "A survey on the application of genetic programming to classification." *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 40.2 (2009): 121-144.
- [14] Xiao, Yiyong, et al. "Variable neighbourhood simulated annealing algorithm for capacitated vehicle routing problems." *Engineering Optimization* 46.4 (2014): 562-579.
- [15] Luc, Dinh The. "Pareto optimality." *Pareto optimality, game theory and equilibria*. New York, NY: Springer New York, 2008. 481-515.
- [16] Lv, Junshuai, et al. "Multi-objective optimization research of printed circuit heat exchanger based on RSM and NSGA-II." *Applied Thermal Engineering* 254 (2024): 123925.
- [17] Coello, CA Coello, and Maximino Salazar Lechuga. "MOPSO: A proposal for multiple objective particle swarm optimization." *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No. 02TH8600)*. Vol. 2. IEEE, 2002.
- [18] 龚超, 王翼, 梁宵, 等. 强化学习[M]. 化学工业出版社: 2024 05: 251.

- [19] W.JiaandZ.He,"OutlierdataMiningoflargeDataSetsrelyingonfastdecompositionsimulatedannealingalgorithm,"2020InternationalConferenceonRobots&IntelligentSystem(ICRIS),Sanya,China,2020,pp.679-682.
- [20] 李欢.基于遗传算法的低碳物流车辆配送路径优化[J].内燃机与配件,2025,(17):30-32.DOI:10.19475/j.cnki.issn1674-957x.2025.17.031.
- [21] 杨二龙,齐梦,甘庭源,等.基于多目标粒子群算法的油田水驱后加密聚驱注采参数优化方法[J/OL].钻采工艺,2025,(05):100-106[2025-09-23].
- [22] 刘超男,黄勇,赵成勇.基于多目标粒子群算法的新能源汇集网架规划[J/OL].南方电网技术,1-11[2025-09-23].<https://link.cnki.net/urlid/44.1643.TK.20250826.1738.020>.
- [23] 肖晓伟,肖迪,林锦国,等.多目标优化问题的研究概述[J].计算机应用研究,2011,28(03):805-808+827.
- [24] 高鹰,谢胜利.基于模拟退火的粒子群优化算法[J].计算机工程与应用,2004,(01):47-50.
- [25] Shieh,T.D.,G.E.L.;Lu,C.;Li,J.;Zhao,C.H.;Zhu,Y.F.;Cao,S.Z.Optimalschedulingalgorithmforhybridflowshopwithbatchingmachinesconsideringenergyconsumption.*Energy*,2020.200,117565.
- [26] 艾玉.变邻域模拟退火算法在农村生活垃圾收运中的应用[J].计算机系统应用,2024,33(09):192-200.DOI:10.15888/j.cnki.csa.009622.
- [27] 罗正亮,夏冰,鲍萌,等.基于多目标优化的海上风电场运维任务动态调度规划方法[J/OL].南方能源建设,1-11[2025-09-21].<https://doi.org/10.16516/j.ceec.2024-392>.
- [28] 何玉钧,张棋峰,牛宇宏,等.基于贪心算法的电力分布式调度通信拓扑优化[J/OL].电子测量技术,1-9[2025-09-21].<https://link.cnki.net/urlid/11.2175.TN.20250701.1128.008>.
- [29] 熊傲,杨杰芳,张云华.基于 NSGA-II算法的重频组多指标优化设计[J/OL].现代雷达,1-8[2025-09-25].<https://doi.org/10.16592/j.cnki.1004-7859.2025142>.

附录 A 人工智能产品辅助答题规范说明

人工智能产品使用情况	
工具名称: ChatGPT 版本/型号: GPT-5 开发机构/公司: OpenAI 版本发布日期: 2025 年 8 月 7 日	(1) 使用了 ChatGPT 帮助了解如遗传算法、模拟退火、禁忌搜索等元启发式算法的核心思想、适用场景。 (2) 使用了 ChatGPT 协助修改部分代码的错误。
工具名称: DeepSeek 版本/型号: DeepSeek-V3.1 开发机构/公司: 深度求索 版本发布日期: 2025 年 8 月 21 日	(1) 使用了 DeepSeek 了解 SIMD 架构和华为 Davinci 架构平台。 (2) 使用了 DeepSeek 协助梳理复杂的问题求解逻辑。
算法与技术组合的关键开发框架	
本文在数学建模过程中主要使用了以下开源框架与工具: matplotlib:用于可视化和绘图 numPy: 用于数据计算操作 pandas: 用于数据处理与分析 tabu_search: 用于实现禁忌搜索算法 pymoo: 用于实现多目标优化算法 DEAP: 用于实现遗传算法与遗传编程	
数据分析与处理工具	
本文使用了 Gephi 来可视化附件计算图中各节点之间的关系,绘制了有向图。此外还通过 Gephi 进行模块化分析与绘制节点大小分布图,用于分析附件数据的复杂程度。	

附录 B 关键代码展示

问题一基于贪心算法的求解的关键代码

```
import json
import heapq
from collections import deque, defaultdict
from tqdm import tqdm
import time
import psutil
import matplotlib.pyplot as plt
import pandas as pd
import os
import numpy as np
import random
import sys
# 计算最大缓存驻留量及其位置
def calculate_max_cache_residency(schedule, nodes):
    current_cache_residency = 0
    max_cache_residency = 0
    max_residency_step = 0
    residency_history = []
    id_to_node = {node['Id']: node for node in nodes}
    for step, node_id in enumerate(schedule):
        node = id_to_node[node_id]
        if node['Op'] == 'ALLOC':
            current_cache_residency += node.get('Size', 0)
        elif node['Op'] == 'FREE':
            current_cache_residency -= node.get('Size', 0)
        if current_cache_residency < 0:
            current_cache_residency = 0
        residency_history.append(current_cache_residency)
        if current_cache_residency > max_cache_residency:
            max_cache_residency = current_cache_residency
            max_residency_step = step
    return max_cache_residency, max_residency_step, residency_history

# 1. 最小化峰值策略
def greedy_min_peak(nodes, edges):
    print("\n 开始最小化峰值策略...")
    start_time = time.time()
    process = psutil.Process(os.getpid())
    initial_mem = process.memory_info().rss / (1024 * 1024) # MB
    graph = {node['Id']: [] for node in nodes}
    in_degree = {node['Id']: 0 for node in nodes}
    for edge in edges:
        graph[edge[0]].append(edge[1])
        in_degree[edge[1]] += 1
    heap = []
    for node in nodes:
        if in_degree[node['Id']] == 0:
            if node['Op'] == 'FREE':
                priority = (0, -node.get('Size', 0))
```

```

        elif node['Op'] == 'ALLOC':
            priority = (1, -node.get('Size', 0))
        else:
            priority = (2, 0)
        heapq.heappush(heap, (priority, node['Id']))
    schedule = []
    with tqdm(total=len(nodes), desc="MinPeak 策略", unit="节点") as pbar:
        while heap:
            _, node_id = heapq.heappop(heap)
            schedule.append(node_id)
            pbar.update(1)
            for neighbor in graph.get(node_id, []):
                in_degree[neighbor] -= 1
                if in_degree[neighbor] == 0:
                    neighbor_node = next(n for n in nodes if n['Id'] == neighbor)
                    if neighbor_node['Op'] == 'FREE':
                        priority = (0, -neighbor_node.get('Size', 0))
                    elif neighbor_node['Op'] == 'ALLOC':
                        priority = (1, -neighbor_node.get('Size', 0))
                    else:
                        priority = (2, 0)
                    heapq.heappush(heap, (priority, neighbor))
    if len(schedule) != len(nodes):
        raise ValueError("图中存在环，无法生成完整调度")
    end_time = time.time()
    elapsed_time = end_time - start_time
    final_mem = process.memory_info().rss / (1024 * 1024) # MB
    mem_usage = final_mem - initial_mem
    node_rate = len(nodes) / elapsed_time if elapsed_time > 0 else float('inf')
    max_residency, max_residency_step, residency_history = calculate_max_cache_resi-
dency(schedule, nodes)
    print(f"最小化峰值策略完成! 最大缓存驻留量: {max_residency} bytes")
    return schedule, max_residency, max_residency_step, residency_history, elapsed_time,
    mem_usage, node_rate

```

2. 最早释放策略

```

def greedy_early_free(nodes, edges):
    print("\n 开始最早释放策略...")
    start_time = time.time()
    process = psutil.Process(os.getpid())
    initial_mem = process.memory_info().rss / (1024 * 1024) # MB
    graph = {node['Id']: [] for node in nodes}
    in_degree = {node['Id']: 0 for node in nodes}
    for edge in edges:
        graph[edge[0]].append(edge[1])
        in_degree[edge[1]] += 1
    heap = []
    for node in nodes:
        if in_degree[node['Id']] == 0:
            if node['Op'] == 'FREE':
                priority = (0, node['Id'])
            else:
                priority = (1, node['Id'])
            heapq.heappush(heap, (priority, node['Id']))

```

```

schedule = []
with tqdm(total=len(nodes), desc="EarlyFree 策略", unit="节点") as pbar:
    while heap:
        _, node_id = heapq.heappop(heap)
        schedule.append(node_id)
        pbar.update(1)
        for neighbor in graph.get(node_id, []):
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                neighbor_node = next(n for n in nodes if n['Id'] == neighbor)
                if neighbor_node['Op'] == 'FREE':
                    priority = (0, neighbor_node['Id'])
                else:
                    priority = (1, neighbor_node['Id'])
                heapq.heappush(heap, (priority, neighbor))
    if len(schedule) != len(nodes):
        raise ValueError("图中存在环，无法生成完整调度")
    end_time = time.time()
    elapsed_time = end_time - start_time
    final_mem = process.memory_info().rss / (1024 * 1024)
    mem_usage = final_mem - initial_mem
    node_rate = len(nodes) / elapsed_time if elapsed_time > 0 else float('inf')
    max_residency, max_residency_step, residency_history = calculate_max_cache_resi-
dency(schedule, nodes)
    print(f"最早释放策略完成! 最大缓存驻留量: {max_residency} bytes")
    return schedule, max_residency, max_residency_step, residency_history, elapsed_time,
mem_usage, node_rate

```

3. 延迟分配策略

```

def greedy_late_alloc(nodes, edges):
    start_time = time.time()
    process = psutil.Process(os.getpid())
    initial_mem = process.memory_info().rss / (1024 * 1024) # MB
    graph = {node['Id']: [] for node in nodes}
    in_degree = {node['Id']: 0 for node in nodes}
    for edge in edges:
        graph[edge[0]].append(edge[1])
        in_degree[edge[1]] += 1
    heap = []
    for node in nodes:
        if in_degree[node['Id']] == 0:
            if node['Op'] != 'ALLOC':
                priority = (0, node['Id'])
            else:
                priority = (1, -node.get('Size', 0))
            heapq.heappush(heap, (priority, node['Id']))
    schedule = []
    with tqdm(total=len(nodes), desc="LateAlloc 策略", unit="节点") as pbar:
        while heap:
            _, node_id = heapq.heappop(heap)
            schedule.append(node_id)
            pbar.update(1)
            for neighbor in graph.get(node_id, []):
                in_degree[neighbor] -= 1

```

```

        if in_degree[neighbor] == 0:
            neighbor_node = next(n for n in nodes if n['Id'] == neighbor)
            if neighbor_node['Op'] != 'ALLOC':
                priority = (0, neighbor_node['Id'])
            else:
                priority = (1, -neighbor_node.get('Size', 0))
            heapq.heappush(heap, (priority, neighbor))
    if len(schedule) != len(nodes):
        raise ValueError("图中存在环, 无法生成完整调度")
    end_time = time.time()
    elapsed_time = end_time - start_time
    final_mem = process.memory_info().rss / (1024 * 1024) # MB
    mem_usage = final_mem - initial_mem
    node_rate = len(nodes) / elapsed_time if elapsed_time > 0 else float('inf')
    max_residency, max_residency_step, residency_history = calculate_max_cache_resi-
dency(schedule, nodes)
    print(f'延迟分配策略完成! 最大缓存驻留量: {max_residency} bytes')
    return schedule, max_residency, max_residency_step, residency_history, elapsed_time,
mem_usage, node_rate

```

问题一基于禁忌搜索算法的关键求解代码

```

import json
import pandas as pd
import random
import math
import matplotlib.pyplot as plt
from collections import deque

if __name__ == "__main__":
    json_file = "Conv_Case1.json"
    nodes, edges = load_graph_data(json_file)
    nodes_dict = {}
    for node in nodes:
        node_id = int(node['Id'])
        nodes_dict[node_id] = node
    tabu_size = 50 # 禁忌列表大小
    max_iterations = 200 # 最大迭代次数 (按要求设置为 300 代)
    max_neighbors = 100 # 每次迭代生成的邻域解数量
    no_improvement_limit = 200 # 无改善迭代次数限制
    diversification_threshold = 500 # 多样化阈值
    print("=" * 60)
    print("禁忌搜索算法参数设置")
    print("=" * 60)
    print(f'禁忌列表大小 (tabu_size): {tabu_size}')
    print(f'最大迭代次数 (max_iterations): {max_iterations}')
    print(f'邻域大小 (max_neighbors): {max_neighbors}')
    print(f'无改善限制: {no_improvement_limit}')
    print(f'多样化阈值: {diversification_threshold}')
    print("=" * 60)

```

```

results = {}
initial_files = [
    ("初始解 1C1.xlsx", "延迟分配策略"),
    ("初始解 2C1.xlsx", "随机生成"),
    ("初始解 3C1.xlsx", "最早释放策略")
]
for excel_file, solution_name in initial_files:
    print(f"\n 处理 {solution_name} 初始解...")
    try:
        initial_sequence = load_initial_solution(excel_file)
    except FileNotFoundError:
        print(f"文件 {excel_file} 不存在, 跳过...")
        continue
    except Exception as e:
        print(f"读取 {excel_file} 时出错: {e}")
        continue
    if not is_valid_sequence(initial_sequence, edges):
        print(f"{solution_name} 初始解无效! 请检查依赖关系。")
        continue
    print(f"初始序列长度: {len(initial_sequence)}")
    print(f"初始序列示例: {initial_sequence[:5]}")
    tabu_search = TabuSearch(
        nodes_dict=nodes_dict,
        edges=edges,
        tabu_size=tabu_size,
        aspiration_criteria=True
    )
    print("使用禁忌搜索算法...")
    best_sequence, best_cost = tabu_search.run(
        initial_sequence=initial_sequence,
        max_iterations=max_iterations,
        max_neighbors=max_neighbors,
        no_improvement_limit=no_improvement_limit,
        diversification_threshold=diversification_threshold
    )
    use_local_search = False
    if use_local_search:
        print("\n 进行局部搜索后处理...")
        best_sequence, best_cost = local_search_post_processing(best_sequence, nodes_dict,
edges)
    history_filename = f'tabu_cost_history_{solution_name}.txt'
    with open(history_filename, "w", encoding='utf-8') as f:
        f.write(f"禁忌搜索算法成本历史 - {solution_name}\n")
        f.write(f"参数设置: tabu_size={tabu_size}, max_iterations={max_iterations},
max_neighbors={max_neighbors}, "
            f"no_improvement_limit={no_improvement_limit}, diversification_thresh-
old={diversification_threshold}\n")
        f.write("迭代次数\t当前成本\t最佳成本\n")
        for i in range(len(tabu_search.cost_history)):
            f.write(f"{i+1}\t{tabu_search.cost_history[i]}\t{tabu_search.best_costs_his-
tory[i]}\n")
    print(f"成本历史已保存到: {history_filename}")

```

```

with open(f'tabu_optimized_sequence_{solution_name}.txt', "w", encoding='utf-8') as f:
    f.write(f'{solution_name} 禁忌搜索优化结果\n')
    f.write(f'最佳成本: {best_cost}\n')
    f.write("最佳序列:\n")
    for node_id in best_sequence:
        f.write(f'{node_id}\n')
results[solution_name] = {
    'best_sequence': best_sequence,
    'best_cost': best_cost,
    'best_costs_history': list(tabu_search.best_costs_history)
}
print(f'\n{solution_name} 优化完成!')
print(f'最佳最大 V_stay: {best_cost}')
if results:
    plt.figure(figsize=(12, 8))
    greedy_optimal = 9728
    colors = ['blue', 'red', 'orange', 'purple']
    for i, (solution_name, data) in enumerate(results.items()):
        iterations = range(1, len(data['best_costs_history']) + 1)
        plt.plot(iterations, data['best_costs_history'],
                 color=colors[i % len(colors)], linewidth=2,
                 label=f'{solution_name} (最终: {data["best_cost"]})')
    plt.xlabel('迭代次数')
    plt.ylabel('最大缓存驻留量 (Max V_stay)')
    plt.title(f'禁忌搜索算法收敛曲线对比 (贪婪最优={greedy_optimal})')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.savefig('tabu_convergence_comparison.png', dpi=300, bbox_inches='tight')
    plt.close()
    print("\n 收敛曲线图已保存到: tabu_convergence_comparison.png")

# 读取初始解
def load_initial_solution(excel_file):
    df = pd.read_excel(excel_file, header=None)
    first_col = pd.to_numeric(df.iloc[:, 0], errors='coerce').dropna()
    return first_col.astype(int).tolist()

# 计算调度序列的成本
def calculate_cost(sequence, nodes_dict):
    V_stay = 0
    max_V_stay = 0
    for node_id in sequence:
        node = nodes_dict[node_id]
        if node['Op'] == 'ALLOC':
            V_stay += node['Size']
        elif node['Op'] == 'FREE':
            V_stay -= node['Size']
        if V_stay > max_V_stay:
            max_V_stay = V_stay
    return max_V_stay

def is_valid_sequence(sequence, edges):

```

```

pos_map = {}
for idx, node_id in enumerate(sequence):
    pos_map[node_id] = idx
for edge in edges:
    u, v = edge
    if u not in pos_map or v not in pos_map:
        continue
    if pos_map[u] > pos_map[v]:
        return False
return True

class TabuList:
    def __init__(self, max_size=50):
        self.tabu_list = deque(maxlen=max_size)
        self.max_size = max_size
    def add(self, move):
        self.tabu_list.append(move)
    def is_tabu(self, move):
        return move in self.tabu_list
    def size(self):
        return len(self.tabu_list)
    def clear(self):
        self.tabu_list.clear()

class TabuSearch:
    def __init__(self, nodes_dict, edges, tabu_size=50, aspiration_criteria=True):
        self.nodes_dict = nodes_dict
        self.edges = edges
        self.tabu_list = TabuList(tabu_size)
        self.aspiration_criteria = aspiration_criteria
        self.best_global_cost = float('inf')
        self.best_global_sequence = None
        self.cost_history = []
        self.best_costs_history = []

```

问题二基于遗传编程算法的关键求解代码

```

import json
import csv
import time
import random
import numpy as np

if __name__ == "__main__":
    if args.batch:
        cases = [
            {"json": "Conv_Case0.json", "schedule": "Conv_Case0_schedule.txt", "out_prefix":
"Conv_Case0"},
            {"json": "FlashAttention_Case0.json", "schedule": "FlashAttention_Case0_sched-
ule.txt", "out_prefix": "FlashAttention_Case0"},
            {"json": "Matmul_Case0.json", "schedule": "Matmul_Case0_schedule.txt", "out_pre-
fix": "Matmul_Case0"},
        ]

```

```

rows, written_csv = run_batch_cases(cases, use_gp_victim=(not args.no_gp),
ga_pop=args.ga_pop, ga_gen=args.ga_gen, out_csv="batch_summary.csv")
print("\n=== 批量评估完成 ===")
print(f'已生成 {written_csv} (包含各图最优规则与总搬运量)')
for r in rows:
    print(r)
    for case in cases:
        prefix = case.get("out_prefix") or case["json"].replace('.json', '')
        id2node, edges, schedule, copyin_bufs = load_graph(case["json"], case["schedule"])
        _ = evolve_gp_for_spill(id2node, schedule, copyin_bufs,
                                pop_size=30, max_depth=3, generations=200,
                                cx_prob=0.7, mut_prob=0.3, seed=42, al-
loc_rule="best_fit", log_csv=f'{prefix}_gp_fitness_curve.csv')
    else:
        id2node, edges, schedule, copyin_bufs = load_graph(args.json, args.schedule)
        summary = run_best_and_trace(
            id2node, schedule, copyin_bufs,
            pop_size=args.ga_pop, generations=args.ga_gen,
            out_prefix=args.out_prefix,
            use_gp_victim=(not args.no_gp)
        )
        print("\n=== 运行完成 ===")
        print("规则:", summary.get("best_rule"))
        print("GP 规则:", summary.get("gp_rule"))
        print("总额外数据搬运量:", summary.get("total_spill_bytes"))
        print("输出文件:")
        print(f' {args.out_prefix}_address_offsets.json')
        print(f' {args.out_prefix}_spill_ops.json')
        print(f' {args.out_prefix}_augmented_schedule.json')
        print(f' {args.out_prefix}_summary.json')

#GA 关键代码
def genetic_algorithm(nodes, schedule, copyin_bufs,
                      pop_size=24, generations=20,
                      cx_prob=0.8, mut_prob=0.3, elitism=2, seed=42):
    random.seed(seed)
    np.random.seed(seed)
    population = [(random.choice(ALLOC_RULES), random.choice(SPILL_RULES)) for _ in
range(pop_size)]
    def eval_ind(ind):
        alloc_rule, spill_rule = ind
        cost = simulate_total_spill(nodes, schedule, copyin_bufs, alloc_rule, spill_rule)
        fitness = -cost if np.isfinite(cost) else -1e18
        return fitness, cost
    fitnesses, costs = [], []
    for ind in population:
        f, c = eval_ind(ind)
        fitnesses.append(f)
        costs.append(c)

    for gen in range(generations):
        best_idx = int(np.argmax(fitnesses))
        print(f'Gen {gen:02d}: best={population[best_idx]} 搬运量={costs[best_idx]}')
        elite_indices = np.argsort(fitnesses)[-elitism:]

```

```

elites = [population[i] for i in elite_indices]
def tour_select(k=3):
    cand = random.sample(range(pop_size), k)
    cand_best = max(cand, key=lambda i: fitnesses[i])
    return population[cand_best]

new_pop = elites.copy()
while len(new_pop) < pop_size:
    p1 = tour_select()
    p2 = tour_select()
    c1, c2 = p1, p2
    # 交叉
    if random.random() < cx_prob:
        c1 = (p1[0], p2[1])
        c2 = (p2[0], p1[1])
    if random.random() < mut_prob:
        if random.random() < 0.5:
            c1 = (random.choice(ALLOC_RULES), c1[1])
        else:
            c1 = (c1[0], random.choice(SPILL_RULES))
    if random.random() < mut_prob:
        if random.random() < 0.5:
            c2 = (random.choice(ALLOC_RULES), c2[1])
        else:
            c2 = (c2[0], random.choice(SPILL_RULES))
    new_pop.append(c1)
    if len(new_pop) < pop_size:
        new_pop.append(c2)
population = new_pop[:pop_size]
fitnesses, costs = [], []
for ind in population:
    f, c = eval_ind(ind)
    fitnesses.append(f)
    costs.append(c)
best_idx = int(np.argmax(fitnesses))
best_ind = population[best_idx]
best_cost = costs[best_idx]
return best_ind, best_cost

```

问题二基于变领域模拟退火算法的关键求解代码

```

import json
import random
import math
import copy

if __name__ == "__main__":
    json_file = "FlashAttention_Case0.json"
    txt_file = "FlashAttention_Case0_schedule.txt"
    nodes, edges = load_graph(json_file)
    initial_schedule = load_schedule(txt_file)
    if not is_valid_schedule(initial_schedule, edges):

```

```

        initial_schedule = topological_sort(nodes, edges)
    best_schedule, best_data_movement = variable_neighborhood_search(
        initial_schedule, nodes, edges, max_iterations=50, k_max=4
    )
    print(f"优化完成！")
    print(f"初始数据搬运量: {fitness_function(initial_schedule, nodes, edges)}")
    print(f"最优数据搬运量: {best_data_movement}")
    print(f"最优调度序列前 10 个操作: {best_schedule[:10]}")
    with open("optimized_schedule.txt", "w") as f:
        for op_id in best_schedule:
            f.write(f"{op_id}\n")
class CacheSimulator:
    def __init__(self, nodes, edges, schedule):
        self.nodes = nodes
        self.edges = edges
        self.schedule = schedule
        self.cache_state = {}
        self.free_blocks = {}
        self.allocated_blocks = {}
        self.spill_ops = []
        self.total_data_movement = 0
        self.initialize_free_blocks()

    def initialize_free_blocks(self):
        for cache_type, capacity in CACHE_CAPACITY.items():
            self.free_blocks[cache_type] = [(0, capacity)]
            self.allocated_blocks[cache_type] = []

    def allocate_address(self, size, cache_type):
        if cache_type not in self.free_blocks:
            return -1
        free_list = self.free_blocks[cache_type]
        for i, (start, block_size) in enumerate(free_list):
            if block_size >= size:
                new_start = start
                new_size = block_size - size
                if new_size > 0:
                    free_list[i] = (start + size, new_size)
                else:
                    del free_list[i]
                return start
        return -1

    def trigger_spill(self):
        if not self.cache_state:
            return None
        return min(self.cache_state.keys(), key=lambda buf_id: self.get_last_use_time(buf_id))

    def get_last_use_time(self, buf_id):
        last_use = 0
        for node in self.nodes:
            if node['Op'] != 'ALLOC' and 'Bufs' in node and buf_id in node['Bufs']:
                last_use = max(last_use, node['Id'])
        return last_use

```

```

def is_used_by_copy_in(self, buf_id):
    for node in self.nodes:
        if node['Op'] == 'COPY_IN' and 'Bufs' in node and buf_id in node['Bufs']:
            return True
    return False

def simulate(self):
    for op_id in self.schedule:
        node = next(n for n in self.nodes if n['Id'] == op_id)
        if node['Op'] == 'ALLOC':
            buf_id = node['BufId']
            size = node['Size']
            cache_type = node['Type']
            offset = self.allocate_address(size, cache_type)
            if offset == -1:
                max_attempts = 3
                for attempt in range(max_attempts):
                    spill_buf_id = self.trigger_spill()
                    if spill_buf_id is None:
                        break
                    self.handle_spill(spill_buf_id)
                    offset = self.allocate_address(size, cache_type)
                    if offset != -1:
                        break
                if offset == -1:
                    raise Exception(f'分配失败 after SPILL for buffer {buf_id}')
            self.cache_state[buf_id] = (offset, size, cache_type)
            self.allocated_blocks[cache_type].append((offset, size, buf_id))
        elif node['Op'] == 'FREE':
            buf_id = node['BufId']
            if buf_id in self.cache_state:
                offset, size, cache_type = self.cache_state[buf_id]
                self.release_address(offset, size, cache_type)
                del self.cache_state[buf_id]
            self.allocated_blocks[cache_type] = [blk for blk in self.allocated_blocks[cache_type] if
                                                    blk[2] != buf_id]

    return self.total_data_movement

def handle_spill(self, buf_id):
    if buf_id not in self.cache_state:
        return
    offset, size, cache_type = self.cache_state[buf_id]
    used_by_copy_in = self.is_used_by_copy_in(buf_id)
    if not used_by_copy_in:
        self.total_data_movement += 2 * size
    else:
        self.total_data_movement += size
    self.release_address(offset, size, cache_type)
    del self.cache_state[buf_id]
    self.spill_ops.append(buf_id)
    self.allocated_blocks[cache_type] = [blk for blk in self.allocated_blocks[cache_type] if
                                            blk[2] != buf_id]

```

```

def release_address(self, offset, size, cache_type):
    if cache_type not in self.free_blocks:
        return
    free_list = self.free_blocks[cache_type]
    free_list.append((offset, size))
    free_list.sort(key=lambda x: x[0])
    merged = []
    current_start, current_size = free_list[0]
    for i in range(1, len(free_list)):
        start, size_block = free_list[i]
        if current_start + current_size == start:
            current_size += size_block
        else:
            merged.append((current_start, current_size))
            current_start, current_size = start, size_block
    merged.append((current_start, current_size))
    self.free_blocks[cache_type] = merged

```

问题三基于 GP 改进算法的关键求解代码

```

import json
import time
import random
import numpy as np
import csv
import matplotlib.pyplot as plt
from collections import defaultdict

if __name__ == "__main__":
    import os
    import pandas as pd
    cases = [
        ("Conv_Case0.json", "Conv_Case0_schedule.txt", "Conv_Case0_joint"),
        ("Conv_Case1.json", "Conv_Case1_schedule.txt", "Conv_Case1_joint"),
        ("FlashAttention_Case0.json", "FlashAttention_Case0_schedule.txt", "FlashAttention_Case0_joint"),
        ("FlashAttention_Case1.json", "FlashAttention_Case1_schedule.txt", "FlashAttention_Case1_joint"),
        ("Matmul_Case0.json", "Matmul_Case0_schedule.txt", "Matmul_Case0_joint"),
        ("Matmul_Case1.json", "Matmul_Case1_schedule.txt", "Matmul_Case1_joint")
    ]

    for json_file, schedule_file, case_name in cases:
        print(f'\n{'=' * 80}')
        print(f'联合优化案例: {case_name}')
        print(f'{'=' * 80}')

        try:
            optimizer = run_joint_optimization(json_file, schedule_file, case_name)
        except Exception as e:

```

```

        print(f'优化失败: {e}')
        continue

# 生成所有任务的适应度变化曲线
print(f'\n{'=' * 80}')
print("生成所有任务的适应度变化曲线")
print(f'{'=' * 80}')
generate_all_fitness_curves()

def _evolve_gp_joint_optimization(self, fitness_func, pop_size=15, max_depth=2,
                                generations=10, cx_prob=0.7, mut_prob=0.3, seed=42,
                                case_name=""):
    random.seed(seed)
    np.random.seed(seed)
    population = gp_ramped_half_and_half(pop_size, max_depth=max_depth, seed=seed)
    print(f'开始评估初始种群({pop_size}个个体)...')
    fitness = []
    for i, tree in enumerate(population):
        if i % 5 == 0:
            print(f'  评估进度: {i + 1}/{pop_size}')
            fitness.append(fitness_func(tree))
    best_fitness_curve = []
    for gen in range(generations):
        best_fitness_curve.append(min(f for f in fitness if f != float("inf")))
        print(f'第{gen + 1}代: 最佳适应度 = {best_fitness_curve[-1]:.6f}')
        if gen >= 3 and len(set(best_fitness_curve[-4:])) == 1:
            print(f'早停: 连续 3 代无改善, 在第{gen + 1}代结束")
            break
    # 选择 (锦标赛选择)
    def tournament_select(k=3):
        candidates = random.sample(range(len(population)), k)
        best_candidate = min(candidates, key=lambda i: fitness[i])
        return population[best_candidate]
    # 精英保留
    elite_indices = np.argsort(fitness)[:2] # 保留前 2 个
    elites = [population[i] for i in elite_indices]
    # 生成新种群
    new_population = elites.copy()
    while len(new_population) < pop_size:
        parent1 = tournament_select()
        parent2 = tournament_select()
        # 交叉
        if random.random() < cx_prob:
            child = gp_crossover(parent1, parent2)
        else:
            child = gp_copy_tree(parent1)
        # 变异
        if random.random() < mut_prob:
            child = gp_mutate(child, max_depth)
        new_population.append(child)

    population = new_population[:pop_size]

```

问题三基于 NAGA-II 算法的关键求解代码

```
import json
import time
import random
import numpy as np
import matplotlib.pyplot as plt
import csv
from collections import defaultdict, deque

if __name__ == "__main__":
    # 运行所有六个任务
    tasks = [
        ("Conv_Case0.json", "Conv_Case0_schedule.txt", "Conv_Case0"),
        ("Conv_Case1.json", "Conv_Case1_schedule.txt", "Conv_Case1"),
        ("FlashAttention_Case0.json", "FlashAttention_Case0_schedule.txt", "FlashAttention_Case0"),
        ("FlashAttention_Case1.json", "FlashAttention_Case1_schedule.txt", "FlashAttention_Case1"),
        ("Matmul_Case0.json", "Matmul_Case0_schedule.txt", "Matmul_Case0"),
        ("Matmul_Case1.json", "Matmul_Case1_schedule.txt", "Matmul_Case1")
    ]

    print("=" * 80)
    print("NSGA-II 多目标优化 - 运行所有六个任务")
    print("节点数超过 10000 使用 fast 版本, 否则使用标准版本")
    print("固定迭代 50 代, 输出最后一代帕累托解")
    print("=" * 80)

    for i, (json_file, schedule_file, case_name) in enumerate(tasks, 1):
        print(f'\n{'=' * 80}')
        print(f'任务 {i}/6: {case_name}')
        print(f'\n{'=' * 80}')

        try:
            # 先检查节点数量
            from 遗传编程 import load_graph

            nodes, edges, schedule, copyin_bufs = load_graph(json_file, schedule_file)
            node_count = len(nodes)

            print(f'节点数: {node_count}')

            if node_count > 10000:
                print("使用 fast 版本优化 (采样策略)")
                from nsga2_optimization_fast import run_fast_nsga2_optimization

                results = run_fast_nsga2_optimization(json_file, schedule_file, case_name)
            else:
                print("使用标准版本优化")
                results = run_nsga2_optimization(json_file, schedule_file, case_name)
```

```

        print(f'\n 任务 {case_name} 优化完成! ")
        print(f'最后一代帕累托解数量: {len(results['pareto_solutions'])}')")

    except Exception as e:
        print(f'任务 {case_name} 优化失败: {e}')
        import traceback

        traceback.print_exc()
        continue

    print(f'\n{'=' * 80}')
    print("所有任务完成! 结果已保存到 'NSGA 结果' 文件夹")
    print(f'\n{'=' * 80}')

class NSGA2Optimizer:
    def __init__(self, nodes, original_schedule, copyin_bufs):
        self.nodes = nodes
        self.original_schedule = original_schedule
        self.copyin_bufs = copyin_bufs
        self.schedule_length = len(original_schedule)
        self.cache_capacity = CACHE_CAPACITY
        self.alloc_strategies = ["first_fit", "best_fit", "worst_fit"]
        self.execution_units = [f"MTE{i}" for i in range(1, 101)]

    def load_existing_solution(self, case_name):
        try:
            schedule_file = f'{case_name}_schedule.txt'
            with open(schedule_file, 'r', encoding='utf-8') as f:
                existing_schedule = [int(line.strip()) for line in f if line.strip()]
            memory_file = f'{case_name}_memory.txt'
            memory_allocations = {}
            with open(memory_file, 'r', encoding='utf-8') as f:
                for line in f:
                    if ':' in line and not line.startswith('#'):
                        buf_id, offset = line.strip().split(':')
                        memory_allocations[buf_id] = int(offset)
            spill_file = f'{case_name}_spill.txt'
            spill_operations = []
            with open(spill_file, 'r', encoding='utf-8') as f:
                for line in f:
                    if ':' in line and not line.startswith('#'):
                        buf_id, new_offset = line.strip().split(':')
                        spill_operations.append((buf_id, int(new_offset)))
            alloc_strategy = "best_fit"
            gp_tree = self._generate_solution_based_gp_tree(spill_operations)
            print(
                f'成功加载现有解决方案: {len(existing_schedule)}个节点, {len(memory_allocations)}个分配, {len(spill_operations)}个 SPILL 操作")
            return self.encode_individual(existing_schedule, alloc_strategy, gp_tree)

        except Exception as e:

```

```

        print(f"加载现有解决方案失败: {e}")
        return None

def _generate_solution_based_gp_tree(self, spill_operations):
    return self._generate_random_gp_tree()

def _generate_random_gp_tree(self, max_depth=3):
    population = gp_ramped_half_and_half(1, max_depth=max_depth, seed=random.randint(0,
10000))
    return population[0] if population else None

def evaluate_objectives(self, individual):
    schedule, alloc_strategy, gp_tree = self.decode_individual(individual)
    spill_cost = simulate_total_spill(
        self.nodes, schedule, self.copyin_bufs,
        alloc_strategy, "gp", gp_tree
    )
    if spill_cost == float("inf"):
        return float("inf"), float("inf")
    exec_time = self._calculate_execution_time(schedule, alloc_strategy, gp_tree)
    if exec_time == float("inf"):
        return float("inf"), float("inf")
    return spill_cost, exec_time

def _calculate_execution_time(self, schedule, alloc_strategy, gp_tree):
    total_spill, first_offsets, spill_ops, aug_schedule = simulate_with_trace(
        self.nodes, schedule, self.copyin_bufs, alloc_strategy, "gp", gp_tree
    )
    if total_spill == float("inf"):
        return float("inf")
    base_time = self._calculate_base_execution_time(aug_schedule)
    spill_penalty = self._calculate_spill_penalty(spill_ops)
    return base_time + spill_penalty

def _calculate_base_execution_time(self, aug_schedule):
    execution_units = {}
    for unit in self.execution_units:
        execution_units[unit] = {"busy_until": 0, "current_node": None}
    for step, node in enumerate(aug_schedule):
        if isinstance(node, dict):
            node_id = node.get("Id", f"aug_{step}")
            op = node.get("Op", "")
            pipe = node.get("Pipe", "")
            cycles = self._get_node_cycles(node)
            if cycles > 0 and pipe in execution_units:
                start_time = execution_units[pipe]["busy_until"]
                execution_units[pipe]["busy_until"] = start_time + cycles
                execution_units[pipe]["current_node"] = node_id
    max_time = max(unit["busy_until"] for unit in execution_units.values())
    return max_time

```

问题三基于 MOPSO 算法的关键求解代码

```
import json
import time
import random
import matplotlib.pyplot as plt
import csv
from collections import defaultdict, deque

if __name__ == "__main__":
    # 运行所有六个任务
    tasks = [
        ("Conv_Case0.json", "Conv_Case0_schedule.txt", "Conv_Case0"),
        ("Conv_Case1.json", "Conv_Case1_schedule.txt", "Conv_Case1"),
        ("FlashAttention_Case0.json", "FlashAttention_Case0_schedule.txt", "FlashAttention_Case0"),
        ("FlashAttention_Case1.json", "FlashAttention_Case1_schedule.txt", "FlashAttention_Case1"),
        ("Matmul_Case0.json", "Matmul_Case0_schedule.txt", "Matmul_Case0"),
        ("Matmul_Case1.json", "Matmul_Case1_schedule.txt", "Matmul_Case1")
    ]

    print("=" * 80)
    print("MOPSO 多目标优化 - 运行所有六个任务")
    print("节点数超过 10000 使用 fast 版本, 否则使用标准版本")
    print("固定迭代 50 代, 输出最后一代帕累托解")
    print("=" * 80)

    for i, (json_file, schedule_file, case_name) in enumerate(tasks, 1):
        print(f"\n{'=' * 80}")
        print(f"任务 {i}/6: {case_name}")
        print(f"{'=' * 80}")

        try:
            # 先检查节点数量
            from 遗传编程 import load_graph

            nodes, edges, schedule, copyin_bufs = load_graph(json_file, schedule_file)
            node_count = len(nodes)

            print(f"节点数: {node_count}")

            if node_count > 10000:
                print("使用 fast 版本优化 (采样策略)")
                # 这里可以调用 fast 版本, 暂时使用标准版本
                results = run_mopso_optimization(json_file, schedule_file, case_name)
            else:
                print("使用标准版本优化")
                results = run_mopso_optimization(json_file, schedule_file, case_name)

            print(f"\n任务 {case_name} 优化完成!")
            print(f"最后一代帕累托解数量: {len(results['pareto_solutions'])}")
```

```

except Exception as e:
    print(f'任务 {case_name} 优化失败: {e}')
    import traceback

    traceback.print_exc()

class MOPSOOptimizer:
    def __init__(self, nodes, original_schedule, copyin_bufs):
        self.nodes = nodes
        self.original_schedule = original_schedule
        self.copyin_bufs = copyin_bufs
        self.schedule_length = len(original_schedule)
        self.cache_capacity = CACHE_CAPACITY
        self.alloc_strategies = ["first_fit", "best_fit", "worst_fit"]
        self.execution_units = [f'MTE{i}' for i in range(1, 101)]
        self.external_archive = []

    def encode_individual(self, schedule_order, alloc_strategy, spill_gp_tree):
        return {
            'schedule': schedule_order,
            'alloc_strategy': alloc_strategy,
            'gp_tree': spill_gp_tree
        }

    def decode_individual(self, individual):
        return individual['schedule'], individual['alloc_strategy'], individual['gp_tree']

    def generate_random_individual(self):
        schedule = self.original_schedule.copy()
        random.shuffle(schedule)
        alloc_strategy = random.choice(self.alloc_strategies)
        gp_tree = self._generate_random_gp_tree()
        return self.encode_individual(schedule, alloc_strategy, gp_tree)

    def load_existing_solution(self, case_name):
        try:
            schedule_file = f'{case_name}_schedule.txt'
            with open(schedule_file, 'r', encoding='utf-8') as f:
                existing_schedule = [int(line.strip()) for line in f if line.strip()]
            memory_file = f'{case_name}_memory.txt'
            memory_allocations = {}
            with open(memory_file, 'r', encoding='utf-8') as f:
                for line in f:
                    if ':' in line and not line.startswith('#'):
                        buf_id, offset = line.strip().split(':')
                        memory_allocations[buf_id] = int(offset)
            spill_file = f'{case_name}_spill.txt'
            spill_operations = []
            with open(spill_file, 'r', encoding='utf-8') as f:
                for line in f:
                    if ':' in line and not line.startswith('#'):
                        buf_id, new_offset = line.strip().split(':')
                        spill_operations.append((buf_id, int(new_offset)))

```

```

        alloc_strategy = "best_fit"
        gp_tree = self._generate_solution_based_gp_tree(spill_operations)
        print(
            f"成功加载现有解决方案: {len(existing_schedule)}个节点, {len(memory_allocations)}个分配, {len(spill_operations)}个 SPILL 操作")
        return self.encode_individual(existing_schedule, alloc_strategy, gp_tree)

    except Exception as e:
        print(f"加载现有解决方案失败: {e}")
        return None

    def _generate_solution_based_gp_tree(self, spill_operations):
        return self._generate_random_gp_tree()

    def _generate_random_gp_tree(self, max_depth=3):
        population = gp_ramped_half_and_half(1, max_depth=max_depth, seed=random.randint(0, 10000))
        return population[0] if population else None

```